

Универсальное пособие • полный исходный код

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 13.09.2026, 16:56:31
Файлов исходного кода: 92
Суммарный объём кода: 1.38 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planar-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

.....

- 24. src/data/vizStepBus.ts

- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_5.ts

- 27. src/data/tickets/ticket14_20.ts

- 28. src/data/tickets/ticket21_24.ts

- 29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 30. src/components/ArticulationPointsViz.tsx

- 31. src/components/BellmanFordViz.tsx

- 32. src/components/BfsViz.tsx

- 33. src/components/ChapterImage.tsx

- 34. src/components/ChapterNav.tsx

- 35. src/components/ChapterView.tsx

- 36. src/components/ColoringViz.tsx

- 37. src/components/ComplexityClassesViz.tsx

- 38. src/components/ComponentsViz.tsx

- 39. src/components/DfsBridgesSimulator.tsx

- 40. src/components/DijkstraViz.tsx

- 41. src/components/DPVisualizer.tsx

- 42. src/components/EulerSimulator.tsx

- 43. src/components/ExpressQuiz.tsx

- 44. src/components/FloydViz.tsx

- 45. src/components/GraphTraversalViz.tsx

- 46. src/components/HeapViz.tsx

- 47. src/components/hints/demoKit.tsx

- 48. src/components/hints/demosExtra.tsx

- 49. src/components/hints/demosGraph.tsx

- 50. src/components/hints/demosStruct.tsx

- 51. src/components/hints/MiniDemo.tsx

- 52. src/components/hints/SimulatorModal.tsx

- 53. src/components/hints/TermPopover.tsx

- 54. src/components/JohnsonViz.tsx

- 55. src/components/KosarajuViz.tsx

- 56. src/components/KruskalSimulator.tsx

- 57. src/components/MnemonicCards.tsx

Универсальное пособие • полный исходный код

```
.....
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
```

CI / автоматизация

```
91. .github/workflows/deploy-pages.yml
92. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

```
.....
ФАЙЛ 1/92: add.md
```

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml `

Полное содержимое после изменений

```yaml

## Универсальное пособие • полный исходный код

```
.....
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
.....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
```

Универсальное пособие • полный исходный код

```
+
+   - name: Upload Pages artifact
+     uses: actions/upload-pages-artifact@v5
+     with:
+       path: ./dist
+
+ deploy:
+   name: Publish to GitHub Pages
+   needs: build
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+
+ )
```

node-version: "24"

cache: none

- ```
- name: Install ImageMagick + DejaVu fonts
```

run: 1

```
sudo apt-get update -qq
```

```
sudo apt-get install -y --no-install-recommen
```

```
На Ubuntu политика ImageMagick иногда запре
```

```
разрешаем то, что нужно пайплайну (text ->
```

```
for policy in /etc/ImageMagick-6/policy.xml /
```

```
if [-f "$policy"]; then
```

```
sudo sed -i 's/rights="none" pattern="\ (P
```

fi

done

```
convert -version
```

- ```
- name: Install dependencies
```

```
run: npm ci --no-audit --no-fund
```

- name: Step 1 - код → txt

```
run: npm run export:txt
```

- ```
- name: Step 2 - txt → png
```

env:

```
EXPORT DENSITY: ${github.event.inputs.density}
```

```
run: npm run export:png
```

- name: Step 3 - png → pdf

```
run: npm run export:pdf
```

- name: Type-check приложения

```
run: npx tsc --noEmit
```

```
continue-on-error: true
```

- name: Summary

run: 1

4

```
echo "### Экспорт пересобран"
```

#####  
### Patch относительно `main`

```diff

diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

Универсальное пособие • полный исходный код

```
.....
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
+    <link rel="icon" type="image/svg+xml" href="%BASE_
    <title> Дискретка для тупых – Интерактивный гид п
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

`public/favicon.svg`

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

diff --git a/public/favicon.svg b/public/favicon.svg

new file mode 100644

index 00000000..3bbbbba0

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 6

+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>

+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#

+ <circle cx="47" cy="21" r="4" fill="#34d399"/>

+</svg>

````

`src/App.tsx`


```

.....
    if (e.key === "ArrowRight" && next) goTo(next.id)
    if (e.key === "ArrowLeft" && prev) goTo(prev.id);
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown",
}, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />

        <div className="flex flex-col lg:flex-row max-w-7xl" >
          {/* Сайдбар только на десктопе – на мобильном скрывается */}
          <div className="hidden lg:block lg:w-80 shrink-0" >
            <Sidebar selectedChapterId={activeChapterId}>
          </div>

          <main id="main-content" className="flex-1 min-w-0" >
            {/* Шапка главы с быстрыми переходами */}
            <div className="flex items-center justify-between" >
              <div className="min-w-0">
                <div className="text-[10px] uppercase" >
                  {activeChapter.category || "Универсальное"}
                </div>
                <h2 className="text-base sm:text-xl font-medium" >
                  {activeChapter.title}

```

```
.....

    <SimulatorModal vizId={modalVizId} onClose={() =>

    <footer className="p-8 text-center text-slate-600"
      © 2026 Universal Educational Guide. Интерактивный
    </footer>
  </div>
);
}
....
```

Patch относительно `main`

```
````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator">"guide";
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">"guide";
 const [activeChapterId, setActiveChapterId] = useState<string>("");
 const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
```

```

.....

return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <CodeIcon className="w-4 h-4" /> Симулятор
 </button>
 </div>
 </div>
)

```

## Универсальное пособие • полный исходный код

```
.....
 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
</div>
</div>
</header>
);
};
},...
```

### Patch относительно `main`

```
```diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
```

Универсальное пособие • полный исходный код

```
.....
        rel="noreferrer"
....

## `src/components/PythonCompiler.tsx`

### Полное содержимое после изменений

```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL);
 }
}
```

```

.....
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не вывела");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код – подробности в консоли");
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5

```

```

.....
 onKeyDown= {(event) => {
 if ((event.ctrlKey || event.metaKey) &&
 event.preventDefault());
 void runCode();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] res
- none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код.."
 />

 <div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
 </div>
 </section>

 <section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

```

## Универсальное пособие • полный исходный код

.....  
@@ -0,0 +1,205 @@

```
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>;
+ setStdout: (options: { batched: (message: string) => void }) => void;
+ setStderr: (options: { batched: (message: string) => void }) => void;
+ setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
+ (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL })
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) {
+ return error.message;
+ }
+ return String(error);
+}
```



```

.....
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.
+ >
+ Как это работает <ExternalLink className="
+
+ </div>
+
+ <div className="grid xl:grid-cols-2 {minmax(0,1.
+ <section className="rounded-2xl border borde

```

## Универсальное пособие • полный исходный код

```
.....
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код.."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-i
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emera
+ </div>
+ <span className={`inline-flex items-cent
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800
```

Универсальное пособие • полный исходный код

```
.....
// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
....
```

### Patch относительно `main`

```
````diff  
diff --git a/vite.config.ts b/vite.config.ts  
index c0456e7..e19d945 100644  
--- a/vite.config.ts  
+++ b/vite.config.ts  
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)  
  
// https://vite.dev/config/  
export default defineConfig({
```

Универсальное пособие • полный исходный код

.....

****строго**** придерживаться готовых шаблонов разметки,

Правила для малых страниц (Билетов / Глав)

1. ****Единый стиль:**** Всегда используй Tailwind CSS и HTML
2. ****Структура JSON/TS:**** Каждая новая малая страница должна иметь структуру `{ "id": ..., "content": ... }` для вставки в ``src/data/content.ts``.
3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
 - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``)
 - Обязательная сетка аналогий (2 колонки: неформальный текст и формальный текст)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и не используй классы в файлах ``src/data/content.ts``. Не используй чистый markdown.

.....

ФАЙЛ 4/92: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

.....

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Универсальное пособие по алгоритмам – билет №4
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

.....

ФАЙЛ 5/92: metadata.json

Универсальное пособие • полный исходный код

```
.....  
  "motion": "^12.40.0",  
  "pdfkit": "^0.20.1",  
  "react": "19.2.6",  
  "react-dom": "19.2.6",  
  "svg-to-pdfkit": "^0.1.8",  
  "tailwind-merge": "3.4.0"  
},  
"devDependencies": {  
  "@tailwindcss/vite": "4.1.17",  
  "@types/node": "22.19.17",  
  "@types/pdfkit": "^0.17.6",  
  "@types/react": "19.2.7",  
  "@types/react-dom": "19.2.3",  
  "@vitejs/plugin-react": "5.1.1",  
  "esbuild": "^0.24.0",  
  "tailwindcss": "4.1.17",  
  "typescript": "5.9.3",  
  "vite": "7.3.2",  
  "vite-plugin-singlefile": "2.3.0"  
},  
"description": "Интерактивное пособие по алгоритмам и  
}
```

.....
ФАЙЛ 7/92: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 125 | РАЗМЕР: 1.1 КБ

.....
algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с симуляцией кода
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
```

## ## Аудит контента

```
```bash
```

```
node scripts/audit-coverage.mjs          # какие темы без
```

```
node scripts/audit-coverage.mjs --md     # markdown-табли
```

```
```
```

Актуальный отчёт: [ `TOPICS\_AUDIT.md` ](./TOPICS\_AUDIT.md)

## ## Как устроен интерфейс

- \* **Темы 1–24** — канонический список билетов; одна строка в файле ``src/data/content.ts``, таблица ``CANON`` задаёт точные названия тем.
- \* **Python-компилятор** — боковая панель (кнопка «Python-компилятор») и отладчик. Список «переменных темы» для каждой страницы в файле ``src/data/vizSync.ts``; значения из выполняемого Python-кода — объекты демонстрации (матрицы, графы, деревья) напрямую в код, инициализация подставляет заранее заданные значения.
- \* **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки.
- \* **Переходы между темами** — кнопки «Назад / Вперёд» в боковой панели, «Предыдущая / Следующая тема» под текстом; на клавиатуре — стрелки.
- \* **Всплывающие подсказки по терминам** — текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта термина мини-анимацией и кнопкой «Показать симулятор» (симуляторы — ``src/data/simulators.ts``, словарь — ``src/data/glossary.ts``, мини-анимации — ``src/data/animations.ts``).
- \* **Реестр интерактивов** — ``src/components/vizRegistry.ts`` и для панели под главой, и для модалки из подсказки.

## ## Структура

```
...
```

```
src/
```

```
 App.tsx — оболочка, привязка глав к визуализации
```

```
 components/ — интерактивные симуляторы (Действия, Симуляторы)
```

```
 data/ — контент пособия (главы/билеты/карты/анимации)
```

Универсальное пособие • полный исходный код

.....

Экспорт перехватывает основные структурные компоненты HTML:

1. Заголовки `

## ` и `` заменяются на `##` и `###`

2. Элементы ``.analogy-badge` получают выделение маркера`

3. Теги ``<svg>`` заменяются строкой-заглушкой ``{ Иллюстрация}`.

4. Элементы списка ``<li>`` получают маркер ``-``.

5. Результат извлекается в виде текста и чистится от лишнего

.....

ФАЙЛ 8/92: TOPICS\_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

.....

# Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.mjs`

> Критерии:

> \* **Аналогия** — в HTML главы есть блок «Аналогия ...»

> \* **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

---

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |

| --- | --- | --- |

| Алгоритмы на карте | `alg-map` | Страница-заглушка (8)

---

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны,

но соответствующих глав в `src/data/content.ts` нет — и

## Универсальное пособие • полный исходный код

```

.....
| 3. Декартово дерево (Tgear) | `tgear` | Нет ни визуализации |
|
| 8. Графы. Компоненты связности | `graph-components` |
| 24. Классы сложности, сведение задач | `complexity-classes` |

```

### ## Сводная таблица по всем главам

| #  | Глава                                           | Аналогий | 2D-виз. | SVG | Картинка |
|----|-------------------------------------------------|----------|---------|-----|----------|
| 2  | 2. Двумерная разреженная матрица (Sparse Table) |          |         |     |          |
| 3  | 3. Декартово дерево (Tgear)                     | 2        | -       | -   | -        |
| 4  | 4. Splay-дерево                                 | 2        |         | -   | -        |
| 5  | 5. Двумерная матрица префиксных сумм            | 1        |         |     | -        |
| -  | Планарность: K5, K3,3 и формула Эйлера          | -        |         |     | -        |
| 6  | 6. Графы. Представление, DFS, BFS               | 4        |         | -   | -        |
| 8  | 8. Графы. Компоненты связности                  | 1        | -       | -   | -        |
| 9  | 9. Графы. Топологическая сортировка             | 1        |         |     | -        |
| 10 | 10. Графы. Компоненты сильной связности (SCC)   |          |         |     | -        |
| 12 | 12. Графы. Точки сочленения                     | 1        |         | -   | -        |
| 14 | 14. Графы. Кратчайший путь. Дейкстра            | 3        |         |     | -        |
| 15 | 15. Графы. Кратчайший путь. Форд-Беллман        | 1        |         |     | -        |
| 16 | 16. Графы. Кратчайший путь. Флойд               | -        |         |     | -        |
| 17 | 17. Графы. Алгоритм Джонсона                    | -        |         | -   | -        |
| 18 | 18. Графы. Остовное дерево. Краскал             | 1        |         |     | -        |
| 19 | 19. Графы. Остовное дерево. Прима               | 1        |         |     | -        |
| 20 | 20. Графы. Остовное дерево. Борувак             | 1        |         |     | -        |
| 21 | 21. Префикс-функция ( $\pi$ ), КМП              | 4        |         | -   | -        |
| 22 | 22. Z-функция                                   | 4        |         | -   | -        |
| 23 | 23. Алгоритм Ахо-Корасик                        | 2        |         | -   | -        |
| 24 | 24. Классы сложности, сведение задач            | 4        | -       |     | -        |
| -  | Обзор: Кратчайшие пути и Графы                  | -        |         | -   | -        |
| -  | Алгоритмы на карте                              | -        | -       | -   | -        |
| -  | Эйлер: Путь ≠ Цикл                              | -        |         | -   | -        |
| -  | Мосты в графах: код + визуализация              | 1        |         |     | -        |



```
.....

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

.....  
ФАЙЛ 10/92: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

```
.....

import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algov0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
```

```

.....
/** Боковая (на мобильном – нижняя) панель компилятора
const [compilerOpen, setCompilerOpen] = useState(false);
/** Ширина панели компилятора (desktop), растягивается
const [compilerWidth, setCompilerWidth] = useState<number>(
 const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerWidth")) : null;
 return Number.isFinite(cached) && cached >= 380 ? cached : 380;
));
useEffect(() => {
 window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту
const [compilerHeight, setCompilerHeight] = useState<number>(
 const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerHeight")) : null;
 return Number.isFinite(cached) && cached >= 260 ? cached : 260;
));
useEffect(() => {
 if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем сам компилятор)
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>(
 const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null;
 if (cached !== null) return cached === "1";
 return typeof window !== "undefined" && window.innerWidth < 380;
));
const toggleSidebar = useCallback(() => {
 setSidebarCollapsed((c) => {
 window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
 return !c;
 });
}, []);

// Desktopный отступ контента = ширине панели компилятора
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.innerWidth < 380 ? compilerWidth : 0;
 };
 apply();
}, [compilerOpen, compilerWidth]);

```

```

.....
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
}, {goTo, next, prev});

const progress = useMemo(() => ((index + 1) / chapters

return (
 <div className="min-h-screen bg-slate-950 text-slate-
 <Navbar
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) =>
 />

 {/* При открытой панели контент сдвигается: снизу
 <div ref={contentRef} className={compilerOpen ? "
 <
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row m
 {/* Сайдбар только на десктопе — на мобил
 <div
 className={`hidden lg:block shrink-0 bo
 tion-all duration-200 ${sidebarCollapsed ?
 >
 {!sidebarCollapsed && <Sidebar selected
 </div>
 {/* Вкладка-переключатель содержания у ле
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center
 l-0 border-slate-700 bg-slate-900/90 px-1 p

```

```

 />
 </main>
 </div>
 </>

 <footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное
 </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 {compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 width={compilerWidth}
 height={compilerHeight}
 onWidthChange={setCompilerWidth}
 onHeightChange={setCompilerHeight}
 onOpenGuide={() => {
 document.getElementById("chapter-viz")?.scrollIntoView()
 }}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>
);
}

```

ФАЙЛ 12/92: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 KB

```

import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

```

```

.....
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0)
 return NodeFilter.FILTER_ACCEPT;
 },
});

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
}

for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE)
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createTextNode(
 const span = document.createElement("span");

```

```
.....
 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

.....  
ФАЙЛ 13/92: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
.....

@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
 .no-scrollbar::-webkit-scrollbar {
 display: none;
 }
 .no-scrollbar {
 scrollbar-width: none;
 }
}

@keyframes pulse-gold {
```

```
.....
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
```

```
.....
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```



```
.....

declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

.....  
ФАЙЛ 17/92: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 817 | РАЗМ:

```
.....

import { Chapter } from "../types";

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных",
 category: "Продвинутые структуры",
 content: `
```

```

.....
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px; margin: 10px 0;">
 <pre class="bg-slate-950 p-4 rounded" style="font-family: monospace; font-size: 0.9em; margin: 0;">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
 </div>
</div>
</section>',
 }, {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 3
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Декартово дерево (Treap)</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Описание</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Декартово дерево (Treap) – это бинарное дерево поиска, в котором каждый узел содержит пару (ключ, приоритет). Узлы упорядочены по ключу в левом поддереве и по приоритету в правом поддереве. Это позволяет выполнять операции вставки, удаления и поиска за $O(\log n)$ времени. В данном контексте оно используется для построения двоичной кучи. Пара – точка на декартовой плоскости.
 </div>
 <p class="text-slate-300 text-sm mb-4">Построение происходит за $O(n \log n)$ времени. Алгоритм работает так: для каждой точки (x, y) мы вставляем ее в дерево. При этом мы гарантируем, что дерево остается AVL-деревом, выполняя повороты. В итоге мы получаем бинарное дерево, которое является двоичной кучей.
 «соединить сам» – кликаешь две точки,

```

```
.....
 быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «Уб
аются merge за $O(h)$.</p>
</div>

<p class="text-slate-300 text-sm mt-4"><b c
"bg-slate-950 px-2 py-0.5 rounded text-xs f
оение – <code class="bg-slate-950 px-2 py-0
code> (сортировка + n спусков). Худший случ
text-xs font-mono text-rose-300 border bord
ed text-xs font-mono text-rose-300 border b

<p class="text-slate-500 text-xs mt-3 leading
и останется набор букв, как treap без y оста
но кричат по «!»: корень самый громкий, род
; равные x – идут влево по уставу.</p>

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: split (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
</div>
```

```

{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Самонастраивающееся дерево: поднимаем",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ротов поднимает x в корень. Единственный ин
 </div>

 <div class="grid grid-cols-1 md:grid-cols-3">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">мож
 <p class="text-slate-400 text-xs mt">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-emerald-300 font-bold">
 <p class="text-slate-400 text-xs mt">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">0{n
 <p class="text-slate-400 text-xs mt">
 </div>
 </div>
 </div>
 </div>
`

```

.....  
 омент «отстёжки», а врезка «Механика одного

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Смотря
<div class="overflow-x-auto -mx-2 px-2">
 <table class="w-full text-xs sm:text-sm"
 <thead>
 <tr class="text-left text-slate-300">
 <th class="py-2 pr-3 font-bold">
 <th class="py-2 pr-3 font-bold">
 <th class="py-2 pr-3 font-bold">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-300">
 <td class="py-3 pr-3 font-bold">
 <td class="py-3 pr-3">деда
 <td class="py-3 pr-3">сам х
 <td class="py-3">х – корень
 </tr>
 <tr class="border-b border-slate-300">
 <td class="py-3 pr-3 font-bold">
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3"><span
 <td class="py-3">ветка укор
 </tr>
 <tr>
 <td class="py-3 pr-3 font-bold">
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3"><span
 <td class="py-3">р и г стал
 </tr>
 </tbody>
 </table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>
```

```

.....
 мума левой половины не будет правого сына —
 а под «горячие» ключи, дешёвые split/merge.
 во меняется даже при чтении (плохо для много

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: rotate + splay + find (Скрыт
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

узел: { key, left, right, parent }

def rotate_up(x): # x встаёт на место
 p = x.parent
 g = p.parent
 if p.left is x: # правый поворот
 p.left = x.right
 if x.right: x.right.parent = p
 x.right = p
 else: # левый (зеркально)
 p.right = x.left
 if x.left: x.left.parent = p
 x.left = p
 p.parent = x
 x.parent = g
 if g:
 if g.left is p: g.left = x
 else: g.right = x

def splay(x): # поднять x в корень
 while x.parent:
 p, g = x.parent, x.parent.parent
 if g is None: # ZIG
 rotate_up(x)
 elif (g.left is p) == (p.left is x): # ZIG-ZIG
 rotate_up(p)
 rotate_up(x)

```

```

.....
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb-1">
з него маленький целевой прямоугольник в цел
ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-1"
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```
.....
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-1">
 еская сортировка – это расписание, которое
 <p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-1">
 получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
</div>
</section>',
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 нвертированному в порядке top-sort.</p>
 </div>
 </div>
 </div>
</section>`
 },
}
```



```

.....
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb
этот мост, экономикой обоих кусков придет
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-rose-300 italic
 <p class="text-xl font-mono text-white">
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>

```

```

 </div>
 </div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">17. Графы. Ускорения поиска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Вспомогательная таблица
 <p class="text-xl font-mono text-white">
 d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под землей.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Если копать туннель по прямой, то придется копать
 в 2 РАЗА меньше работы (геометрически: площадь
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

.....
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",

```

```

.....
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
i}), который одновременно является её суффикс
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4"
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb-1">
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300"
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;

```

```
.....
 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
 </div>
</div>
</section>',
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-40
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-purple-300 itali
 <p class="text-xl font-mono text-white"
Сведение (Reduction) A->B: Если я умею реша
```

```

.....
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Эйлер
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-4
 <p class="text-lg text-blue-300 italic
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
 </div>
 <div>
 <p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font

```

```

.....
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400">Теорема Эйлера
 <p>
 Связный граф содержит эйлеров цикл $\Leftrightarrow$
 Эйлеров путь (не цикл) $\Leftrightarrow$ ровно 2 вершины с нечётной степенью
 </p>
 <p class="text-rose-400">Почему Гамильтон?
 <p>
 Задача коммивояжёра (TSP) – это взвешенный вариант задачи о гамильтоновом цикле.
 TSP – NP-трудная: полиномиальный алгоритм не найден.
 В общем, не парься. Просто запомни.
 </p>
 </div>
 </details>
</div>
</section>',
},
{
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 space-y-4">
 <div class="flex items-center mb-6">
 Код
 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">Задача коммивояжёра (TSP)
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Вспомогательная информация:
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin{v} – минимальное количество вершин,
 <p><strong class="text-white">low{v} – минимальное количество рёбер,
 <div class="p-3 bg-slate-950 rounded">

```

```

 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Полный код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space
 <p>Классическая реализация – <code class="
 оходов.</p>

 <div class="bg-slate-950 p-4 rounded-lg
 <pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем :

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 }
 }
}

```



```
.....
<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400

 <div class="bg-slate-900 p-4 rounded-lg mb-4
 <p class="text-lg text-blue-300 italic
 <p class="text-3xl font-mono text-white
 <div class="text-sm text-slate-400 space
 <p><span class="text-blue-400 font-
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
 order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
 -white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 – эталон н
 Если внутри графа спрятан K5 – забудь
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
 der-rose-500 shadow-md">
```

ают</p>

</div>

<div class="grid grid-cols-1 xl:grid-cols-2

<div class="bg-slate-800 p-6 rounded-lg

<div class="absolute -top-3 left-4 i  
rder border-fuchsia-500 shadow-md">

Аналогия: карта на стене

</div>

<p class="text-slate-300 text-sm mb

Красим карту регионов: соседние

Регион = вершина, общая граница

Для плоской карты (планарный граф)

теорема о четырёх красках (Аппеля)

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg

<div class="absolute -top-3 left-4 i  
er border-amber-500 shadow-md">

Что нужно знать на экзамен

</div>

<ul class="text-slate-300 text-sm sp

<li><strong class="text-white">Всего  
вободный у соседей. Дал  $\leq \Delta + 1$  цветов ( $\Delta$  – макс.

<li><strong class="text-white">Достаточно  
тит 2 цветов, хоть он и непланарен.</li>

<li><strong class="text-white">Хроматический  
=  $n$ ,  $\chi$ (цикла чётной длины) = 2,  $\chi$ (цикла нечётной

<li><strong class="text-white">Хроматический  
й и не нечётный цикл,  $\chi(G) \leq \Delta$ .</li>

</ul>

</div>

</div>

<p class="text-slate-400 text-xs mt-4">

Почему раскраска «сложная»: проверить раскраску  
- полная задача.

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком. Если перекресток исчезнет, то система будет хрупкой (удалят вершину) – районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной сети, соединенных в звезду (мостом). Сами свитчи – это точки со

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
```

Душный мод: Код и корень DFS (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Особый
```

```
<p>
```

Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для неё у него **больше 1** независимого ребенка.

Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая `<strong>` и есть критическая SCC! Если удалить эту `<strong>`. Одно слабое звено изолирует целые касты `<strong>`

`</u>`

`</div>`

`</div>`

`</section>`,  
},  
;

ФАЙЛ 19/92: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 86 | РАЗМЕР: 1000x1000

```
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets1to5 } from "../tickets/ticket1_5";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as extraChapters } from "../content_tem

/**
 * Канонический порядок тем (билеты 1–24) и «правильные»
 * одна страница может раскрывать одну тему или несколько
 * поэтому названия выверяются по единому списку ниже.
 */
const CANON: Record<string, { num: number; title: string }> = {
 "segment-trees": { num: 1, title: "1. Дерево отрезков" },
 "sparse-table": { num: 2, title: "2. Двумерная разреженная таблица" },
 "treap": { num: 3, title: "3. Декартово дерево (Treap)" },
 "splay-tree": { num: 4, title: "4. Splay-дерево" },
 "prefix-sums-2d": { num: 5, title: "5. Двумерная матрица префиксных сумм" },
 "graph-dfs-bfs": { num: 6, title: "6. Графы. Представление" },

```

```

.....
 "pathfinding-accel",
 1));

// Дубликаты: берём последнюю (самую свежую) версию стр
const dedup = new Map<string, Chapter>();
merged.forEach((c) => {
 if (c.id === "alg-map") return; // страница-заглушка
 if (stale.has(c.id)) return;
 dedup.set(c.id, c);
});

// Нормализуем названия по каноническому списку тем.
const withCanon = Array.from(dedup.values()).map((c) => {
 const canon = CANON[c.id];
 if (canon) return { ...c, title: canon.title, category: canon.category };
 return c;
});

// Сначала вводная страница, затем билеты 1–24 по порядку
const numbered = withCanon.filter((c) => CANON[c.id]);
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);
const extras = withCanon
 .filter((c) => !CANON[c.id])
 .sort((a, b) => a.title.localeCompare(b.title, "ru"));

export const chapters: Chapter[] = [...extras, ...numbered];

```

ФАЙЛ 20/92: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 184 | РАЗМЕР: 1.2 КБ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * код под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

.....
 const srcLiteral = JSON.stringify(userSrc); // JSON-с
 return `import sys as __sys, json as __json

__src = ${srcLiteral}
__steps = {}
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без пар
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"),
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 кл
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k,

```

Универсальное пособие • полный исходный код

```
.....
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __file__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

__OUT = __OUT.replace("\n", "\\n")
__OUT = __OUT.replace("\r", "\\r")
__OUT = __OUT.replace("\t", "\\t")

/**
 * Какие переменные изменились на этом шаге относительно предыдущего
 * они подсвечиваются (как amber-подсветка изменений в коде)
 * Новые переменные тоже считаются изменёнными.
 */
export function changedVars(
 prev: Record<string, string> | null | undefined,
 cur: Record<string, string>
```

## Универсальное пособие • полный исходный код

```
.....
* Любое слово из `labels`, встреченное в тексте главы,
* в подчёркнутый термин: при наведении (или тапе на мо
* с объяснением на пальцах и мини-визуализацией.
*/
export interface GlossaryEntry {
 id: string;
 /** Варианты написания в тексте (регистр не важен). */
 labels: string[];
 title: string;
 /** Объяснение «на пальцах» – 1–2 фразы. */
 short: string;
 /** Строгая формулировка / что важно не забыть. */
 formal?: string;
 complexity?: string;
 /** Какая мини-анимация показывается в карточке. */
 demo?: DemoKind;
 /** id полного симулятора (см. vizRegistry) – кнопка
 simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
 {
 id: "bfs",
 labels: ["BFS", "обход в ширину", "поиск в ширину",
 title: "BFS – обход в ширину",
 short:
 "Волна от источника: сначала все соседи, потом со
 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
 },
 {
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину",
 title: "DFS – обход в глубину",
```



```

.....
 formal: "push / pop / top за $O(1)$. На стеке живёт р
 demo: "stack",
 simulator: "stack-dfs",
},
{
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот п
 formal: "push_back / pop_front за $O(1)$. Основа BFS
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "СНМ", "система непересекающихся мн
 title: "DSU – система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего
 formal:
 "find со сжатием путей + union по рангу/размеру.
 complexity: "почти $O(1)$ – обратная функция Аккерман
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "t
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово
 formal: "Из корня по символам; в вершине хранится ф
 complexity: "поиск слова – $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",

```

```

.....
 "Порядок «сначала штаны, потом ботинки»: каждая с
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набегало от н
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]
 complexity: "препроцессинг O(n), запрос O(1)",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp
 "разреженная", "двоичные подьёмы", "двоичный подь
],
 title: "Разреженная таблица (метод двоичных подьёмов)
 short:

```

```

.....
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и
 но).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи
 "амортизированное", "амортизированного", "амортизи
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер
 formal: "Метод потенциалов: дорогая операция оплачи
 ассив.",
 demo: "amortized",
},
{
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP – «решение проверить легко, найти трудно» (су

```

## Универсальное пособие • полный исходный код

```
.....
 "п[i] – длина наибольшего собственного префикса s
 возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи
 аждой букве.",
 formal:
 "z[i] = длина наибольшего общего префикса s и s[i
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин
 "мемоизация", "мемоизацию", "подзадач", "подзадач
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся
 ция).",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
```

```

 "Пройти по каждому ребру ровно один раз, не отрываясь",
 formal:
 "Связный граф: эйлеров цикл все степени чётные;",
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планарности"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги",
 formal:
 " $V - E + F = 2$ для связного планарного графа. Курсив",
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вытягивание",
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают»",
 formal:
 "Три случая: zig (родитель – корень), zig-zig (узлы в одну сторону), zig-zag (узлы в противоположные стороны).",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "матрица смежности"],
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быстрый доступ к ребру, экономно для разреженных графов.",
 formal:
 "Матрица смежности – таблица «есть ли ребро», быстрый доступ к ребру, экономно для разреженных графов.",
 simulator: "adjacency",
}

```

```

.....
 шить пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Поряд
 ами.",
 complexity: "O(V^3) времени, O(V^2) памяти",
 demo: "floyd",
 simulator: "floyd",
},
{
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажд
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prim",
},
{
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
},
{
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruska
 title: "Алгоритм Краскала",
 short:

```

```

.....
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам ну
 рядковым номером.",
 formal:
 "sort + unique + binary search: значение → индекс
 ,
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова де
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу
 ча. Случайность и держит его сбалансированным."
 formal:
 "Базис – split(t, x) (разрезать по ключу) и merge
 я через них. Ожидаемая высота O(log n).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа – всё больше. Пои
 formal:
 "Без балансировки дерево может вырождаться в списо
 complexity: "O(высоты): от O(log n) до O(n)",
 demo: "segment-tree",
},
{
 id: "inclusion-exclusion",

```

```

.....
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с",
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где (Z-функция) и где встречаются слова словаря (бор),
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо",
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на движшемуся месту. Порядок ключей при этом не ломается",
 formal:
 "Правый поворот вокруг p: $x = p.left$; $p.left = x$.
 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig — одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корне и канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig-zig",

```



```

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

</details>

<div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Форд-Беллман
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-rose-300 italic">Параноик считает, что все вокруг него заговорили против него.
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-6">
 Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman" class="mt-8">
 </div>

```

```
.....
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-6">
 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-6">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
```

```

.....
<summary class="p-4 font-bold text-slate-400 outline-
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>go</code> и
 ссылку <code>term_link</code> (ближайшая терминальная вершина)
 <div class="bg-black/50 p-4 rounded-lg font-mono">
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по
 int term_link = 0; // Терминальная ссылка: мат. слово
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные ссылки
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход по ссылке родителя
 nodes[child].link = nodes[nodes[v].link].go[ch];

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].link;
 } else {

```

ФАЙЛ 23/92: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": {
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет – это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 – это эйлеров путь. Если нечётных больше 2 – эйлеров пути нет, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл – это NP-полная задача?",
 options: [
 "Потому что Гамильтон – болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, можно ли решить за полиномиальное время."
 },
 {
 question: "Куда ведёт эйлеров путь, если нечётных вершин ровно 2?",
 options: [
 "В любую вершину с нечётной степенью",
 "В вершину с минимальной степенью",
 "В вершину с максимальной степенью",
 "В любую вершину с чётной степенью"
],
 correctIndex: 0,
 explanation: "Эйлеров путь всегда начинается и заканчивается в вершинах с нечётными степенями. Если их ровно 2, то путь ведёт от одной к другой."
 }
]
}
```

```

.....
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
 }
},
"planarity-euler-formula": {
 {
 question: "Планарный граф имеет $V=6$, $E=12$. Возможн
 options: {
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
 },
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ — на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с $V=7$
 options: {
 "3 грани",
 "5 граней",
 "10 граней"
 },
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K_5 непланарен?",
 options: {
 "Потому что $10 > 3*5 - 6 = 9$ — нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин — несчастливое число"
 },
 correctIndex: 0,
 explanation: "У K_5 : $V=5$, $E=10$, $3V - 6 = 9$. $10 > 9$
 }
};

```

```

.....
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...snapshot };
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>(
 "viz-state", { detail }
));
}

/** Вернуть демонстрацию в ручной режим при закрытии/перезагрузке */
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>(
 "clear-viz-state", { detail: { chapterId } }
));
}

/** Текущий снимок компилятора для прямой отрисовки масива */
export function useVizRuntime(): VizStateEventDetail | null {
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventDetail>(
 chapterId ? latestByChapter.get(chapterId) ?? null : null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId) : null);
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDetail>).detail;
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !== null ? detail : null);
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
 }, [chapterId]);

 return runtime;
}

```

```
.....

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined
 typeof value === "number" && Number.isFinite(value) ?

export const vizString = (value: DebugValue | undefined
 typeof value === "string" ? value : null;

export const vizArray = (value: DebugValue | undefined)
 Array.isArray(value) ? value : null;

export const vizRecord = (value: DebugValue | undefined
 value !== null && typeof value === "object" && !Array
 ? (value as Record<string, DebugValue>)
 : null;

/* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>(
 "viz-demo", { detail: latestDemoByChapter[chapterId] }
));
}

/**
* Подписка: панель компилятора следует за активной вкладкой
* Сразу отдаём последнее значение: компилятор часто отстает
* как пользователь переключился с 1D на 2D.
```



## Универсальное пособие • полный исходный код

```
.....
/** Заголовок демонстрации на странице (если интерактив)
vizTitle?: string;
/** Чему соответствует один шаг визуализации (для тул)
stepNote?: string;
/** Переменные синхронизации. Пусто – на странице нет
variables: SyncVariable[];
/** Полный референсный Python-код, который кнопка-главн
code?: string;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков: build → query → массовое
 stepNote: "Шаг = спуск по вершине v: целиком внутри
 lazy детям 2·v и 2·v+1.",
 code: `n = 6 # длина массива ,
A = [5, 8, 3, 12, 7, 2]
tree = [0] * (4 * n)
lazy = [0] * (4 * n) # lazy[v] – невыполненное «
v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
ul, ur, add = 1, 3, 10 # массовое обновление: A[ul

--- тело алгоритма ---
def build(v, l, r):
 if l == r:
 tree[v] = A[l]
 return
 m = (l + r) // 2
 build(2 * v, l, m)
 build(2 * v + 1, m + 1, r)
 tree[v] = tree[2 * v] + tree[2 * v + 1]

build(1, 0, n - 1)
print("корень:", tree[1])`,
 variables: {
 { name: "n", role: "длина исходного массива; реку
```

## Универсальное пособие • полный исходный код

```

.....
 { name: "keys", role: "демо-дерево страницы (BST)",
 { name: "key", role: "ключ, который splay поднимает",
 { name: "i", role: "номер кадра анимации", range: 1,
 { name: "case", role: "случай поворота для вкладки",
 1,
 },
 },
 "splay-rotations": {
 vizTitle: "Splay: случай поворота",
 stepNote: "Шаг = один кадр разбора случая: zig (1 п
 code: `case = "zig-zig" # "zig" / "zig-z
i = 0 # номер кадра внутри случая
x, p, g = 20, 40, 60 # узел, родитель, дед – как

print(f"{case}: кадр {i}, крутим ребро {p}·{g if case =
 variables: {
 { name: "case", role: "какой случай крутить: zig",
 { name: "i", role: "номер кадра в разборе случая",
 { name: "x, p, g", role: "тройка узлов: поднимаем
 1,
 },
 "sparse-table": {
 vizTitle: "Разреженная таблица (1D и 2D)",
 stepNote: "Шаг = одна ячейка st[i][j] = min(st[i][j
 code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 << (j -
 variables: {
 { name: "a", role: "исходный массив демо (вкладка
 { name: "n, LOG", role: "длина массива и число ур
 { name: "i", role: "начало блока: st[i][j] отвеча

```

## Универсальное пособие • полный исходный код

```

.....
 variables: {
 { name: "n", role: "целевой номер числа Фибоначчи"},
 { name: "k", role: "аргумент текущего рекурсивного вызова"},
 { name: "memo", role: "кэш уже посчитанных значений"},
 { name: "r1, r2", role: "ответы двух рекурсивных вызовов"},
 },
 "heap-beam-search": {
 vizTitle: "Куча (priority queue)",
 stepNote: "Шаг = просеивание: вставленный элемент в кучу",
 code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: {
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас просеиваем"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
 },
 },
 "stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой тарелки на стек или pop тарелки",
 code: `st = [] # стек = рекурсионная таблица
st.append("A") # push – зашли в вершину
st.append("B")
top = st.pop() # pop – возврат на уровень выше
print("pop →", top, "стек:", st)`,
 variables: {
 { name: "st", role: "стек (массив): тарелки стран"},
 { name: "top", role: "что сняли с вершины – текущая вершина"},
 },
 },

```

```

.....
 visited.add(v)
 order.append(v)
 q += [u for u in adj[v] if u not in visited]
print("BFS:", order)`,
 variables: [
 { name: "adj", role: "список смежности демо-графа"
 { name: "v", role: "текущая вершина обхода – подс
 { name: "stack", role: "стек DFS: последний зашёл
 { name: "q", role: "очередь BFS: первый зашёл – п
 { name: "visited / order", role: "множество посещ
],
},
"graph-components": {
 vizTitle: "Компоненты связности: BFS-разметка",
 stepNote: "Шаг = вершина извлечена из очереди или с
 code: `from collections import deque

демо-граф страницы: три «острова» A-D, E-G, H-J
adj = {"A": ["B", "C", "D"], "B": ["A", "C"], "C": ["A",
 "E": ["F", "G"], "F": ["E", "G"], "G": ["E", "F"]
comp = {} # comp[v] – номер компонен
count = 0 # сколько раз запускали об
v, q = None, [] # текущая вершина и очередь

--- тело алгоритма ---
for s in adj:
 if s in comp:
 continue
 count += 1
 q = deque([s])
 comp[s] = count - 1
 while q:
 v = q.popleft()
 for to in adj[v]:
 if to not in comp:
 comp[to] = comp[v]
 q.append(to)
print(comp, "компонент:", count)`,

```

## Универсальное пособие • полный исходный код

```
.....
def dfs1(v): # проход 1: порядок выхода
 used.add(v)
 for to in adj[v]:
 if to not in used:
 dfs1(to)
 order.append(v)

--- тело алгоритма ---
for v in adj:
 if v not in used:
 dfs1(v)

adjT = {v: [] for v in adj} # транспонированный граф
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)

used = set()
comp = {}
num = 0
for v in reversed(order): # проход 2: по order задом
 if v in used:
 continue
 num += 1
 stack = [v]
 used.add(v)
 while stack:
 u = stack.pop()
 comp[u] = num
 for to in adjT[u]:
 if to not in used:
 used.add(to)
 stack.append(to)
print("компонент:", num, comp),
variables: {
 { name: "v", role: "текущая вершина DFS (проход 1"
 { name: "order", role: "порядок выхода прохода 1"
 { name: "adjT", role: "транспонированный граф: вс
```

## Универсальное пособие • полный исходный код

```

.....
 { name: "low", role: "минимальный tin, достижимый
 { name: "v, to", role: "ребро дерева DFS, для кото
 { name: "is_bridge", role: "low[to] > tin[v] ⇒ из
 },
},
"euler-path-vs-cycle": {
 vizTitle: "Эйлера путь и цикл",
 stepNote: "Шаг = один клик по следующей вершине марш
 code: `edges = [{"A","B"}, {"A","C"}, {"B","C"}, {"
path = [{"A", "B", "C"}] # маршрут: рисуется на графе
deg = {v: 0 for v in "ABCDE"}
for u, v in edges: # степени вершин
 deg[u] = deg.get(u, 0) + 1
 deg[v] = deg.get(v, 0) + 1

odd = []
for v in deg:
 if deg[v] % 2:
 odd.append(v)
print(deg, "нечётных:", len(odd))
0 нечётных → цикл · 2 → путь · иначе – нельзя`,
 variables: [
 { name: "path", role: "маршрут списком меток верши
 { name: "deg", role: "степень вершины: чётная всю
 },
],
},
"planarity-euler-formula": {
 vizTitle: "Планарность: K5/K3,3 + раскраска",
 stepNote: "Шаг = проверка формулы Эйлера для K5/K3,3
 code: `V, E = 5, 10 # K5 (K3,3: V=6, E=9)
F = 2 - V + E # формула: V - E + F = 2
not_planar = E > 3 * V - 6 # 10 > 9 → K5 непланарен

покраска (вторая демонстрация страницы):
смежности демо-графа из 7 вершин A..G
adj = [[1, 4, 5], [0, 2, 5], [1, 3, 5, 6], [2, 4, 5, 6],
 [3, 0, 5], [0, 1, 2, 3, 4, 6], [2, 3, 5]]

```

```

.....
 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)\,
 variables: [
 { name: "u", role: "вершина с минимальным dist среди",
 { name: "v", role: "сосед вершины u, до которого",
 { name: "w", role: "вес ребра (u, v)", range: "по",
 { name: "dist[v]", role: "лучшее известное расстояние",
],
},
"bellman-ford": {
 vizTitle: "Форд-Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
 code: `# рёбра демо-графа страницы (включая коварное ребро A-B)
E = [("S", "A", 4), ("S", "B", 5), ("B", "C", -2), ("A", "C", 3),
 ("C", "D", 3), ("C", "E", 4), ("D", "F", 2), ("E", "F", 4)]
n = 7 # вершин: S, A, B, C, D, E, F
dist = {"S": 0}
u, v, w = E[0] # шаг: очередное ребро (u, v, w)
i = 1 # номер итерации

--- тело алгоритма ---
for i in range(1, n): # n-1 прохода по всем рёбрам
 for u, v, w in E:
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
print(dist)
n-й проход улучшил бы dist => отрицательный цикл (на э
 variables: [
 { name: "E", role: "список рёбер (u, v, w) – демо-граф",
 { name: "n", role: "число вершин: после n-1 итерации",
 { name: "i", role: "номер итерации: после i итерации",
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
 { name: "dist", role: "расстояния от S; улучшение",
],
},

```

## Универсальное пособие • полный исходный код

```

.....
 { name: "h", role: "потенциалы: кратчайшие пути о
 { name: "u, v, w", role: "ребро (u, v) перевзвешив
 { name: "w_new", role: "новый вес – всегда ≥ 0 , к
 { name: "d[v]", role: "расстояние Дейкстры в новы
 },
],
},
"mst-kruskal": {
 vizTitle: "Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных
 code: `# демо-граф страницы: 9 вершин A..I, 12 рёбер
E = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10,
 (12, "H", "I"), (13, "F", "I")]
E.sort() # сортируем по весу
parent = {v: v for v in "ABCDEFGHI"} # DSU: каждый сам
w, u, v = E[0] # шаг: очередное ребро
i = 0

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[x] # сжатие пути
 x = parent[x]
 return x

mst = []
for i, (w, u, v) in enumerate(E):
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
print("остов:", mst, "вес:", sum(w for w, _, _ in mst))
variables: [
 { name: "E", role: "рёбра демо-графа, отсортирова
 { name: "w, u, v", role: "текущее ребро: вес и ко
 { name: "i", role: "индекс текущего ребра в отсор
 { name: "parent", role: "DSU: представитель компо
 { name: "mst", role: "рёбра остова; в ответе равн

```



## Универсальное пособие • полный исходный код

```
.....
vizTitle: "Борувка",
stepNote: "Шаг = каждая компонента одновременно выб
code: `# демо-граф страницы: 9 вершин A..I, 12 рёбер
E = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10,
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGHI"} # каждая вершина —
cheapest = {} # компонента → её самое дешёвое ребро
u, v, w = "A", "B", 2 # шаг: ребро-кандидат
phase = 1

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len({find(v) for v in comp}) > 1:
 cheapest = {}
 for w, u, v in E: # каждая компонента выбира
 ru, rv = find(u), find(v)
 if ru == rv:
 continue
 if ru not in cheapest or cheapest[ru][0] > w:
 cheapest[ru] = (w, u, v)
 if rv not in cheapest or cheapest[rv][0] > w:
 cheapest[rv] = (w, u, v)
 for ru, (w, u, v) in cheapest.items(): # все слия
 comp[find(u)] = find(v)
 print(f"фаза {phase}: осталось компонент", len({find
 phase += 1
print("готово за", phase - 1, "фазы (на демо — 2)")`,
variables: [
 { name: "E", role: "рёбра демо-графа", range: "12
 { name: "comp", role: "DSU: номер компоненты (пле
 { name: "cheapest", role: "самое дешёвое исходяще
 { name: "u, v, w", role: "ребро-кандидат: концы и
```

## Универсальное пособие • полный исходный код

```
.....
z = [None] * len(s) # пустые клетки: значения по умолчанию
z[0] = 0

--- тело алгоритма ---
for i in range(1, len(s)): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - 1]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < len(s) and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`,
 variables: [
 { name: "pattern", role: "образец: питон подставлял"},
 { name: "text", role: "текст, в котором ищем", gap: 1},
 { name: "s", role: "склейка pattern + \"#\n\" + text"},
 { name: "i", role: "позиция, для которой считаем z[i]"},
 { name: "l, r", role: "правый Z-блок: отрезок, соотв. r"},
 { name: "z", role: "z[i] – длина общего префикса s[i:] и pattern"},
],
},
"aho-corasick": {
 vizTitle: "Ахо–Корасик",
 stepNote: "Шаг = построение бора по образцам, затем поиск",
 code: `s = "USHERS" # текст, в котором ищем
i, v = 0, 0 # позиция в тексте и вершина автомата
trie = {"": {}} # бор: вершина → переходы
state, c = "", "u" # шаг: автомат идёт по символу
nxt = trie.get(state, {}).get(c, "") # go(state, c) с помощью словаря
print(f"шаг {i + 1}: v={v}, символ {c!r} → {nxt!r}")`,
 variables: [
 { name: "s", role: "текст для поиска (можно и text)"},
 { name: "i", role: "текущая позиция в тексте – левый индекс"},
 { name: "v", role: "номер текущей вершины автомата"},
 { name: "c / state", role: "символ и состояние – правый индекс"}
]
}
```

## Универсальное пособие • полный исходный код

```
.....
База не отдельный уровень вычислений: блок 1x1 — это
Записываем её одним шагом, а не проигрываем 64 одности
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(((r, c, 0, 0), A[r][c]) for r in range(1, 9)
 for c in range(1, 9)))

for k in range(1, 4):
 print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
 size = 9 - (1 << k)
 half = 1 << (k - 1)
 for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
 (r + half, c + half, k - 1, k - 1)])
 variables: {
 { name: "r, c", role: "левый верхний угол блока, "
 { name: "k", role: "уровень: блоки 2^k x 2^k (шаг 2^k)"
 { name: "st2[(r,c,k,k)]", role: "минимум квадратов"
 },
 1,
},
"sparse-table#2d-query": {
 vizTitle: "Разрезанная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[(r, c, kx, ky)] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня {kx, ky}")
 variables: {
 { name: "r1, c1, r2, c2", role: "углы запрашиваем"
 { name: "kx, ky", role: "уровень самого крупного
 },
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3]]`
```

```

const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .
 : `# «${chapterTitle}»\n# демо на странице нет – св

if (!sync.code) return head;
return `${head}\n${sync.code}\n`;
}

/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * заменяет редактор полным референсным кодом.
 */
function extractInit(code: string): string {
 const keep: string[] = [];
 for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while|
 keep.push(line);
 }
 // убрать пустые строки с конца
 while (keep.length > 0 && keep[keep.length - 1].trim()
 return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО иници
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, ch
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#
 : `# «${chapterTitle}»\n# демо на странице нет – св

```

```

.....
 omise (lazy).</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 Аналогия 1: Корпоративная пре
 </div>
 <p class="text-slate-300 text-sm mb
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сот
женное обновление).</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
border-rose-500 shadow-md">
 Аналогия 2: Матрешки-коробки
 </div>
 <p class="text-slate-300 text-sm mb
ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Строгое доказательство / Код (Скр
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];

```

```

.....
 зок длины 7. Ты берешь две заготовленные за
 ь отрезок длины 7 (перекрывая друг друга по
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb
ужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1

```

```

.....
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}
}
</pre>
</div>
</details>
</div>
</div>
</section>
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансирует",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизи
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl">
 <h4 class="text-lg font-bold text-indigo-300">
 <div class="space-y-4 text-sm text-slate-300">
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 /p>
 </div>
 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключ
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит п
 </div>
 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоять
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 }
}

```



```
.....

import { Chapter } from '../..types';

export const tickets14to20: Chapter[] = [
 {
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 14
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы. Поиск кратчайшего пути. Дейкстра
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Дейкстра: Поиск кратчайшего пути.
 <p class="text-xl font-mono text-white">Графы. Поиск кратчайшего пути. Дейкстра
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>

```

```

border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4">
опали во временную петлю.</p>
 </div>
</div>
<div id="slot-bellman-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border-1 border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-700/50
border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл

```

```

.....
 er border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Дейкстра (Билет 14) быстрая, но
 ает очень медленно. Мы хотим сделать Джонсона
 е пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-4">
 Добавляем фиктивную вершину
 Запускаем от неё медленный поиск
 «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое р
 Магия! Все веса тепер
 ренные потенциалы сокращаются).
 Теперь смело запускаем быстрый поиск

 </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">

```

```

</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 ✱ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-6">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
</div>
</div>
</section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4

```

```
.....
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
сом, оканчивающимся в позиции i.</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb
 Мы идём по строке слева направо. К
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слов
x</code>. Тупой алгоритм начнет искать занов
</code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg i
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
```

```

</div>
</section>`
},
{
 id: "string-z-func",
 title: "22. Z-функция – Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb">
 Тут мы берём всю строку и «прик
 я начну читать строку отсюда, насколько длин
 Это самый быстрый способ найти
 >, считаешь Z-функцию, и там, где <code>Z[i
 </p>
 </div>

```

```
.....
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">1

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
 <p class="text-sm text-blue-300 italic mb-2">Осложнение: $O(n)$
 <p class="text-lg font-mono text-white">Бор (Тренировка)
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
 </div>
```

```

 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
 }
 return t[v].go[c];
}
</pre>
</div>
</details>
</div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Сведение (Reduction) $A \leq_p B$: вход задачи A сводится к задаче B
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс NP – множество задач, которые можно проверить за полиномиальное время.
 </div>
 </div>
 </div>
 </div>
`

```



```
.....
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
</div>
<div class="space-y-8">
 { /* База представлений */ }
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
(V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
группо, но проверка "знает ли Вася Петю" мгно
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>

 { /* DFS vs BFS */ }
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
.....
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код обходов C++ (Основа)
 </summary>
 <div class="grid grid-cols-1 md:grid-co
 <pre class="bg-slate-950 p-3 rounded
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}
</pre>
 <pre class="bg-slate-950 p-3 rounded
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}
</pre>
 </div>
 </details>
</div>
</section>',
},
```

```

.....
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Графы
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Графы – это структуры, состоящие из вершин и ребер.
 <p class="text-xl font-mono text-white">Графы используются для моделирования отношений между объектами.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">Острова – это вершины, а проливы – ребра. Если вы хотите добраться с одной карты – остались ли серые (неисследованные) территории – это ребра. Если вы хотите добраться с одной карты – остались ли серые (неисследованные) территории – это ребра.
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <p class="text-slate-300 text-sm mb-4">Графы используются для моделирования отношений между объектами.
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 },

```

```

</summary>
<div class="p-5 text-sm text-slate-300">
 <p class="text-indigo-400 font-bold">
 <div class="bg-slate-950 p-4 rounded">
 <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
 </div>
 </div>
 </details>
</div>
</div>
</section>',
},
{
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
 category: "Графы. Продвинутое",

```

```
.....
 йти наружу (к директору), но обратно в клуб
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Связь с Билетом 12
 </div>
 <p class="text-slate-300 text-sm mb">
 SCC (Билет 10) склеивает
 </p>

 Точки сочленения (Билет 12)
 ость монолита. Вырвешь точку сочле
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе

 Разворачиваем этот список (то есть делаем reverse)

 Переворачиваем все ребра в графе

 Идем по <code>order</code>:

 </p>
 </div>
</details>
</div>
</div>
</section>`,
```

```
.....
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Вершины
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">Вершины
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400">Вершины
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Вершина, удаление которой увеличивает число компонент связности, называется точкой articulation point.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка articulation point - это вершина, удаление которой увеличивает число компонент связности. Например, в графе с 4 вершинами и 5 ребрами (1-2, 2-3, 3-4, 1-3, 2-4) вершина 2 является точкой articulation point, так как ее удаление приводит к разрыву графа на две компоненты (1-3 и 2-4).</p>
 <div class="p-3 bg-slate-950 rounded">
 Пример:
 <p class="text-xl font-bold text-rose-400">Вершина 2
 <p class="text-xs text-slate-400">Вершина 2
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Важно: Это работает только для вершин, которые являются точками сочленения. То есть, Точка articulation point - это тупик со степенью 1.
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 <div class="text-rose-400 font-weight-bold">Вершины
 <div class="text-rose-400 font-weight-bold">Вершины
 </div>
 </div>
 </div>
`
```



---

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

ФАЙЛ 30/92: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
```



```

useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curre
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${
 v,
 });
 }
 }
 };

```

```

 }

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
 }, []);

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />

```

```

return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : ""}
 />
);
}}}

{/* Nodes */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (

```

```

.....
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t

```

```

width: `${currentStepIndex / Math.max(
 width: `
 }}
 />
</div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 31/92: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);

```

```

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start'
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0'
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations'
 { line: 4, text: ' for (u, v, weight) in E: // E = edges'
 { line: 5, text: ' if dist[u] + weight < dist[v]:'
 { line: 6, text: ' dist[v] = dist[u] + weight'
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательные циклы'
 { line: 8, text: ' if dist[u] + weight < dist[v]:'
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"'
 { line: 10, text: ' return dist' },
];

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999'
 activeCodeLine: 2,
 });

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 }
 }
 if (!anyUpdate) break;
 simulationSteps.push({
 iteration: iter, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: `    Итерация ${iter}. Проверка на отрицательные циклы`
 activeCodeLine: 7,
 });
}

```

```

.....
 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev }
 log: ' Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
 }
 }

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарплата...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== nu
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,

```

```

<div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (Шаг {currentStep.step})
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-rose-500/20
 animate-pulse shadow-lg shadow-rose-600/50">
 ⚠ Отрицательный цикл!
 </div>
)}

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking = currentStep.checkingEdges === idx;

 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; marker = 'url(#arrow-bf-active)'; }
 else if (currentStep.hasNegativeCycle && isNeg) { strokeColor = '#e11d48'; marker = 'url(#arrow-bf-cycle)'; }
 })}
 </svg>

```



```

.....
 { /* Список смежности */ }
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
)
 })}
 </div>
)
)
 })}
 </div>
)
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 { /* Таблица расстояний */ }
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div>
 <h4 className="text-lg font-bold text-rose-
 <div className="grid grid-cols-4 gap-2 text
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 round
 : 'border-slate-700/60 bg-slate-800/40'}`}>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
 v>

```

```

.....
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];

const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."
```

```
.....
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.
 });
 }
}
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};
```

```

.....
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведе
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Далее вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
 {/* Визуал графа */}
 <div className="bg-slate-800 border border-slate
 50px]">
 <svg className="w-full h-full min-h-[350px]"
 {/* Рёбра */}
 {EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n
 return (
 <line
 key={idx}

```

```

.....
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration-200"
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-sm"
 >
 {n.label}
 </text>
 </g>
);
}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-2 font-mono font-size tracking-wider">
 bfs(graph, start)
 var {steps}
 </div>
 <div className="p-4">
 {
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visit' },
 { line: 3, code: '}' },
 { line: 4, code: 'while not queue.isEmpty()' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node]' },
 { line: 7, code: ' if neighbor not in visited' }
 }
 </div>
 </div>

```

```
.....
 </div>
 </div>
);
}
```

ФАЙЛ 33/92: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
.....
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

Универсальное пособие • полный исходный код

```
.....
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }

 return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2">
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-slate-100
 transition-all"

```

```
import { TermPopover, type HintAnchor } from "../hints/TermPopover";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлен код */
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart
```

```
export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

 useTermHints(contentRef, [chapter.id]);

 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);

 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(

```



```

 };

 const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
 };

 const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
 };

 const viz = getViz(chapter.id);

 return (
 <
 <article className="chapter-body">
 /* панель загрузки: сохранить именно эту страницу */
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small border-1px
 border-emerald-500 hover:text-white hover:border-emerald-500"
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
 </button>

```

```

<button
 onClick={() => onOpenSimulator(chapter)
 className="flex items-center gap-1.5
 slate-300 hover:text-white hover:border-ind
>
 <Maximize2 className="w-3.5 h-3.5" />
</button>
{onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5
 t-emerald-400 hover:text-white hover:border
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
</div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
 <viz.Component />
</VizChapterContext.Provider>
</section>
}}

<ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}

```

```
};
```

```
const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [3, 4], [4, 0],
 [0, 5], [1, 5], [2, 5], [3, 5], [4, 5],
 [2, 6], [3, 6], [5, 6],
];
```

```
const ADJ: number[][] = (() => {
 const adj: number[][] = NODES.map(() => []);
 EDGES.forEach(([u, v]) => {
 adj[u].push(v);
 adj[v].push(u);
 });
 return adj;
})();
```

```
const COLORS = [
 { name: "красный", hex: "#ef4444" },
 { name: "синий", hex: "#3b82f6" },
 { name: "зелёный", hex: "#22c55e" },
 { name: "жёлтый", hex: "#eab308" },
];
```

```
const CONFLICT = "#e11d48";
```

/\*\* Жадная раскраска по возрастанию номеров — честный у

```
function greedy(): number[] {
 const color: number[] = NODES.map(() => -1);
 for (let v = 0; v < NODES.length; v++) {
 const used = new Set(ADJ[v].map((u) => color[u])).fi
 let c = 0;
 while (used.has(c)) c++;
 color[v] = c;
 }
 return color;
}
```

```

const liveCurrent = (() => {
 if (liveVLabel) {
 const i = NODES.findIndex((n) => n.label === liveVLabel);
 if (i >= 0) return i;
 }
 if (liveVNum !== null && liveVNum >= 0 && liveVNum <= NODES.length) {
 return null;
 }
})();

const conflicts = useMemo(() => {
 const bad = new Set<number>();
 EDGES.forEach(([u, v]) => {
 const cu = shownColors[u];
 const cv = shownColors[v];
 if (cu >= 0 && cu !== cv) {
 bad.add(u);
 bad.add(v);
 }
 });
 return bad;
}, [shownColors]);

const usedColors = useMemo(
 () => new Set(shownColors.filter((c) => c >= 0)).size,
 [shownColors]
);

const paint = (id: number) => {
 const next = [...colors];
 next[id] = next[id] === brush ? -1 : brush;
 setColors(next);
};

return (
 <div className="bg-slate-800/90 p-5 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-center">
 <div>
 <h3 className="text-lg font-bold text-white">

```

```

>
 <text x={n.x} y={n.y + 4} textAnchor=
">
 {n.label}
 </text>
 {c >= 0 && {
 <text x={n.x} y={n.y - 20} textAnchor=
vents-none">
 color[{n.id}]= {c}
 </text>
 }}
</g>
);
}}
</svg>
{compilerLinked && {
 <div className="absolute top-2 right-2 bg-emerald-300 z-20">
 Python красит: {liveVLabel ?? ({liveVNum
 </div>
})
{conflicts.size > 0 && {
 <div className="absolute bottom-2 left-1/2
- [11px] text-rose-300 z-20">
 ▲ Конфликт: смежные вершины одного цвета
 </div>
})
</div>

<div className="space-y-3">
 <div className="bg-slate-950 rounded-xl border
 <div className="text-[10px] uppercase tracking
 <Palette className="w-3 h-3" /> Кисть
 </div>
 <div className="grid grid-cols-4 gap-2">
 {COLORS.map((c, i) => {
 <button
 key={c.name}

```

```

.....

/**
 * Тема 24 «Классы сложности, сведение задач».
 * Диаграмма вложенности классов $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$.
 * полиномиальных сведений $A \leq_p B$: если B решается легко
 */

type ClassId = "P" | "NP" | "PSPACE" | "EXPTIME";

interface Problem {
 id: string;
 name: string;
 cls: ClassId;
 x: number;
 y: number;
 npComplete?: boolean;
}

const PROBLEMS: Problem[] = [
 { id: "sort", name: "сортировка\n $O(n \log n)$ ", cls: "P", x: 15, y: 15, npComplete: false },
 { id: "mst", name: "остов (Краскал)", cls: "P", x: 15, y: 185, npComplete: false },
 { id: "2sat", name: "2-SAT", cls: "P", x: 115, y: 185, npComplete: false },
 { id: "shortpath", name: "Дейкстра / Флойд", cls: "P", x: 15, y: 215, npComplete: false },
 { id: "sat", name: "3-SAT", cls: "NP", x: 330, y: 120, npComplete: true },
 { id: "vc", name: "Vertex Cover", cls: "NP", x: 385, y: 185, npComplete: true },
 { id: "ham", name: "Гамильтонов цикл", cls: "NP", x: 385, y: 215, npComplete: true },
 { id: "knapsack", name: "Рюкзак (точный)", cls: "NP", x: 385, y: 245, npComplete: true },
 { id: "circuit", name: "QBF / игра на формуле", cls: "PSPACE", x: 385, y: 275, npComplete: false },
 { id: "chess", name: "обобщённые шахматы", cls: "EXPTIME", x: 385, y: 305, npComplete: false },
];

const REDUCTIONS: [string, string, string][] = [
 ["sat", "vc", "3-SAT $\leq_p$ VC"],
 ["sat", "ham", "3-SAT $\leq_p$ Гамильтон"],
 ["vc", "knapsack", "VC $\leq_p$ Рюкзак"],
];

const CLASS_META: Record<ClassId, { title: string; color: string; x: number; y: number; width: number; height: number; }> = {
 P: { title: "P", color: "#00FF00", x: 15, y: 15, width: 100, height: 50 },
 NP: { title: "NP", color: "#FF0000", x: 330, y: 120, width: 100, height: 50 },
 PSPACE: { title: "PSPACE", color: "#0000FF", x: 385, y: 275, width: 100, height: 50 },
 EXPTIME: { title: "EXPTIME", color: "#FF00FF", x: 385, y: 305, width: 100, height: 50 },
};

```

```

.....
 return [String(liveRed[0]), String(liveRed[1])] as
}, {liveRed});

const activeCls: ClassId | null =
 compilerLinked && liveCls
 ? (Object.keys(CLASS_META).find((k) => k.toLowerCase
 : selected;

const problemHit = (p: Problem) => {
 if (compilerLinked && liveTask) return p.name.toLowerCase
 return selectedProblem === p.id;
};

const infoText = (() => {
 if (compilerLinked && liveTask) {
 const p = PROBLEMS.find((q) => q.name.toLowerCase
 if (p) return `${p.name.replace("\n", " ")} ∈ ${p
 }
 if (compilerLinked && liveCls) {
 const cls = (activeCls ?? "NP") as ClassId;
 return CLASS_INFO[cls];
 }
 if (selectedProblem) {
 const p = PROBLEMS.find((q) => q.id === selectedP
 return `${p.name.replace("\n", " ")} ∈ ${p.cls}${
 .cls]} `;
 }
 if (activeCls) return CLASS_INFO[activeCls as Class
 return "Клики на класс или задачу. «A ≤p B» = «A н
}}());

const reset = () => {
 setSelected(null);
 setSelectedProblem(null);
};

return (
 <div className="bg-slate-800/90 p-5 rounded-2xl bor

```

```

.....
 const a = PROBLEMS.find((p) => p.id === from)
 const b = PROBLEMS.find((p) => p.id === to)
 const hot = compilerLinked && liveRedPair[0] && a.name.toLowerCase === b.name.toLowerCase
 const selectedProblem = from || selectedProblem
 const mx = (a.x + b.x) / 2;
 const my = (a.y + b.y) / 2 - 4;
 return (
 <g key={`${from}-${to}`} opacity={hot ? 0.5 : 1}>
 <line x1={a.x} y1={a.y} x2={b.x} y2={b.y} />
 <marker id="arrowhead" markerWidth="7" markerHeight="5" viewBox="0 0 7 5" fill="black" />
 <text x={mx} y={my} textAnchor="middle">
 {selectedProblem}
 </text>
 </g>
);
 })}
</defs>
<marker id="arrowhead" markerWidth="7" markerHeight="5" viewBox="0 0 7 5" fill="black">
 <polygone points="0 0, 7 2.5, 0 5" fill="black" />
</marker>
</defs>

{PROBLEMS.map((p) => {
 const dim = activeCls !== null && activeCls === p.id ? 1 : 0;
 const hit = problemHit(p);
 return (
 <g key={p.id} onClick={() => setSelectedProblem(p.id)}>
 <rect
 x={p.x - 46} y={p.y - 13} width={92} height={26}
 fill={hit ? "#c2410c" : p.npCompleted ? "#fb923c" : "#fb923c"}
 stroke={hit ? "#fb923c" : CLASS_METADATA[p.id]}
 strokeWidth={hit ? 2.5 : 1.2}
 />
 {p.name.split("\n").map((line, li, arr) => (
 <text key={li} x={p.x} y={p.y + 4 + li * 1.2}
 fontWeight="600">
 {line}
 </text>
))}
 </g>
);
})}

```





## Универсальное пособие • полный исходный код

```
.....
const comp: (number | null)[] = NODES.map(() => null)
let count = 0;
const push = (f: Omit<StepFrame, "comp" | "count"> &
 frames.push({ comp: [...comp], count, ...f }));

push({ queue: [], current: null, scanning: null, log:
 ной вершины." }));

for (let s = 0; s < NODES.length; s++) {
 if (comp[s] !== null) continue;
 count += 1;
 const queue = [s];
 comp[s] = count - 1;
 push({ queue: [...queue], current: s, scanning: null,
 арует с неё.` }));
 while (queue.length) {
 const v = queue.shift()!;
 push({ queue: [...queue], current: v, scanning: null,
 й.` }));
 for (const to of ADJ[v]) {
 if (comp[to] === null) {
 comp[to] = comp[v];
 queue.push(to);
 push({ queue: [...queue], current: v, scanning: null,
 {comp[v]}, в очередь.` }));
 } else {
 push({ queue: [...queue], current: v, scanning: null,
 });
 }
 }
 }
 push({ queue: [], current: null, scanning: null, log:
}
push({ queue: [], current: null, scanning: null, log:
 });
return frames;
}
```

```

.....
 if (liveVLabel) {
 const idx = NODES.findIndex((n) => n.label === li
 return idx >= 0 ? idx : null;
 }
 return liveVNum !== null && liveVNum >= 0 && liveVN
 };
 const liveCurrent = liveIndexOf();

 const liveCompOf = (n: V): number | null => {
 if (liveCompByLabel && n.label in liveCompByLabel)
 const c = vizNumber(liveCompByLabel[n.label]);
 if (c !== null) return c;
 }
 if (liveCompByLabel && String(n.id) in liveCompByLabel)
 const c = vizNumber(liveCompByLabel[String(n.id)]);
 if (c !== null) return c;
 }
 return frame.comp[n.id];
 };
 const liveQueueLabels = (liveQ
 ? liveQ.map((x) => (typeof x === "string" ? x : viz
 : null
)?.filter(Boolean);
 const displayCount = liveCount ?? frame.count;
 const displayLog = compilerLinked
 ? ` Компилятор: v=${liveVLabel ?? liveVNum ?? "..."}
 ES[q].label)).join(", ")}`
 : frame.log;

 const shownCurrent = compilerLinked ? liveCurrent : f
 const shownScanning = compilerLinked ? null : frame.s
 const shownQueue: number[] = liveQueueLabels
 ? liveQueueLabels
 .map((lab) => NODES.findIndex((n) => n.label ===
 .filter((i) => i >= 0)
 : frame.queue;

 return {

```

```

</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-[1fr
 <div className="bg-slate-950 rounded-xl border
 <div className="absolute inset-0 bg-[radial-g
 <svg className="w-full h-[320px] relative z-10
 {EDGES.map(({[u, v], i) => {
 const a = NODES[u];
 const b = NODES[v];
 const sameComp = liveCompOf(a) !== null &
 return (
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={sameComp ? (COMP_COLORS[live
 strokeWidth={sameComp ? 3.5 : 2}
 opacity={sameComp ? 0.9 : 0.6}
 />
);
 })}
 {NODES.map((n) => {
 const c = liveCompOf(n);
 const isCurrent = shownCurrent === n.id;
 const isScanned = shownScanning === n.id;
 const inQueue = shownQueue.includes(n.id);
 const color = c === null ? "#334155" : CO
 return (
 <g key={n.id}>
 {(isCurrent || isScanned) && (
 <circle cx={n.x} cy={n.y} r={18} fil
e-pulse" />
)}
 <circle cx={n.x} cy={n.y} r={13} fill
 <text x={n.x} y={n.y + 4} textAnchor=
">{n.label}</text>
 <text x={n.x} y={n.y - 18} textAnchor
nts-none">

```

```

.....
 0 text-slate-500" : "border-transparent text-
 style={c === null ? {} : { background
LORS.length] }}>
 <div className="text-slate-400">{n.
 <div>{c === null ? "-" : c}</div>
 </div>
);
 }}}
</div>
</div>
<div className="bg-slate-900/60 rounded-xl bo
 {displayLog}
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 39/92: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....
import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from "."
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },

```

```

.....
 { id: 6, label: "G", x: 340, y: 200 }
};

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridge
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],

```

```

.....
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) –
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,
 activeCodeLine: [11,13,16,17]
},
{

```

```

.....
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Шаг {step}
 </h4>
 <div className="flex items-center gap-1 bg-slate-800 p-1">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 disabled={dfsIdx === 0}
 className="p-1.5 hover:bg-slate-700 rounded"
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${
 dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
 }`}
 >
 {dfsAuto ? <Pause className="h-3 w-3" /> : <Play className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1);
 disabled={dfsIdx === DFS_STEPS.length - 1}
 className="p-1.5 hover:bg-slate-700 rounded"
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 className="p-1.5 hover:bg-slate-700 rounded"
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
)

```



```

.....
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#f3f3f3"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#ccc"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle" />
 {label}</text>
 {vis && (
 <div
 <rect x={n.x-18} y={n.y-28} width={36} height={10}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center" />
 {l: {step.low[n.id] || "?"}}</text>
 </div>
)}
 </g>
)}
</svg>
<div className="flex items-center justify-between" />
 Step {dfsIdx+1} / {DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10 rounded" />
 <div className="bg-indigo-600 h-full transition" />
 </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border" />
 <div className="bg-slate-900 px-3 py-1.5 text-slate-300" />
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed">
 {CODE_LINES.map((cl, i) => {

```

```
.....
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
</div>
</div>
</div>
};
}
```

ФАЙЛ 40/92: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
 vizString, vizBoolean, vizFunction } from 'viz';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
];
```

```

.....
 { line: 11, text: ' return dist // Все кратчайшие
 };

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пи
 activeCodeLine: 2,
 });

 for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited }
 log: ` Выбираем вершину ${u} с мин. непосещённой

```

```

 }, []);

 useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
 }, [isPlaying, currentStepIndex, steps.length]);

 const baseStep = steps[currentStepIndex] || null;

 if (!baseStep) return <div className="text-white">Зарядка
 const vars = runtime?.variables;
 const liveU = vizString(vars?.u);
 const liveV = vizString(vars?.v);
 const liveDist = vizRecord(vars?.dist);
 const livePrev = vizRecord(vars?.prev);
 const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
 const compilerLinked = liveU !== null || liveV !== null;
 const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node, liveDist[node]])),
 previous: Object.fromEntries(nodes.map((node) => [node, livePrev[node]])),
 visited: Object.fromEntries(nodes.map((node) => [node, liveDone[node]])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}`
 }
 : baseStep;

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"

```

```

<div className="flex flex-col lg:flex-row gap-6 md:gap-10" style={{width: '100%'}}>
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20 p-6 rounded-2xl flex flex-direction: column align-items: center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20 p-6 rounded-2xl font-bold shadow-sm">
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" style={{width: '100%'}}>
 <defs>
 <marker id="arrow-dh-normal" markerWidth="10" markerHeight="10" fill="#47556b" stroke="#47556b" stroke-width="1">
 <path d="M0,0 L6,3 L0,6 z" fill="#47556b" stroke="#47556b" stroke-width="1"/>
 </marker>
 <marker id="arrow-dh-active" markerWidth="10" markerHeight="10" fill="#f59e00" stroke="#f59e00" stroke-width="1">
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e00" stroke="#f59e00" stroke-width="1"/>
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" fill="#10b98f" stroke="#10b98f" stroke-width="1">
 <path d="M0,0 L6,3 L0,6 z" fill="#10b98f" stroke="#10b98f" stroke-width="1"/>
 </marker>
 </defs>
 {/* Рисуем */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u && currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.u] === edge.v;

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>

```

```

.....
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill={node.id}>
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white" className="bg-indigo-950 px-1 py-0.5">
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white" className="bg-indigo-950 px-1 py-0.5">
 {node.name.split(' ')[0]}
 </text>
 </g>
);
}
}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">
 Список смежности
 </p>
 <p className="min-h-[40px] flex items-center">
 ...
 </p>
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded">
 <h4 className="text-lg font-bold text-emerald-400">
 Список смежности
 </h4>
 <div className="space-y-2.5 flex-1 overflow-y-auto">
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u;
 return (
 <div key={u} className={`p-3 rounded-lg ${isCurrentNode ? 'bg-emerald-900/60 text-white' : 'bg-slate-300 text-slate-300'}>
 {u}
 </div>
);
 })}
 </div>
 <div className="flex items-center gap-2">

 {currentStep.currentNode}

 →

 {currentStep.currentNode}

 </div>
</div>

```

```

</thead>
<tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-1
 isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-medium">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev || '-'}
 </td>
 </tr>
);
 })}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-200">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.id
 })}
 </div>
 </div>
</div>

```

## Универсальное пособие • полный исходный код

```
.....
const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo hit',
```



```

.....
 memoState: { ...memoMap }
 });

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: ` Расчет завершен! fibMemo(${targetN}) = $
 codeLine: 6,
 memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);

```

```

disabled={isPlaying}
className="bg-transparent text-white font
>
<option value={4}>N = 4</option>
<option value={5}>N = 5</option>
<option value={6}>N = 6</option>
<option value={7}>N = 7</option>
</select>
</div>

<div className="flex items-center bg-slate-950
 <button
 onClick={() => { setIsPlaying(false); set
 className="p-2 text-slate-400 hover:text-
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-
 title="Вар назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:
 : 'bg-emerald-600 text-white hover:bg
 }`}
 >
 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button

```

```

 }}}
 </div>

 {currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-400">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-400 p-2">
 Текущий N: <strong className="text-emerald-400">{currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-400">
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-slate-400'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Eye className="w-4 h-4" />
 Стен-бай-стен состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg-slate-400'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Code className="w-4 h-4" />
 </button>
</div>

```

```

.....
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate-
 }` }
 >

 {line}
 </div>
);
 }}}
</pre>
<div className="p-6 bg-slate-900/50 border-l-4 border-emerald-500/20">

 {currentStep?.desc}
 </div>
</div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
 <div className="flex items-center justify-between">
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
</div>
</div>
);
}

```

```

// Компилятор управляет обходом, если в коде есть path
const compilerLinked = livePathLabels !== null && livePathLabels !== undefined;
const livePath = useMemo(() => {
 if (!compilerLinked) return null;
 const ids = livePathLabels!
 .map((lab) => C1_NODES.find((n) => n.label === lab))
 .filter((x): x is number => x !== undefined);
 return ids;
}, [compilerLinked, livePathLabels]);
const shownPath = livePath ?? c1Path;
const shownVisited = useMemo(() => {
 if (livePath) {
 const keys: string[] = [];
 for (let i = 1; i < livePath.length; i++) {
 const a = livePath[i - 1];
 const b = livePath[i];
 if (C1_EDGES.some((e) => (e.u === a && e.v === b))) {
 keys.push(`${Math.min(a, b)}-${Math.max(a, b)}`);
 }
 }
 return keys;
 }
 return c1VisitedEdges;
}, [livePath, c1VisitedEdges]);

const degrees = useMemo(
 () => C1_NODES.map((n) => ({ label: n.label, deg: C1_DEGREES[n.label] })),
 []
);

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы продолжить");
 return;
 }
 const last = c1Path[c1Path.length - 1];

```

```
</div>
```

```
<div className="bg-slate-950 rounded-xl p-4 border-1
-hidden">
```

```
<div className="absolute inset-0 bg-[radial-gradient(1px
#6366f1 1px, transparent 1px)] [background-size: 20px 20px]
```

```
<svg className="w-full h-[260px] z-10 relative">
```

```
{C1_EDGES.map((e, i) => {
```

```
const u = C1_NODES.find(n => n.id === e.u)!
```

```
const v = C1_NODES.find(n => n.id === e.v)!
```

```
const key = `${Math.min(e.u, e.v)}-${Math.max(e.u, e.v)}`
```

```
const used = shownVisited.includes(key);
```

```
return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
```

```
stroke={used ? "#6366f1" : "#475569"}
```

```
strokeWidth={used ? 4 : 2}
```

```
className="transition-all duration-300" />
```

```
}})
```

```
{C1_NODES.map(n => {
```

```
const isLast = shownPath[shownPath.length - 1].id === n.id
```

```
const visited = shownPath.includes(n.id);
```

```
return <g key={n.id} className="cursor-pointer">
```

```
{isLast && <circle cx={n.x} cy={n.y} r={11} />
```

```
<circle cx={n.x} cy={n.y} r={11} />
```

```
fill={isLast ? "#6366f1" : visited ? "#475569" : "white"}
```

```
stroke={isLast ? "#fff" : visited ? "#6366f1" : "black"}
```

```
strokeWidth={2.5}
```

```
className="transition-all duration-200" />
```

```
<text x={n.x} y={n.y + 4} textAnchor="middle">
```

```
{n.label}</text>
```

```
</g>
```

```
}})
```

```
</svg>
```

```
<div className="absolute top-2 left-2 bg-slate-950 p-2 rounded">
```

```
Решёв: {shownVisited.length}/{C1_EDGES.length}
```

```
{compilerLinked &&
```

```
</div>
```

```
<div className="absolute bottom-2 left-2 bg-slate-950 p-2 rounded">
```

```
0">
```

```
Степени: {degrees.map((d) => `${d.label}: ${d.degrees}
```

```

.....
options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф пополам',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра',
],
correctAnswer: 2,
explanation: 'Верно! Если вершины уже в одном клане, добавление ребра
 Это нарушит структуру дерева.',
},
{
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?',
 options: [
 'Потому что он написан только для положительных чисел',
 'Он жадный и считает, что каждый следующий шаг должен минимизировать сумму по закрытым вершинам.',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на ребрах',
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты приедешь в город,
 для чего нужен алгоритм Беллмана-Форда.',
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борузки?',
 options: [
 'Он использует меньше памяти благодаря коммунистическим идеалам',
 'Он умеет работать с отрицательными весами без циклов',
 'Он позволяет выполнять шаги слияния колхозов (коммуны)',
 'Он был разработан в Токио и поэтому самый быстрый',
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая коммуна должна
 делить вычисления на GPU или многопроцессорных кластерах',
},
{

```

```

.....
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };

const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты
 <p className="text-slate-400 text-sm mb-6">Ваш результат: {score} из {quizQuestions.length}
 </p>
 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase">Правильные ответы:
 <p className="text-5xl font-black text-indigo-400">100%

```



```

.....
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer

 let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90"
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border-emerald-500"
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500"
 } else {
 optionStyle = "bg-slate-800/40 border-slate-700"
 }
 }

 return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4 rounded-lg"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full " +
 "border-indigo-500 bg-indigo-500 text-white"}
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-base">
 {option}
 </div>
 </div>
)
 })}
 </div>

```

ФАЙЛ 44/92: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber } from '../utils';
```

```
interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}
```

```
export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольный
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
```

```
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];
```

```
 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
];
```

```

simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => {...r}),
 log: `+ Внешний цикл. Разрешаем пути через пром
 activeCodeLine: 3,
});

for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => {...r}), u
 log: ` Улучшение пути [${i}]→[${j}] чере
 ∞'}.`,
 activeCodeLine: 7,
 });
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => {...r})
 log: ` Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
 let timer: any = null;

```

```

.....
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Полн
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false);
 text-sm"> Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying
 -amber-600' : 'bg-emerald-600')}>{isPlaying
 <button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/2
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></f
 </defs>

```

.....

```
 });
 }}}
</svg>
```

```

<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>
```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2 text-sm font-mono text-white">
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap ${
 isI ? 'bg-amber-900/40 border-amber-400' :
 isK ? 'bg-indigo-900/40 border-indigo-400' : ''
 }`} >
 →
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== u
 const isDown = currentStep.j === edge.v);
 return (
 <span key={idx} className={`px-2 py-1 rounded-sm
 dow' : isVia ? 'bg-indigo-500 text-white border-indigo-400' :
 {edge.v} ({edge.w})

)
 })}
 </div>
);
 })}
 </div>
);
```

```

 </tr>
)})
 </tbody>
 </table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate-100">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4" >
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5 px-4 ${
 item.line === 1 ? 'bg-blue-900/80 text-amber-400' : ''
 }`} >
 {item.line}
 {item.code}
 </div>
 })}
 </div>
 </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 45/92: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from 'viz';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

```

```

type Node = { id: string; label: string; x: number; y: number;

```

```

.....
const vis = new Set<string>();
const stack: string[] = []; // for visualization, just for visualization

function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зависит от visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс: ${v} -> ${u}, visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Обратный шаг: ${v}, visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начало: ${start}, visitedNodes: Array.from(vis) });

 while (q.length > 0) {
 const v = q[0];

```

```

 });
 return candidates.at(-1)?.index ?? null;
 });

 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [mode]);

 useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
 }, [autoPlay, steps.length]);

 const step = steps[stepIdx] || steps[0];

 const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
 const isCurrent = (nodeId: string) => step.node === nodeId;

 return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}

```



```

.....
 textColor = "fill-white"
 }

 if (current && step.type !== "edge") {
 strokeColor = "stroke-amber"
 textColor = "fill-amber"
 radius = 22;
 }

 return (
 <g key={n.id} className={className} >
 <circle cx={n.x} cy={n.y}
 className={`stroke-${strokeColor}`}
 />
 <text x={n.x} y={n.y}
 className={`text-${textColor}`}
 {n.label}
 />

 {
 /* Ripple for BFS mode */
 current && mode === "bfs" && n === current
 ? <circle cx={n.x} cy={n.y}
 className="stroke-amber"
 />
 : null
 }
 </g>
)
}
}}}
</svg>
</div>

/* Structure View (Stack / Queue) */
<div className="bg-slate-900 border border-emerald-500
r">
 <div className={`absolute -top-3 left-3 right-3
 ${mode === 'dfs' ? 'border border-emerald-500' : ''}
 id-300 border border-emerald-500`} >
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

```

```
.....
 <button onClick={() => {setAutoPlay
g-slate-800 rounded-lg ml-2 border-1 border
 <RefreshCw strokeWidth={3} size=
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50
ow-hidden">
 <div className="text-[10px] text-sla
{steps.length}</div>
 <div className="text-sm text-slate-
 {step.desc}
 </div>
 </div>
 </div>
 </div>
);
}
```

ФАЙЛ 46/92: src/components/HeartViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
.....
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

Универсальное пособие • полный исходный код

```
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}
```

```
// Priority presets for realistic ER scenarios
```

```
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
];
```

```
const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {
```

```

.....
const finalName = name || preset.name.split(' ')[0]
const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x => x)
// Mark specifically this new patient as 'in' in the heap
const target = nextHeap.find(x => x.priority === priority)
const marked = nextHeap.map(x => x.id === (target?.id ? target.id : ''))

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
}, 500);
}, [busy, heap]));

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)]
 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift))
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
};

```

```

 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, {busy, heap});

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))), 100);
};

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
});

```

```

 { /* Добавить кастомного */ }
 <form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder
 />
 <div className="flex flex-col items-center min-w-[300px]
 <span className="text-[9px] font-mono text-slate-400"
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Вести
 </button>
 </form>

 { /* Забрать самого тяжелого */ }
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-slate-800 to-slate-900 border border-emerald-500 transition-colors"
 >
 Вывести
 </button>

```

```

.....
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isCompilerNode ? 'ring-4 ring-amb
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
 });
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun

```

```
.....

 { /* ЛОГ ДЕЙСТВИЙ */ }
 <div className="bg-slate-900 border border-slate-800" >
 <div className="text-[10px] font-mono text-slate-200" >
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center`}>
 >
 {idx === 0 ?
 {entry}
 </div>
)}}
 </div>
 </div>
);
 });
 };
};
```

.....  
ФАЙЛ 47/92: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

.....  
import React, { createContext, useContext, useEffect, use

/\*\* Общие примитивы для мини-визуализаций подсказок. \*/

export const W = 260;

export const H = 130;

/\*\*

\* Общий множитель скорости. Кадры были слишком короткими

\* заметить, что именно переехало, и анимация читалась

\*/

export const SPEED = 2.1;

/\*\* Длительность переезда узлов. Должна быть заметно мен

export const MOVE\_MS = 800;



```

</svg>
{caption && (
 <div className="text-[10px] text-slate-400 text-
 {caption}
 </div>
)}
<div className="text-[9px] text-slate-600 text-se
 {paused ? "пауза – курсор на карточке" : "навед
</div>
</div>
});
};

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={str
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBas
 {label}
 </text>
)}
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (

```

```

.....
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 [["A", "B"], ["B", "C"]],
 [["D", "E"]],
 [["F", "G"], ["G", "H"], ["F", "H"]],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return {

```

```

 y={{(a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
}}}
{Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={C.idl}
))}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
 >
 {raw.map((v, i) => {

```

```

.....
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id
 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})`}>
 <circle r={13} fill={color(id)} stroke={C.idl} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={['р – корень, х под ним', 'Один поворот: ']}
 frames={['

```

```

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = ["A", "B", "C", "D", "E", "F"];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
)}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
)}
)}
 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i

```

```

 }
 <text x={224} y={62} textAnchor="middle" fontSize=
 <text x={224} y={74} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь

 <Edge a={{60, 70}} b={{200, 70}} color={improved ? "red" : "blue"} />
 <text x={130} y={54} textAnchor="middle" fontSize=16 />
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize=16 />
 <text x={200} y={106} textAnchor="middle" fontSize=16 />
 <text x={130} y={22} textAnchor="middle" fontSize=16 />
 </Svg>
);
};

```

```

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C—D → граф развалился на 2 компонента" : "Граф связен"} />
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 })}
);
};

```

```

 };
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map(({e, i}) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

```

```

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={step =
))}
 </Svg>
);
};

```

```

 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26}
 fill={i === items.length - 1 ? {popping ? C : "#ccc"} : "#ccc"} />
 <text x={130} y={111 - i * 26} textAnchor="middle">{v}</text>
 </g>
)}
 <text x={130} y={16} textAnchor="middle" fontSize={16}>{v}</text>
</Svg>
);
};

```

```
export const QueueDemo: React.FC = () => {
 const states = [
 ["A", "B"],
 ["A", "B", "C"],
 ["B", "C"]
];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа - к"
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>aux</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>bx</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={20} />
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};
```

```
export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2
```



```

};

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : null;
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M - step)}`}>
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
 })}
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={wave?.includes(n.id) ? C.active : doneIds.has(n.id) ? C.dim : C.default} />
 })}
 <text x={6} y={12} fontSize={9} fill={C.dim}>очередной уровень
 </Svg>
);
};

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3, 0], [4, 1], [5, 1], [6, 2], [7, 2], [8, 3], [9, 3], [10, 4], [11, 4], [12, 5], [13, 5], [14, 6], [15, 6], [16, 7], [17, 7], [18, 8], [19, 8], [20, 9], [21, 9], [22, 10], [23, 10], [24, 11], [25, 11], [26, 12], [27, 12], [28, 13], [29, 13], [30, 14], [31, 14], [32, 15], [33, 15], [34, 16], [35, 16], [36, 17], [37, 17], [38, 18], [39, 18], [40, 19], [41, 19], [42, 20], [43, 20], [44, 21], [45, 21], [46, 22], [47, 22], [48, 23], [49, 23], [50, 24], [51, 24], [52, 25], [53, 25], [54, 26], [55, 26], [56, 27], [57, 27], [58, 28], [59, 28], [60, 29], [61, 29], [62, 30], [63, 30], [64, 31], [65, 31], [66, 32], [67, 32], [68, 33], [69, 33], [70, 34], [71, 34], [72, 35], [73, 35], [74, 36], [75, 36], [76, 37], [77, 37], [78, 38], [79, 38], [80, 39], [81, 39], [82, 40], [83, 40], [84, 41], [85, 41], [86, 42], [87, 42], [88, 43], [89, 43], [90, 44], [91, 44], [92, 45], [93, 45], [94, 46], [95, 46], [96, 47], [97, 47], [98, 48], [99, 48], [100, 49], [101, 49], [102, 50], [103, 50], [104, 51], [105, 51], [106, 52], [107, 52], [108, 53], [109, 53], [110, 54], [111, 54], [112, 55], [113, 55], [114, 56], [115, 56], [116, 57], [117, 57], [118, 58], [119, 58], [120, 59], [121, 59], [122, 60], [123, 60], [124, 61], [125, 61], [126, 62], [127, 62], [128, 63], [129, 63], [130, 64], [131, 64], [132, 65], [133, 65], [134, 66], [135, 66], [136, 67], [137, 67], [138, 68], [139, 68], [140, 69], [141, 69], [142, 70], [143, 70], [144, 71], [145, 71], [146, 72], [147, 72], [148, 73], [149, 73], [150, 74], [151, 74], [152, 75], [153, 75], [154, 76], [155, 76], [156, 77], [157, 77], [158, 78], [159, 78], [160, 79], [161, 79], [162, 80], [163, 80], [164, 81], [165, 81], [166, 82], [167, 82], [168, 83], [169, 83], [170, 84], [171, 84], [172, 85], [173, 85], [174, 86], [175, 86], [176, 87], [177, 87], [178, 88], [179, 88], [180, 89], [181, 89], [182, 90], [183, 90], [184, 91], [185, 91], [186, 92], [187, 92], [188, 93], [189, 93], [190, 94], [191, 94], [192, 95], [193, 95], [194, 96], [195, 96], [196, 97], [197, 97], [198, 98], [199, 98], [200, 99], [201, 99], [202, 100], [203, 100], [204, 101], [205, 101], [206, 102], [207, 102], [208, 103], [209, 103], [210, 104], [211, 104], [212, 105], [213, 105], [214, 106], [215, 106], [216, 107], [217, 107], [218, 108], [219, 108], [220, 109], [221, 109], [222, 110], [223, 110], [224, 111], [225, 111], [226, 112], [227, 112], [228, 113], [229, 113], [230, 114], [231, 114], [232, 115], [233, 115], [234, 116], [235, 116], [236, 117], [237, 117], [238, 118], [239, 118], [240, 119], [241, 119], [242, 120], [243, 120], [244, 121], [245, 121], [246, 122], [247, 122], [248, 123], [249, 123], [250, 124], [251, 124], [252, 125], [253, 125], [254, 126], [255, 126], [256, 127], [257, 127], [258, 128], [259, 128], [260, 129], [261, 129], [262, 130], [263, 130], [264, 131], [265, 131], [266, 132], [267, 132], [268, 133], [269, 133], [270, 134], [271, 134], [272, 135], [273, 135], [274, 136], [275, 136], [276, 137], [277, 137], [278, 138], [279, 138], [280, 139], [281, 139], [282, 140], [283, 140], [284, 141], [285, 141], [286, 142], [287, 142], [288, 143], [289, 143], [290, 144], [291, 144], [292, 145], [293, 145], [294, 146], [295, 146], [296, 147], [297, 147], [298, 148], [299, 148], [300, 149], [301, 149], [302, 150], [303, 150], [304, 151], [305, 151], [306, 152], [307, 152], [308, 153], [309, 153], [310, 154], [311, 154], [312, 155], [313, 155], [314, 156], [315, 156], [316, 157], [317, 157], [318, 158], [319, 158], [320, 159], [321, 159], [322, 160], [323, 160], [324, 161], [325, 161], [326, 162], [327, 162], [328, 163], [329, 163], [330, 164], [331, 164], [332, 165], [333, 165], [334, 166], [335, 166], [336, 167], [337, 167], [338, 168], [339, 168], [340, 169], [341, 169], [342, 170], [343, 170], [344, 171], [345, 171], [346, 172], [347, 172], [348, 173], [349, 173], [350, 174], [351, 174], [352, 175], [353, 175], [354, 176], [355, 176], [356, 177], [357, 177], [358, 178], [359, 178], [360, 179], [361, 179], [362, 180], [363, 180], [364, 181], [365, 181], [366, 182], [367, 182], [368, 183], [369, 183], [370, 184], [371, 184], [372, 185], [373, 185], [374, 186], [375, 186], [376, 187], [377, 187], [378, 188], [379, 188], [380, 189], [381, 189], [382, 190], [383, 190], [384, 191], [385, 191], [386, 192], [387, 192], [388, 193], [389, 193], [390, 194], [391, 194], [392, 195], [393, 195], [394, 196], [395, 196], [396, 197], [397, 197], [398, 198], [399, 198], [400, 199], [401, 199], [402, 200], [403, 200], [404, 201], [405, 201], [406, 202], [407, 202], [408, 203], [409, 203], [410, 204], [411, 204], [412, 205], [413, 205], [414, 206], [415, 206], [416, 207], [417, 207], [418, 208], [419, 208], [420, 209], [421, 209], [422, 210], [423, 210], [424, 211], [425, 211], [426, 212], [427, 212], [428, 213], [429, 213], [430, 214], [431, 214], [432, 215], [433, 215], [434, 216], [435, 216], [436, 217], [437, 217], [438, 218], [439, 218], [440, 219], [441, 219], [442, 220], [443, 220], [444, 221], [445, 221], [446, 222], [447, 222], [448, 223], [449, 223], [450, 224], [451, 224], [452, 225], [453, 225], [454, 226], [455, 226], [456, 227], [457, 227], [458, 228], [459, 228], [460, 229], [461, 229], [462, 230], [463, 230], [464, 231], [465, 231], [466, 232], [467, 232], [468, 233], [469, 233], [470, 234], [471, 234], [472, 235], [473, 235], [474, 236], [475, 236], [476, 237], [477, 237], [478, 238], [479, 238], [480, 239], [481, 239], [482, 240], [483, 240], [484, 241], [485, 241], [486, 242], [487, 242], [488, 243], [489, 243], [490, 244], [491, 244], [492, 245], [493, 245], [494, 246], [495, 246], [496, 247], [497, 247], [498, 248], [499, 248], [500, 249], [501, 249], [502, 250], [503, 250], [504, 251], [505, 251], [506, 252], [507, 252], [508, 253], [509, 253], [510, 254], [511, 254], [512, 255], [513, 255], [514, 256], [515, 256], [516, 257], [517, 257], [518, 258], [519, 258], [520, 259], [521, 259], [522, 260], [523, 260], [524, 261], [525, 261], [526, 262], [527, 262], [528, 263], [529, 263], [530, 264], [531, 264], [532, 265], [533, 265], [534, 266], [535, 266], [536, 267], [537, 267], [538, 268], [539, 268], [540, 269], [541, 269], [542, 270], [543, 270], [544, 271], [545, 271], [546, 272], [547, 272], [548, 273], [549, 273], [550, 274], [551, 274], [552, 275], [553, 275], [554, 276], [555, 276], [556, 277], [557, 277], [558, 278], [559, 278], [560, 279], [561, 279], [562, 280], [563, 280], [564, 281], [565, 281], [566, 282], [567, 282], [568, 283], [569, 283], [570, 284], [571, 284], [572, 285], [573, 285], [574, 286], [575, 286], [576, 287], [577, 287], [578, 288], [579, 288], [580, 289], [581, 289], [582, 290], [583, 290], [584, 291], [585, 291], [586, 292], [587, 292], [588, 293], [589, 293], [590, 294], [591, 294], [592, 295], [593, 295], [594, 296], [595, 296], [596, 297], [597, 297], [598, 298], [599, 298], [600, 299], [601, 299], [602, 300], [603, 300], [604, 301], [605, 301], [606, 302], [607, 302], [608, 303], [609, 303], [610, 304], [611, 304], [612, 305], [613, 305], [614, 306], [615, 306], [616, 307], [617, 307], [618, 308], [619, 308], [620, 309], [621, 309], [622, 310], [623, 310], [624, 311], [625, 311], [626, 312], [627, 312], [628, 313], [629, 313], [630, 314], [631, 314], [632, 315], [633, 315], [634, 316], [635, 316], [636, 317], [637, 317], [638, 318], [639, 318], [640, 319], [641, 319], [642, 320], [643, 320], [644, 321], [645, 321], [646, 322], [647, 322], [648, 323], [649, 323], [650, 324], [651, 324], [652, 325], [653, 325], [654, 326], [655, 326], [656, 327], [657, 327], [658, 328], [659, 328], [660, 329], [661, 329], [662, 330], [663, 330], [664, 331], [665, 331], [666, 332], [667, 332], [668, 333], [669, 333], [670, 334], [671, 334], [672, 335], [673, 335], [674, 336], [675, 336], [676, 337], [677, 337], [678, 338], [679, 338], [680, 339], [681, 339], [682, 340], [683, 340], [684, 341], [685, 341], [686, 342], [687, 342], [688, 343], [689, 343], [690, 344], [691, 344], [692, 345], [693, 345], [694, 346], [695, 346], [696, 347], [697, 347], [698, 348], [699, 348], [700, 349], [701, 349], [702, 350], [703, 350], [704, 351], [705, 351], [706, 352], [707, 352], [708, 353], [709, 353], [710, 354], [711, 354], [712, 355], [713, 355], [714, 356], [715, 356], [716, 357], [717, 357], [718, 358], [719, 358], [720, 359], [721, 359], [722, 360], [723, 360], [724, 361], [725, 361], [726, 362], [727, 362], [728, 363], [729, 363], [730, 364], [731, 364], [732, 365], [733, 365], [734, 366], [735, 366], [736, 367], [737, 367], [738, 368], [739, 368], [740, 369], [741, 369], [742, 370], [743, 370], [744, 371], [745, 371], [746, 372], [747, 372], [748, 373], [749, 373], [750, 374], [751, 374], [752, 375], [753, 375], [754, 376], [755, 376], [756, 377], [757, 377], [758, 378], [759, 378], [760, 379], [761, 379], [762, 380], [763, 380], [764, 381], [765, 381], [766, 382], [767, 382], [768, 383], [769, 383], [770, 384], [771, 384], [772, 385], [773, 385], [774, 386], [775, 386], [776, 387], [777, 387], [778, 388], [779, 388], [780, 389], [781, 389], [782, 390], [783, 390], [784, 391], [785, 391], [786, 392], [787, 392], [788, 393], [789, 393], [790, 394], [791, 394], [792, 395], [793, 395], [794, 396], [795, 396], [796, 397], [797, 397], [798, 398], [799, 398], [800, 399], [801, 399], [802, 400], [803, 400], [804, 401], [805, 401], [806, 402], [807, 402], [808, 403], [809, 403], [810, 404], [811, 404], [812, 405], [813, 405], [814, 406], [815, 406], [816, 407], [817, 407], [818, 408], [819, 408], [820, 409], [821, 409], [822, 410], [823, 410], [824, 411], [825, 411], [826, 412], [827, 412], [828, 413], [829, 413], [830, 414], [831, 414], [832, 415], [833, 415], [834, 416], [835, 416], [836, 417], [837, 417], [838, 418], [839, 418], [840, 419], [841, 419], [842, 420], [843, 420], [844, 421], [845, 421], [846, 422], [847, 422], [848, 423], [849, 423], [850, 424], [851, 424], [852, 425], [853, 425], [854, 426], [855, 426], [856, 427], [857, 427], [858, 428], [859, 428], [860, 429], [861, 429], [862, 430], [863, 430], [864, 431], [865, 431], [866, 432], [867, 432], [868, 433], [869, 433], [870, 434], [871, 434], [872, 435], [873, 435], [874, 436], [875, 436], [876, 437], [877, 437], [878, 438], [879, 438], [880, 439], [881, 439], [882, 440], [883, 440], [884, 441], [885, 441], [886, 442], [887, 442], [888, 443], [889, 443], [890, 444], [891, 444], [892, 445], [893, 445], [894, 446], [895, 446], [896, 447], [897, 447], [898, 448], [899, 448], [900, 449], [901, 449], [902, 450], [903, 450], [904, 451], [905, 451], [906, 452], [907, 452], [908, 453], [909, 453], [910, 454], [911, 454], [912, 455], [913, 455], [914, 456], [915, 456], [916, 457], [917, 457], [918, 458], [919, 458], [920, 459], [921, 459], [922, 460], [923, 460], [924, 461], [925, 461], [926, 462], [927, 462], [928, 463], [929, 463], [930, 464], [931, 464], [932, 465], [933, 465], [934, 466], [935, 466], [936, 467], [937, 467], [938, 468], [939, 468], [940, 469], [941, 469], [942, 470], [943, 470], [944, 471], [945, 471], [946, 472], [947, 472], [948, 473], [949, 473], [950, 474], [951, 474], [952, 475], [953, 475], [954, 476], [955, 476], [956, 477], [957, 477], [958, 478], [959, 478], [960, 479], [961, 479], [962, 480], [963, 480], [964, 481], [965, 481], [966, 482], [967, 482], [968, 483], [969, 483], [970, 484], [971, 484], [972, 485], [973, 485], [974, 486], [975, 486], [976, 487], [977, 487], [978, 488], [979, 488], [980, 489], [981, 489], [982, 490], [983, 490], [984, 491], [985, 491], [986, 492], [987, 492], [988, 493], [989, 493], [990, 494], [991, 494], [992, 495], [993, 495], [994, 496], [995, 496], [996, 497], [997, 497], [998, 498], [999, 498], [1000, 499], [1001, 499], [1002, 500], [1003, 500], [1004, 501], [1005, 501], [1006, 502], [1007, 502], [1008, 503], [1009, 503], [1010, 504], [1011, 504], [1012, 505], [1013, 505], [1014, 506], [1015, 506], [1016, 507], [1017, 507], [1018, 508], [1019, 508], [1020, 509], [1021, 509], [1022, 510], [1023, 510], [1024, 511], [1025, 511], [1026, 512], [1027, 512], [1028, 513], [1029, 513], [1030, 514], [1031, 514], [1032, 515], [1033, 515], [1034, 516], [1035, 516], [1036, 517], [1037, 517], [1038, 518], [1039, 518], [1040, 519], [1041, 519], [1042, 520], [1043, 520], [1044, 521], [1045, 521], [1046, 522], [1047, 522], [1048, 523], [1049, 523], [1050, 524], [1051, 524], [1052, 525], [1053, 525], [1054, 526], [1055, 526], [1056, 527], [1057, 527], [1058, 528], [1059, 528], [1060, 529], [1061, 529], [1062, 530], [1063, 530], [1064, 531], [1065, 531], [1066, 532], [1067, 532], [1068, 533], [1069, 533], [1070, 534], [1071, 534], [1072, 535], [1073, 535], [1074, 536], [1075, 536], [1076, 537], [1077, 537], [1078, 538], [1079, 538], [1080, 539], [1081, 539], [1082, 540], [1083, 540], [1084, 541], [1085, 541], [1086, 542], [1087, 542], [1088, 543], [1089, 543], [1090, 544], [1091, 544], [1092, 545], [1093, 545], [1094, 546], [1095, 546], [1096, 547], [1097, 547], [1098, 548], [1099, 548], [1100, 549], [1101, 549], [1102, 550], [1103, 550], [1104, 551], [1105, 551], [1106, 552], [1107, 552], [1108, 553], [1109, 553], [1110, 554], [1111, 554], [1112, 555], [1113, 555], [1114, 556], [1115, 556], [1116, 557], [1117, 557], [1118, 558], [1119, 558], [1120, 559], [1121, 559], [1122, 560], [1123, 560], [1124, 561], [1125, 561], [1126, 562], [1127, 562], [1128, 563], [1129, 563], [1130, 564], [1131, 564], [1132, 565], [1133, 565], [1134, 566], [1135, 566], [1136, 567], [1137, 567], [1138, 568], [1139, 568], [1140, 569], [1141, 569], [1142, 570], [1143, 570], [1144, 571], [1145, 571], [1146, 572], [1147, 572], [1148, 573], [1149, 573], [1150, 574], [1151, 574], [1152, 575], [1153, 575], [1154, 576], [1155, 576], [1156, 577], [1157, 577], [1158, 578], [1159, 578], [1160, 579], [1161, 579], [1162, 580], [1163, 580], [1164, 581], [1165, 581], [1166, 582], [1167, 582], [1168, 583], [1169, 583], [1170, 584], [1171, 584], [1172, 585], [1173, 585], [1174, 586], [1175, 586], [1176, 587], [1177, 587], [1178, 588], [1179, 588], [1180, 589], [1181, 589], [1182, 590], [1183, 590], [1184, 591], [1185, 591], [1186, 592], [1187, 592], [1188, 593], [1189, 593], [1190, 594], [1191, 594], [1192, 595], [1193, 595], [1194, 596], [1195, 596], [1196, 597], [1197, 597], [1198, 598], [1199, 598], [1200, 599], [1201, 599], [1202, 600], [1203, 600], [1204, 601], [1205, 601], [1206, 602], [1207, 602],
```

```

.....
return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
 о префиксные суммы P">
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim}>
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }}>
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={12}
 fill={C.dim}>
 <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
 </g>
 })}
 </Svg>
);
};

```

```

 });
};

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.dim}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">π</text>
 <text x={32 + idx * 33} y={100} textAnchor="middle">{idx}</text>
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {

```

```
.....

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
```

```

.....
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end" >
 <div className="absolute inset-0 bg-slate-950/80" >
 <div className="relative w-full sm:max-w-5xl max-h-400 sm:rounded-2xl shadow-2xl flex flex-col" >
 <div className="flex items-center gap-3 px-4 py-4" >
 <div className="min-w-0 flex-1" >
 <h3 className="font-bold text-white text-sm" >{viz.title}</h3>
 <p className="text-[11px] text-white" >{viz.hint} && <p className="text-[11px] text-white" >{viz.description}</p>
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700 text-white"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain" >
 <Component />
 </div>
 </div>
 </div>
 , document.body)

```

```

.....
useLayoutEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height;

 setPos({ top: Math.max(8, Math.min(top, vh - height)), left: left });
}, [anchor]);

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

```

```

.....
 <div className="text-[11px] font-mono text-
 Сложность: {entry.complexity}
 </div>
 })

 {viz && simulatorId && {
 <button
 onClick={() => onOpenSimulator(simulatorId
 className="w-full flex items-center justify
 unded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 })
 </div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 54/92: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });
}

```

```

 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
 };

 const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
 };

 const getEdgeContent = (e: any) => {

```



```

 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соеди
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алго
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mon
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.
 Старый вес: <span className="text-gr
 Вычисляем: $-2 + 0 - (-2) =$ <span cl
 </div>
);
 case 5: return (
 <div>
```



```

 {pot !== null && (
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
 </g>
)
 }}}}
 </svg>
 </div>

 <div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 ro
 justify-center leading-relaxed text-sm te
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
 -slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
 -slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(

```

```

.....
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

```

```

.....
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u })
 : [...initialEdges],
));
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(

```

```

.....
 `DFS 2: заходим в ${u} на транспонированном графе
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(

```

```

.....
 setCurrentStepIdx(0);
 };

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>

```

```

.....
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
 </marker>
</defs>

{/* Edges */}
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id ==
 const v = initialNodes.find((n) => n.id ==

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e0";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }
})

// Adjust slightly for bidirectional link

```



```

.....
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
}}
</svg>

```

```

 </div>
))
 })
 </div>
</div>

{/* List of actions log */}
<div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >

```

```

.....
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

```

```

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },

```

```

.....
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
 1));

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верш
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в остове',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветны
 type: 'info'
 }, ...prev]));
 },

```

```

},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
 setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {weight: 6} : e));
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, что создает цикл!',
 type: 'warning'
 }, ...prev]);
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {weight: 6} : e));
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе мы создадим цикл!',
 type: 'error'
 }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {weight: 7} : e));
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину E и синюю вершину F',
 type: 'info'
 }, ...prev]);
},

```

```

.....
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветную H.',
 type: 'info'
 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в остоаве',
 description: 'Вершина H перекрашена в фиолетовый.',
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединение образует цикл!',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H

```

```

.....
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую точку для следующей итерационной очереди (Heap): D-E(3), B-E(6), E-F(7)',
 type: 'info'
 }, ...prev]);
},
// Step 2: Pop D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 id: '3-4',
 status: 'in_heap'
 } : e));
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (вес 3)',
 description: 'Плесень тянется по ребру D-E к вершине D',
 type: 'info'
 }, ...prev]);
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 id: 3,
 status: 'in_heap'
 } : v));
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 id: '3-4',
 status: 'in_heap'
 } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D (5), D-F (7)',
 type: 'success'
 }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 id: '0-3',
 status: 'in_heap'
 } : e));
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины A',
 type: 'info'
 }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {

```

```

 }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину B',
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',

```



```

.....
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]));
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
},

```

```

.....
 type: 'success'
 }, ...prev]);
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Concurrently merge into Kolkhozes (Components)
 () => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
]));
 },
];

```

```

.....
 description: 'Оба колхоза объединились по мосту'
 type: 'success'
 }, ...prev]);
 },
 // Step 5: Check remaining edges & mark as rejected
 () => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шага
 description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]);
 }
};

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',

```

```

const getEdgeStroke = (status: string, color: string)
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Сунер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white" >
 Алгоритм Крускала
 </h3>
 </div>
 <p className="text-slate-400 text-sm" >
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800 rounded-lg" >
 <button
 onClick={() => handleReset('kruskal')}
 >
 Сброс
 </button>
 </div>
 </div>
 </div>
 </div>
 </div>
);

```

.....

```

<button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2
 -200 font-semibold text-xs rounded-xl border
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
</button>

```

```

<button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
 initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex + 1}
</button>

```

</div>

</div>

```

<div className="grid grid-cols-1 lg:grid-cols-3
 { /* Graph Canvas */ }
 <div className="lg:col-span-2 bg-slate-950 rounded
 active min-h-[480px] overflow-hidden">

 { /* Legend */ }
 <div className="absolute top-4 left-4 bg-slate-900

```

```
</div>
```

```
<div className="absolute top-4 right-4 bg-slate-300">
```

```
 ⚬ {stepIndex} / {currentSteps.length}
</div>
```

```
{compilerLinked && {
 <div className="absolute bottom-4 right-4 font-mono text-emerald-300">
```

```
 Python: i={liveI ?? "-"}{liveU || liveW}
 {liveW !== null ? `, w=${liveW}` : ""}
 </div>
```

```
)}
```

```
{/* SVG Edges */}
```

```
<svg className="absolute inset-0 w-full h-full aspectRatio="none">
```

```
 {liveEdges.map(edge => {
 const u = vertices[edge.u];
 const v = vertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';
```

```
 return (
```

```
 <g key={edge.id}>
```

```
 <line
```

```
 x1={u.x + "%"}>
```

```
 y1={u.y + "%"}>
```

```
 x2={v.x + "%"}>
```

```
 y2={v.y + "%"}>
```

```
 stroke={strokeColor}>
```

```
 strokeWidth={isInspecting ? 4.5 : 2}>
```

```
 strokeDasharray={isRejected ? '4,4' : ''}>
```

```
 className={"transition-all duration-200"}>
```

```
 />
```

```

.....
 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
shadow-xl z-20">
 <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'}}>
 MST построено с помощью {activeAlgo
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
 </div>
)}
 </div>

 {/* Logs / Heap */}
 <div className="bg-slate-950 rounded-2xl bord
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && (
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ⚡ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? (
 <span className="text-xs text-slate
) : (
 heapQueue.map((item, idx) => (
 <span key={idx} className={"px-2
0 shadow-md animate-pulse" : "bg-slate-800
 {item}

))
)}
 </div>
)}
)}
 }

```

```

.....
 <BookOpen className="w-5 h-5 text-indigo-400" />
 <h4 className="text-xl font-bold text-white" />
 </div>

 <div className="flex bg-slate-950 p-1 rounded" >
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 " " +
 (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300")
 }
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 " " +
 (activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-900 text-slate-300")
 }
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 " " +
 (activeCheatTab === 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-900 text-slate-300")
 }
 >
 Борувка (Билет 20)
 </button>
 </div>
</div>

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" >
 <div className="lg:col-span-1 space-y-4" >
 <div className="bg-slate-950 p-4 rounded" >
 <p className="text-slate-400 text-xs font-medium" >
 <Clock className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white" >
 Краскал
 </p>
 <p className="text-slate-400 text-xs font-medium" >
 Алгоритм Краскала
 </p>
 </div>
 </div>
 </div>
)

```



```

.....
for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </pre>
 </div>
 </div>
</div>
})

{activeCheatTab === 'prim' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" >
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-emerald-500" />
 </p>
 <p className="text-2xl font-black text-white">Прим</p>
 <p className="text-slate-400 text-xs mt-1">Минимальное остовное дерево</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-emerald-500" />
 </p>
 <p className="text-2xl font-black text-white">Авария</p>
 <p className="text-slate-400 text-xs mt-1">Минимальное остовное дерево</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-500" />
 </p>
 <p>
 <code>
 for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </code>
 </p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-500" />
 </p>
 <p>
 <code>
 for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </code>
 </p>
 </div>
 </div>
 </div>
)

```

```

.....
 <Award className="w-3.5 h-3.5 text-rose-500">
 </p>
 <p className="text-2xl font-black text-rose-500">
 <p className="text-slate-400 text-xs mt-2">
</div>
<div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
ении:</p>
 <p className="text-xs text-slate-300 leading-relaxed">
, организовав массовое распараллеленное слияние.
 </div>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
{`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост на
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = (u, v, w)
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = (u, v, w)
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)`}
 </pre>
 </div>
 </div>
</div>

```

## Универсальное пособие • полный исходный код

```
.....
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: ')). Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-400/60',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];
```

```
export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-110"/>
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc}</p>
 <div className={`mt-3 text-xs font-mono px-2 ${c.highlightColor}`}>{c.highlight}</div>
 </div>
 </div>
))}
 </div>
);
}
```

```

useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, []);

return (
 <div className="lg:hidden sticky top-[57px] z-40" >
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2 text-slate-400"
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 {currentSection}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]" >
 <div className="absolute inset-0 bg-slate-950" >
 <div className="absolute inset-y-0 left-0 w-[180px] bg-white" >
 <div className="flex items-center justify-between p-2" >

 Содержание

 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >

```

```

 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };

 return (
 <header className="bg-slate-900 border-b border-sla
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
 gap-3 lg:gap-0">
 <div className="flex items-center gap-3 w-full
 <div className="bg-gradient-to-tr from-indigo
 k-0">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-
 Подготовка к экзаменам: Алгоритмы, Структ
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-a
 >
 <button
 id="tab-guide-btn"
 onClick={() => window.scrollTo({ top: 0, be
 className="hidden sm:flex flex-1 lg:flex-no
 ite bg-indigo-600/80 shadow-lg shadow-indig
 title="Содержание пособия"
 >
 <BookOpen className="w-4 h-4" /> Учебное по
 </button>

```

```

.....

 <a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full_
download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er: bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};

```

.....  
ФАЙЛ 60/92: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....
export function PlanarityDemo() {
 return (
 <div className="space-y-4">

```

```

.....
 if (!crosses.includes(j)) crosses.push(j);
 }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x}
 stroke={xing ? "#f43f5e" : "#4f46e5"} />);
}
const labels = ["A","B","C","D","E"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
 </g>);
}
return <>{lines}{circles}</>;
})();
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [

```

```
.....
 </svg>
 <div className="absolute bottom-2 left-2 right-2
 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-.->'}
 </div>
 </div>
</div>

<div className="bg-amber-950/40 border border-amber-950">
 ▲ Запомни: K5 и K3,3 – два кита непланарности.
 гомеоморфный K5 или K3,3 – он непланарен. Теорема
</div>
</div>
);
}
```

ФАЙЛ 61/92: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
```

```
 ArrowRightLeft,
 Check,
 CheckCircle2,
 ChevronLeft,
 ChevronRight,
 Copy,
 Eye,
 HelpCircle,
 Link2,
 Loader2,
 Pause,
 Play,
 Plus,
 RotateCcw,
```



```

.....
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL:
 PYODIDE_URL
 }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/*
/* Шаблон синхронизации: короткая шапка + минимальный
/* скелет, шаг-в-шаг с подсветкой визуализации (см. da
/*

/*
/* Шпаргалка Python: команды сортировки, циклы, структ
/* «Вставить» кладёт сниппет в редактор под курсор.

```

```

.....
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i – позиция",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = push",
 code: 'st = []\nst.append("A") # push\nst.append("B") # push\nst.pop() # pop\nst.pop() # pop',
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
 code: 'from collections import deque\nq = deque()\nq.append(1)\nq.append(2)\nq.popleft()\nq.popleft()',
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: 'import heapq\nh = []\nheapq.heappush(h, 1)\nheapq.heappush(h, 2)\nheapq.heappop(h)\nheapq.heappop(h)',
 },
 {
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечность",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m] * n',
 },
 {
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)',
 },
],
}

```

```

.....
const initTemplate = useMemo(() => buildInitTemplate(

const [code, setCode] = useState(() => {
 if (!savedEditor) return initTemplate;
 if (savedEditor.code !== savedEditor.template || sa
 return savedEditor.code;
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится .
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(fals
const [runtimeMessage, setRuntimeMessage] = useState(
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);

const [desyncedFrom, setDesyncedFrom] = useState<stri
 savedEditor &&
 savedEditor.chapterId !== chapterId &&
 savedEditor.code !== savedEditor.template &&
 savedEditor.code.trim() !== "" &&
 savedEditor.code !== template &&
 savedEditor.code !== initTemplate
 ? chapterId
 : null
);
const prevTplRef = useRef({ base: initTemplate, full:

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пока
const [editHold, setEditHold] = useState<string | null
/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num

```

```

prevTplRef.current = { base: initTemplate, full: template };
setCode(initTemplate);
setDesyncedFrom(null);
setStripTab("vars");
setPlaying(false);
setEditHold(null);
setDebug(null);
setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

```

```

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать — буфер переполнен");
 }
}, [code]);

```

/\* — Пошаговый отладчик

/\*\* Связь больше не зависит от эталонных строк: совпадение строк  
const hasViz = sync.variables.length > 0;

/\*\*  
\* Снимок ПОСЛЕ выделенной строки. Контейнеры берём из state.  
\* (там уже виден append/st[i][j]=...), но индексы циклов — из state.  
\* события: после st[i][j]= Python уже мог увеличить i.  
\*/

```

const snapshotAt = useCallback((result: DebugResult, idx: number) => {
 const step = result.steps[idx];
 const next = result.steps[idx + 1];
 const afterLocals = next?.locals ?? result.finalLocals;
 const afterValues = next?.values ?? result.finalValues;
 const beforeValues = step?.values ?? {};
 const line = step?.line ? source.split("\n")[step.line - 1] : null;
 // Нас интересуют только простые скалярные цели (i, j, ...)
```

```

.....
 });

 // Единственная точка публикации: любое перемещение т
 // визуализации согласованный снимок, без side-effect
 useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de
 }, [debug, publishDebug]);

 const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

 const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

 const stepTo = useCallback((idx: number) => goDebug(i

 const stopDebug = useCallback(() => {
 setPlaying(false);
 setEditHold(code);
 setDebug(null);
 clearVizState(chapterId);
 }, [chapterId, code]);

```

```

.....
 const result = JSON.parse(String(raw)) as DebugRe

 setDebug({ result, idx: 0, src: source, input, re
 setPlaying(autoPlay && result.steps.length > 1);
 const printed = [...stdout, ...stderr].join("");
 setOutput(printed || "Программа ничего не вывела -
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STEP_
 : "Трасса готова автоматически. ← → — шаги, п
);
} catch (error) {
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, t
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код — про
} finally {
 setRunning(false);
}
}, {chapterId, code, stdin, traceRequest, running, ru

// Код выполняется сам: debounce даёт спокойно допечата
// изменился во время загрузки Pyodide, после заверше
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, {code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
 if (!debug) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {

```

```

.....
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.src
}, {debug, curDebug, snapshotAt});
const debugChanged = curDebug?.step ? changedVars(cur
const debugOrdered = curDebug?.step ? orderVars(shown

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) step
 setPlaying(true);
}, {debug, debugRun, playing, running, stepTo});

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string} => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")
 const start = ta.selectionStart ?? current.length
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, st
 const next = current.slice(0, start) + padded + c
 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + p
 });
 return next;
 });
}, []);

```

```

 }}
 >
 <div className="absolute left-1/2 top-1/2 h-1
 digo-400 transition-colors" />
 </div>
 })

 {/* Левый край меняет ширину на десктопе. */}
 {onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину п
 title="Потяните, чтобы изменить ширину панели
 className="hidden lg:block absolute top-0 bot
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 })
 >
 <div className="absolute left-1/2 top-1/2 -tr
 digo-500 transition-colors" />
 </div>
 })
 {/* — Заголовок панели —————
 <div className="shrink-0 flex items-center gap-2

```



```

.....
 <Tooltip side="bottom" content="Вернуть комме
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скрыть панель
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Закрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip
 side="bottom"
 content="Вставить полный эталонный код этого
 >
 <button
 type="button"
 onClick={() => {
 setCode(template);
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
 }}
 className="p-1.5 rounded-lg text-indigo-300"
 aria-label="Вставить эталонный код"
 >
 <Eye className="w-4 h-4" />

```

```

.....
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5`}
 stripTab === "hints" ? "bg-slate-800 text-white" : "bg-white text-slate-800"
 `}
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказка
 </button>
</div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реальный объект
а не сравнивается."
 : "На выбранной странице нет интерактивных элементов"
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1.5`}
 hasViz ? "text-emerald-400" : "text-slate-400"
 >
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визуализации"}

 </Tooltip>
 </div>
</div>

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-320">
 <div className="flex items-center gap-2 min-h-320">
 <Tooltip side="bottom" content="Открыть эскиз">
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center gap-1.5"
 >

```

```

>
 <code
 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald-300
 >
 {v.name}
 </code>
</Tooltip>
)))
{sync.stepNote && (
 <Tooltip side="bottom" content={`War :
 <span
 tabIndex={0}
 className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3
 <span className="truncate max-w-5

 </Tooltip>
)}
</div>
)}
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map((gItem) => (
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5">
 {gItem.items.map((s) => (
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-ce
 -[10.5px] text-indigo-300 hover:text-white

```

```

.....
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автотест" >
 <button type="button" onClick={() => {
 debug.idx -= 1; if (debug.idx < 0) debug.idx = 0;
 md text-slate-400 hover:text-white hover:opacity-100
 <ChevronLeft className="w-4 h-4" />
 }} />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автотест" >
 <button type="button" onClick={() => {
 debug.idx += 1; if (debug.idx > debug.result.steps.length - 1)
 debug.idx = debug.result.steps.length - 1;
 <ChevronRight className="w-4 h-4" />
 }} />
 </button>
</Tooltip>
<Tooltip content="В конец трассы" >
 <button type="button" onClick={() => {
 debug.idx = debug.result.steps.length - 1;
 <SkipForward className="w-4 h-4" />
 }} />
</button>
</Tooltip>

 {curDebug?.step ? (
 <div>
 Шаг {debug.idx + 1}/{debug.result.steps.length}
 {curDebug.step.func !== "<module>" ? curDebug.step.func : ""}
 </div>
) : (
 "шагов нет – исправьте ошибку и начните тест"
)}

{debug.result.truncated && (
 <Tooltip content="Выполнение остановлено" >

 ЛИМИТ

 </Tooltip>
)}

```

```

.....
 active ? "text-emerald-300" : "
 }` }
 >
 {active ? "▶" : ln}

 {line || " "}
</div>
);
 }}}
</div>

```

```

{/* панель переменных – «Variables», как в
<div className="shrink-0 max-h-[34%] overflow-hidden"
 <div className="flex items-center gap-1.5"
 <Variable className="w-3 h-3 text-indigo-500"
 Переменные на этом шаге
 <span className="normal-case tracking-normal"
 </div>
 {debug.result.error && {
 <pre className="mt-1 whitespace-pre-wrap"
 }}
 {debugOrdered.length !== 0 && !debug.result.error && {
 <div className="mt-1 text-[11px] text-slate-500"
 /div>
 }}
 <div className="mt-1 flex flex-wrap gap-1"
 {debugOrdered.map(([name, value]) => {
 const changed = debugChanged[name];
 const synced = syncVarSet.has(name);
 return {
 <span
 key={name}
 className={`inline-flex items-baseline gap-1.5
 changed
 ? "border-amber-500/50 bg-amber-500/10"
 : synced
 ? "border-emerald-500/40 bg-emerald-500/10"
 : "border-slate-700 bg-slate-700/10"

```

```

</label>
<input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono
/>
</div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800"
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between
 colors"
 >
 <span className="inline-flex items-center gap-2"
 <Terminal className="w-3.5 h-3.5 text-emera

 <span className="inline-flex items-center gap-2"
 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emera
 {outputOpen ? "Вывод" : "Вход"}

 </button>
 {outputOpen && (
 <pre className="flex-1 min-h-[70px] overflow-hidden text-slate-300"
 leading-5 text-slate-300">
 {output}
 </pre>
)}
</div>
<div className="px-3 py-1.5 border-t border-slate-800"

```

```

.....

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red',
 bubbleText: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'blue',
 bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
 bubbleText: 'есть сметаны, плиз.', animState: 'idle' },
]);

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: "🐍",
 color: "from-emerald-500/20 to-emerald-600/20 border",
 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string>('');

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

 // ENQUEUE: Встать в конец очереди (хвост / Tail)
 const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {

```

```

.....
// Step 1: Head guest gets eating animation
setQueue(prev => prev.map((c, idx) => idx === 0 ? {

setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue sh
 setQueue(prev => prev.map((c, idx) => idx === 0 ?

 setServedHistory(prev => [`${headGuest.emoji} ${h

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.
 }, 400);
}, 900);
}, {queue, isServing}));

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', s
 т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: ' ',
 апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: ' ', col
 без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState
 }, 500);
};

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}

```



```

>
 Сброс
</button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
 <div className={`md:col-span-9 bg-slate-900 rounded-
 -[300px] overflow-hidden ${shakeEmpty ? 'anim
 <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ &&
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 <div className="flex items-end justify-between" >

 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
 <div className="flex flex-col items-center" >
 <div className="relative" >
 <div className="absolute -top-10 left-10" >
 <div className="w-1.5 h-6 bg-slate-900" >
 <div className="w-2 h-4 bg-slate-400" >
 </div>

 </div>
 <div className="text-center" >
 <

ase font-mono tracking-wider" >
 ПОВАР

 </div>
 </div>
 </div>
 </div>

 {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}

```

.....

```

 { /* Персонаж */ }
 <div className={`
 w-14 h-14 rounded-2xl border-1
 relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 { /* Индексы HEAD/TAIL */ }
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[10px]
 ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}

)}
 </div>
 </div>
);
})
}
</div>

</div>
</div>

{ /* Пол кухни */ }
<div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-500">
</div>

{ /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Сытые гости (Архив FIFO)
 </div>

```

```
.....
 <div
 key={idx}
 className={`text-xs font-mono flex items-center`}
 >
 {idx === 0 ?
 {entry}
 } : null}
 </div>
)})
</div>
</div>
);
};
```

ФАЙЛ 63/92: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
```

```

.....
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
}
});

```

```

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

```

```

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

```

```

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;

```

```

};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить!');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный суффиксное дерево!'
);
 }
};

```

```

.....
 setCurrentNode(r);
 setSpeechText(logs);
};

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border"
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-700
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>

 {/* Верхняя панель: Говорящий лосось и диалоговое
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center"
 <div className="w-40 h-40 relative flex items-center justify-center"
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}

```

.....

```

 {isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
 </svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
 </div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

 </div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tran
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex

```

```

 { /* SVG Canvas for Automaton Tree */ }
 <div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center" >
 Дерево и Суффиксные (fail) ссылки
 </h5>

 {(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 </defs>

 { /* Draw Edges */ }
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`} >

```



```
.....
 d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
}}}
```

```
{/* Draw Nodes */}
{Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);
```

```
 let fill = '#1e293b';
 let stroke = '#475569';
```

```
 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }
}
```

```
return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
```

```

</h5>
<div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => (
 <span
 key={w}
 className="bg-slate-800 border border-
er gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:tex
 title="Удалить"
 >
 x
 </button>

))}
</div>
</div>

<form onSubmit={handleAddWord} className="fle
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.valu
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-sla
 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-60
 ition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>

```

```

: isInQueue
? 'bg-slate-800/80'
: 'hover:bg-slate-800/40'
}'}
>
<td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full`}
 node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 >

 #{node.id}
 {node.id === 0 &&
 </td>
<td className="py-2.5 px-3">

 {node.char}

</td>
<td className="py-2.5 px-3 text-slate-500" >
 {node.id === 0 ? '-' : `#${node.id}`}
</td>
<td className="py-2.5 px-3" >
 {node.id === 0 ? (

) : (

 #{node.fail}

)}
</td>
<td className="py-2.5 px-3" >

```

Универсальное пособие • полный исходный код

ФАЙЛ 64/92: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizState';
```

```
// ===== TYPES =====
```

```
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index to value
 highlightNodes: number[]; // current highlighted nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range in array
 result?: number; // partial result
 lazyState?: Record<number, number>; // node index to lazy value
 promisePushed?: number; // узел, у которого есть promise
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
export function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

## Универсальное пособие • полный исходный код

```
.....
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 state: { ...tree }, highlightNodes: [v], mergeChildren:
 });
 }
 go(1, 0, n - 1);
 steps.push({ id: id++, desc: `Дерево построено! Корень:
 Highlight: [0, n - 1] });
 return steps;
}
```

### // 2. Top-Down Query steps

```
function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v],
 });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 value: arr[v], codeLine: 4, treeState: tree, highlightNodes: [v],
 });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v],
 });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`,
 mergeChildren: [2 * v, 2 * v + 1], arrayHighlight: [v],
 });
 return res;
 }
}
```

```

 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}`,
 , arrayHighlight: [qL, qR], result: res });
 }
 }
 steps.push({ id: id++, desc: `Запрос sum(${l}..${r})`,
 ighlightNodes: [1], arrayHighlight: [qL, qR], result: res });
 return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 treeState, v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
]

```

```
};
```

```

const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",

```

```

.....
 "// целиком внутри – читаем tree[v]" // 16
};

export function genLazySteps(arr: number[], ul: number,
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};
 const lazy: Record<number, number> = {};
 for (const k of Object.keys(buildTreeFull(arr))) {
 tree[Number(k)] = buildTreeFull(arr)[Number(k)];
 lazy[Number(k)] = 0;
 }
 const full = buildTreeFull(arr);

 const snap = {desc: string, codeLine: number, hl: number,
 steps.push({
 id: id++,
 desc,
 codeLine,
 treeState: { ...tree },
 highlightNodes: {...hl},
 lazyState: { ...lazy },
 ...extra,
 });
 };
 snap(`Старт: дерево построено (корень = ${full[1]}).
 `);

function pushDown(v: number, l: number, r: number) {
 if (lazy[v] !== 0) {
 const m = (l + r) >> 1;
 const d = lazy[v];
 snap(`push(${v}): обещание +${d} с узла ${v} уходит
 lazy.` , 8, [v, 2 * v, 2 * v + 1], { promisePush: true });
 tree[2 * v] = (tree[2 * v] ?? 0) + d * (m - l + 1);
 lazy[2 * v] = (lazy[2 * v] ?? 0) + d;
 tree[2 * v + 1] = (tree[2 * v + 1] ?? 0) + d * (r - m);
 lazy[2 * v + 1] = (lazy[2 * v + 1] ?? 0) + d;
 }
}

```

```

}
if (uL <= l && r <= uR) {
 res += tree[v] ?? 0;
 snap(`query(v=${v}, [${l}..${r}]): целиком внутри
 sult: res));
 return;
}
pushDown(v, l, r);
const m = (l + r) >> 1;
snap(`query(v=${v}, [${l}..${r}]): частичное пересече
);
query(2 * v, l, m);
query(2 * v + 1, m + 1, r);
}
res = 0;
snap(`Теперь запрос sum([${uL}..${uR}]) – проверим, ч
query(1, 0, n - 1);
snap(` sum([${uL}..${uR}]) = ${res} – совпадает с че
return steps;
}

```

// ===== PLAYER COMPONENT =====

```

function Player({ steps, color }: { steps: Step[]; colo
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [s

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx :
 const t = setTimeout(() => setIdx(i => i + 1), speed
 return () => clearTimeout(t);
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;
 }

```



```

.....

 [{nd.left}

 {promise !== 0 && (
 <span className="mt-0.5 px-1 rounded-full border-b
 o whitespace-nowrap">
 +{promise}

)}
 </div>
 ({nd.left || nd.right} && (
 <div className="flex flex-col items-center mt-1">
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full">
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
</div>
);
}, [step, clr, liveNode]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 } /* Header */>
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-bottom">
 <h4 className="font-bold text-sm text-white flex-grow-1">

 </div>

 { /* Array strip */ }
 <div className="px-3 py-2 bg-slate-950 border-bottom border-l
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (

```

```

.....
 ' : color === 'emerald' ? 'text-emerald-300
 = {step.result}

 })
</div>

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t
 <div className="flex items-center bg-slate-900
 <button onClick={() => { setPlaying(false); s
 unded" title="Сброс"><RotateCcw className="
 <button onClick={() => { setPlaying(false); s
 0 hover:text-white hover:bg-slate-800 round
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-5
 {playing ? <Pause className="w-3 h-3" /> :
 </button>
 <button onClick={() => { setPlaying(false); s
 .5 text-slate-400 hover:text-white hover:bg
 </button>
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Тыбы</option>
 </select>
</div>

{/* Progress bar */}
<div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

```

## Универсальное пособие • полный исходный код

```
.....
const queryTDDSteps = useMemo(() => genQueryTopDownSteps)
const queryBUSTeps = useMemo(() => genQueryBottomUpSteps)
// Питон умеет рулить добавкой: переменная add из кода
const effAdd = liveAdd ?? uAdd;
const lazySteps = useMemo(() => genLazySteps(arr, uL,

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-
 /* ---- TOP BAR: array editor + query range ---- */
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 /* Array input */
 <form onSubmit={handleApply} className="flex-1">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">
 label>
 <button type="button" onClick={randomize} className="text-white">
 "><RefreshCw className="w-3.5 h-3.5" /></button>
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2">
 Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (
 <span key={i} className={`border px-2 py-1 text-white text-sm
 ext-amber-200 ring-2 ring-amber-400/30`} : "${v}">


```

```

 </div>
 </div>

 {/* ---- TABS ---- */}
 <div className="flex flex-wrap items-center gap-1" >
 <button onClick={() => setView('build')} className="border-indigo-500 border-2 rounded p-2 text-slate-200" >
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} className="border-emerald-500 border-2 rounded p-2 text-slate-200" >
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('lazy')} className="border-amber-500 border-2 rounded p-2 text-slate-200" >
 <div>Массовое обновление на отрезке (Lazy)</div>
 </button>
 <button onClick={() => setView('theory')} className="border-blue-500 border-2 rounded p-2 text-slate-200" >
 <Info className="w-3.5 h-3.5" /> Почему / Сра
 </button>
 </div>

 {/* ---- TAB CONTENT ---- */}
 {view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=""
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

.....
 <div className="flex items-center gap-2">
 <label className="text-[10px] uppercase t
 <input type="number" value={effAdd} onChange
 border-slate-700 text-white text-xs font-m
 l ? 'задана переменной add в Python-компиля
 </div>
 <p className="text-[11px] text-slate-500 ma
 Ко всем элементам отрезка прибавляем <b с
 емся только по границе, а внутри вешаем сти
 суммы).
 {liveAdd !== null && <span className="ml-1
 {liveV !== null && <span className="ml-2
 </p>
 </div>
 <SimPanel
 key={`lazy-${arr.join('-')}-${uL}-${uR}-${e
 title={`update([${uL}..${uR}], +${effAdd})`
 icon="
 steps={lazySteps}
 code={LAZY_CODE}
 color="amber"
 arr={arr}
 liveNode={liveV}
 />
</div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 /* Why it splits this way */
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="text-base font-bold text-whi
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leadin
 Каждый узел берёт <code className="bg-sla
 к получает <code className="bg-slate-900 px
 -slate-900 px-1 rounded font-mono text-емег
 евая часть забирает на 1 больше (округление

```

Стартуем с двух указателей (l и r) на левом и правом краях массива. Сдвигаем указатель l до тех пор, пока элемент на его левом соседе не будет меньше элемента на его левом соседе. Поднимаемся пока l {".

</p>

<div className="space-y-1.5 text-xs">

```
<div className="flex justify-between border-top border-bottom border-gray-200" data-bbox="111 820 604 853">
```

```
<div className="flex justify-between bo
><span className="text-emerald-400 font-mon
```

```
<div className="flex justify-between"><
00 font-bold">x сложно</div>
```

```

{ /* Verdict */

```

<div className="bg-slate-950 p-5 rounded-xl b

#### className="font-bold text-blue-400 mb-2

<p className="text-sm text-slate-300 leading

```
<strong className="text-white">Итеративный
кальности. Но если нужны <strong className=
ame="text-white">рекурсивный (сверху-вниз)<
```

</p>

17

1.

1

ФАЙЛ 65/92: `src/components/Sidebar.tsx`

**КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО**

```
import React, { useMemo, useState } from "react";
```

```
import { ChevronRight, Compass, Search, Sparkles, X } from 'lucide-react';
```

```
import { chapters } from "../data/content";
```

```

.....
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4" >
 {groups.length === 0 && <p className="text-sm t

 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold upperc
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selec
 const hasViz = Boolean(VIZ_REGISTRY[cha
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior
 }}
 className={`w-full text-left px-3 p
 isSelected
 ? "bg-indigo-600/15 border-indi
 : "bg-slate-800/40 border-transp
 `)}
 >
 {hasViz && {

```

## Универсальное пособие • полный исходный код

```
.....
.....

import React, { useState } from "react";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 * (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего дерева
 * поворот – кадры «прицел → отстёгиваем среднее по высоте»
 * 3. «Собери сам» – песочница: подними узел кликами, разбей
 */
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
 const [tab, setTab] = useState<Tab>(defaultTab);

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border-1 border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white rounded">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Один
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {[
 { id: "cases" as Tab, label: "1 · Механика: Zig / Zig-Zig / Zig-Zag" },
 { id: "big" as Tab, label: "2 · Большое дерево" },
 { id: "play" as Tab, label: "3 · Собери сам" }
]}
```



## Универсальное пособие • полный исходный код

```
.....
/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево
 * поднять отмеченный узел в корень. Клик по узлу = один
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null, right: null });

const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;

/** Заготовки: линия (zig-zig), змейка (zig-zag) и просечка (zig-zag-zig) */
export const PRESETS: { name: string; hintCase: string; target: number; build: () => TNode } = [
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 const n6: TNode = { key: 6, left: n4, right: leaf(7) };
 return { key: 8, left: n6, right: leaf(9) };
 },
 },
 {
 name: "Просечка",
 hintCase: "Zig-Zag-Zig",
 target: 5,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 const n6: TNode = { key: 6, left: n4, right: leaf(7) };
 const n8: TNode = { key: 8, left: n6, right: leaf(9) };
 return { key: 10, left: n8, right: leaf(11) };
 },
 },
];
```

Универсальное пособие • полный исходный код

```
.....
/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}
```

```

.....

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.tree);
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; correct: boolean }>([]);
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() => preset.build(), [presetIdx]);
 const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
 const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)), [tree, target]);

 const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
 };

```

```

 }` }
 >
 {p.name}
 </button>
)))
 </div>

 <div className="bg-slate-900 rounded-xl border bo
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420)
 role="img"
 >
 {edges.map(([, a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-b
 />
);
 })}
 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g

```

```

.....
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-2
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-200"
 }`
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

 </div>

 {showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"
 }`
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500" /> : <XMark className="w-4 h-4 text-rose-500" />}
 </div>
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500" /> : <XMark className="w-4 h-4 text-rose-500" />}
 </div>
 </div>
)}

```

## Универсальное пособие • полный исходный код

```
.....
*
* Что сделано ради понятности:
* • у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
* сразу видно, что дерево остаётся деревом поиска;
* • под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
* это и есть доказательство, что поворот ничего не меняет;
* • кадр «прицел» ничего не двигает, только подсвечивает;
* • на кадре «результат» остаются призраки – пунктирные узлы
* местах и стрелки «откуда → куда»;
* • по умолчанию анимация НЕ крутится сама: пользователь
* идёт в своём темпе. Автопрокрутка включается кнопкой.
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Папа, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
```

```

.....
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддерезо
 ева от 40 – это одно и то же место.",
 moved: [{"x", "p", "B"}],
 ms: RESULT_MS,
 },
],
};

/* Zig-Zig
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: {
 {
 ...ZZ_S0,
 kind: "start",

```

```

.....
 ms: RESULT_MS,
 },
],
};

/* Zig-Zag
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте

```



Универсальное пособие • полный исходный код

```
.....
 sub: { fill: "#1e293b", stroke: "#475569" },
};
```

```
const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};
```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```
const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? {})
```

```
 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>
```

```
 { /* призраки: где узел стоял до поворота */ }
```

```
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill
 {len > pad * 2 + 6 && (
 <line
```

```

.....
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id))
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size={10}
 >{id}</text>
)}
 </g>
);
 }
}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

```

```

.....
<div className="flex gap-2 mb-4 overflow-x-auto"
 {CASES.map((c, i) => {
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 i === caseIdx ? "bg-indigo-600 text-white"
 : ""}
 >
 {c.label}
 </button>
 })}
</div>

<div className="mb-3 text-xs sm:text-[13px] text-
 Кор,
 {{active.rotat
</div>

{/* шаг + пояснение стоят НАД картинкой: сначала
<div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`
 >
 {current.kind === "aim" ? "прицел" : current

 <span className="text-sm font-bold text-white"
 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading

```

```

.....
<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800 i
 d:hover:text-slate-300 text-xs font-bold tr
 >
 ← Назад
 </button>
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 t
 : "bg-slate-800 border-slate-700 text-sla
 `}
 >
 {playing ? "Медленно" : "Быстро"}
 </button>
</div>

```

## Универсальное пособие • полный исходный код

```
.....
import React, { useEffect, useMemo, useRef, useState }
import { useVizRuntime, vizNumber } from "../data/vizSt

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный пов
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, по
 * rot – поворот: зелёным подсвечены рёбра, которых н
 * Рядом с логом всегда висит врезка «Механика одного п
 * На каждом кадре – живая автопроверка in-order (сорти
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, col
 */

export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | nu
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null,
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}
```

```

.....
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]): string[] {
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}→${c}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" | "rebalance";
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
}

```

```

.....
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск уперся в ${last.key}
 rotCount: 0,
 });

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string,
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 note: aimNote,
 rotCount,
 }));
const p = child.parent!;
const mid = p.left === child ? child.right : child.left;
steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `β = поддерезо ${mid.key}` : "β не найдено",
 cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : null,
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерезо β = ${mid.key}
 и у них общий — после поворота переедет ${p.left.key}

```

```

.....
 `${g.key}`, а `${target.key}` пока не трогаем.`),
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро `${target.parent.key}` над `${target.key}`,
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: `${target.key}` над `${target.parent.key}`
);
 } else {
 const side = `${g.key} → ${p.isLeft ? "влево" : "вправо"}`;
 emitRotation(
 target,
 "Zig-Zag",
 [p.key, target.key],
 `Zig-Zag (змейка: `${side}`). Порядок ОБРАТНЫЙ: п
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: `${target.parent.key}`—`${target.key}`,
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zag",
 [target.parent.key, target.key],
 `Доводим: `${target.key}` над `${target.parent.key}`
);
 }
}

```



## Универсальное пособие • полный исходный код

```
.....
{ line: 3, text: " if g is None: # Z
{ line: 4, text: " rotate(x over p)" },
{ line: 5, text: " elif p,g,x на одной линии: #
{ line: 6, text: " rotate(p over g) # B
{ line: 7, text: " rotate(x over p)" },
{ line: 8, text: " else: # Z
{ line: 9, text: " rotate(x over p) # H
{ line: 10, text: " rotate(x over g)" },
};
```

```
/* врезка «механика одного по
```

```
const MechanismInset: React.FC<{ step: SplayStep | null
const isCut = step?.kind === "cut";
const isRot = step?.kind === "rot";
return {
 <div className="bg-slate-950/80 rounded-lg border b
 <h4 className="text-xs font-bold text-slate-300 m
 <p className="text-[10px] text-slate-500 mb-2">
 Как вправление вывиха: одна кость ходит вокруг ,
 </p>
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fon
 <line x1="55" y1="46" x2="30" y2="86" stroke=
 <line x1="55" y1="46" x2="80" y2="86" stroke=
 <line x1="30" y1="86" x2="12" y2="126" stroke=
 <line
 x1="30" y1="86" x2="48" y2="126"
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 3 : 2}
 strokeDasharray={isCut ? "5,3" : "none"}
 />
 <circle cx="55" cy="46" r="15" fill="#1e293b"
 <text x="55" y="50" textAnchor="middle" fill=
 <circle cx="30" cy="86" r="15" fill="#78350f"
 <text x="30" y="90" textAnchor="middle" fill=
 <circle cx="12" cy="126" r="13" fill="#0f172a
```

```

.....
 />
 <text x="63" y="130" textAnchor="middle" fill="black">
 <circle cx="95" cy="126" r="13" fill="#0f172a" stroke="black" />
 <text x="95" y="130" textAnchor="middle" fill="black">
 <text x="112" y="92" fill={isRot ? "#10b981" : "black"}>
 {isRot ? "↓ β теперь при p" : "β при p"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize=14>
 </g>
</svg>
<p className="text-[10px] text-slate-500 leading-5">
 Обход слева-направо α x β p γ не меняется никогд
 числа поворотов. Единственный, кто меняет хозяин
</p>
</div>
);
};

/* рендер

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ??
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 });
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(({[
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py

const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
const hBefore = useMemo(() => (steps ? heightOf(steps
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {

```

```

.....
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шанка */}
 <div className="flex flex-wrap items-center justify"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white"
 <p className="text-sm text-indigo-300">Клик
 </div>
 </div>
 <div className="flex flex-wrap items-center gap"
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current)
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 placeholder="ключ + Enter"
 className="w-28 bg-slate-950 border border-
 s:border-indigo-500"
 />

```



```

.....
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}}}
</svg>

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row"
 <div className="flex-1 bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "× Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Клики по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "
 {checkOk ? " " : " "} проверка кадра: 0
 {inorderKeys(viewRoot).length}
 {movedCount > 0 ? ` · переехало рёбер: `
 </p>
 }}

```

```

.....
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig"
 const zz2 = step?.caseLabel === "Zig-Zig"
 const za1 = step?.caseLabel === "Zig-Zag"
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 |
 return (
 <div key={\line} className={`py-1 px-2
ber-400" : "text-slate-400"}`>
 <span className="text-slate-600 selec
 {\text}
 </div>
);
 }
}
</div>
</div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 70/92: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSte

```

```

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;

```

## Универсальное пособие • полный исходный код

```
.....
const [cleanRack, setCleanRack] = useState<Plate[]>([
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
```



```

.....
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: 🍷,
 imState: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: 🍰,
 mState: 'in' },
]));
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
};

```

```

const topIndex = displayStack.length - 1;

```

```

return (
 <div className="flex flex-col gap-6">
 /* МНЕМОНИКА / ПРАВИЛО */
 <div className="bg-blue-950/40 border border-blue-950/40 p-4">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl"> ПРАВИЛО
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именную тарелку.
 Попробуешь вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>
 </div>
 /* УПРАВЛЕНИЕ */
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 disabled:cursor-not-allowed text-white font-medium rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>
 </div>
 </div>
);

```

```

.....
 { /* Мыльные пузыри при мытье */ }
 { showSoap && {
 <div className="absolute inset-0 pointer-events-none">
 <div className="absolute w-6 h-6 rounded-full bg-white border-1 border-gray-200">
 <div className="absolute w-4 h-4 rounded-full bg-white border-1 border-gray-200">
 <div className="absolute w-5 h-5 rounded-full bg-white border-1 border-gray-200">
 <div className="absolute text-4xl anim-curve">
 </div>
 </div>
 </div>
 </div>
 </div>
 } }

 { displayStack.length !== 0 ? (
 <div className="flex-1 flex flex-col items-center">

 <p className="text-white font-black text-4xl">
 <p className="text-slate-500 text-xs mt-1">
 </p>
 </p>

 </div>
) : (
 <div className={`flex-1 flex flex-col justify-center`} >

 { /* Указатель TOP */ }
 <div
 className="absolute right-8 flex items-center"
 style={{ bottom: `${24 + topIndex * 44}px` }}
 >

 er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>

 { /* Тарелки (отрисовываем в обратном порядке) */ }
 { displayStack.slice().reverse().map((plate) => {
 const idx = displayStack.length - 1 - r

```

```

.....
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 ЧИСТО
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center" >

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь будет
 </div>
) : (
 <div className="flex flex-col gap-2 w-full" >
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-0.5"
 >
 {plate.name}

 </div>
 </div>
))}
 </div>
)}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" >
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800 p-4" >
 <div className="text-[10px] font-mono text-slate-400" >
 {log.map((entry, idx) => (
 <div
 key={idx}

```

```

.....
 яд назад: откат j = pi[$\{j - 1\}$] = $\{pi[j - 1]\}$;
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадения нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни

```

```

.....
const [text, setText] = useState("abacaba");

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPa
 if (mode === "kmp") return generateKmpSteps(safe
 else return generateZSteps(safePattern, safeText
}, {mode, safePattern, safeText});
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const runtimePattern = vizString(runtime?.variables
const s = runtimeString ?? demoString;
// Длина образца: приоритет – переменная pattern из
// разделителя "#" в склейке pattern#text; иначе вс
const patternLength = runtimePattern
 ? runtimePattern.length
 : runtimeString
 ? (runtimeString.includes("#") ? runtimeString
 : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set

```

```

.....
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")}
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2"
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange=
xt-white text-xs font-bold font-mono px-2 p
 </div>
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
 </div>
 </div>
</div>

<div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-ful
 {s.split("").map((char, index) => {
 const isPattern = index < patte
 const isSeparator = index === p

 let borderColor = "border-slate
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "

 // KMP Logic
 if (mode === "kmp") {

```

.....

```

 { /* Value array box */
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {(mode === "kmp" ?
 </div>

```

```

 { /* KMP fingers */
 {mode === "kmp" && step
 <div className="abs
z-20">

```

```

 <span className
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
 <span className
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
20">

```

```

 <span className
 </div>
 })

```

```

 { /* Z fingers */
 {mode === "z" && step.i
 <div className="abs
e z-20">

```

```

 <span className
 </div>
 })
 {mode === "z" && step.z
 <div className="abs
 <span className
 </div>
 })

```

```

.....
 <RefreshCw strokeWidth={3} size={24} />
 </button>
 </div>

 { /* Description Log */ }
 <div className="flex-1 bg-slate-900/50 overflow-hidden">
 <div className="absolute top-0 right-0">

 {steps[steps.length - 1].desc}

 </div>
 <div className="text-[10px] text-slate-400">
 {steps.length}
 </div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>

 { /* Why this matters card */ }
 <div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "проблема не была найдена!" : "проблема найдена!"}
 Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z).
 </p>
 </div>

 { /* Code Sneak Peek */ }
 <div className="bg-slate-900 border border-slate-800">
 <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
 Код C++ {mode === 'kmp' ? 'КМП' : 'ЗКМП'}
 </div>
 <pre className="p-4 text-xs font-mono text-white">

```



```

.....

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content, side }) => {
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

 return (
 <span
 className={`relative inline-flex ${className ?? ""}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2 top-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px] font-normal
 duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full mt-2"
 } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
 >
 {content}
 <span
 className={`absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
 side === "top" ? "bottom-[5px] border-b border-slate-900" : ""
 }`
 />

);
 };
};

```

```

.....
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];

interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});

const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 const step: TStep = {
 desc,
 visited,
 fullyProcessed,
 currentNode,
 topoList,
 dfsStack,
 };
 listSteps.push(step);
 setSteps(listSteps);
 };
 recordStep('Start', 0);
 while (isPlaying) {
 const nextStepIdx = getNextStepIdx(listSteps, currentStepIdx);
 if (nextStepIdx === null) break;
 setCurrentStepIdx(nextStepIdx);
 const step = listSteps[nextStepIdx];
 step.currentNode = step.dfsStack[0];
 step.dfsStack.shift();
 if (step.dfsStack.length === 0) {
 step.fullyProcessed.add(step.currentNode);
 step.currentNode = null;
 recordStep(`${step.desc} finished`, step.currentNode);
 }
 }
 setIsPlaying(false);
}, [vars.v, vars.order]);

```

.....

```

 fullyProcessed.add(u);
 dfsStack.pop();
 topoList.push(u); // prepend behavior: we write i
 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,
);
 };
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

```

```

500 text-white"
>
 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
</button>

<button
 onClick={reset}
 className="flex items-center justify-center
ors"
>
 <RotateCcw className="w-4 h-4" />
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12
 { /* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]"
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"

```

```

.....
 const isVisited = step.visited.has(node.id);
 const isProcessed = step.fullyProcessed.has(node.id);

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y - 4}

```

```

.....
 { /* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРУТ ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded-2xl pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>

 { /* Color key */}
 <div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2 text-[10px]">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2 text-[10px]">

 Черные (готово)

 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

```

## Универсальное пособие • полный исходный код

```
.....
* Интерактивный Tgear – шесть режимов:
* build – сборка: ВИДИМАЯ сортировка точек (и чем с
* координатная плоскость НАВЕРХУ (ассоциаци
* чек-лист инвариантов (автопроверка, как у
* split – разрез по x с двумя корзинами (грамотная
* merge – склейка двух деревьев по y.
* erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
* layers – миф «дети в массиве по $2k/2k+1$ »: работает
* search – «отсортирую бинарным поиском»: поиск не с
* сортировка платит $\geq n \cdot \log n$ сравнений, со
*
* Канонический набор: все x и все y различны $\Rightarrow$ дерево
* Узел идентифицируется парой $id = "x:y"$.
*/
```

```
export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
```

Универсальное пособие • полный исходный код

```
.....
 trace: { atId: NodeId; goLeft: boolean; note: string
}
export function splitTree(root: TNode | null, x: number) {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null;
}
export function mergeTree(a: TNode | null, b: TNode | null) {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
 note: `Объединено целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
 note: `Выбрано левое поддерево с (${b.key}, ${b.grade})` });
 return mergeTree(a, b.right);
 } else {
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
 note: `Выбрано правое поддерево с (${a.key}, ${a.grade})` });
 return mergeTree(b, a.left);
 }
 };
 const res = go(a, b);
 return { root: res, trace };
}
```



```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null) => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

/\* ----- раскладка сцены дерева ----- \*/

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 10 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;

```

```

 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
 strokeWidth={2.5}
 strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 }
}
}
{nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`} >
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)} stroke={pulse ? "#ccc" : "#000"} strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
 <text y={-2} fill="#fff" fontSize="13" font-weight="bold">{n.id}</text>
 <text y={13} fill="#cbd5e1" fontSize="10">{n.name}</text>
 </g>
);
});
}
</svg>
{subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) ----- */

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 60) / 10);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 8);

```

```

{drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.b.key === e.b.key) || (badEdge.a.key === e.b.key && badEdge.b.key === e.a.key));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} strokeDasharray={isBad ? "4,3" : "none"} />
})}

{/* временная линия игры */}
{interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDasharray={[]} />
)}

{points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key));
 const isSel = selected && selected.key === p.key;
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ? (e) => onPointDown(e, p) : ()}
 onPointerUp={interactive && onPointUp ? (e) => onPointUp(e, p) : ()}
 style={interactive ? { cursor: "pointer" } : {}}
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"} strokeWidth={2} />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12} font-weight="bold">{p.key}</text>
 </g>
);
})}

```

Универсальное пособие • полный исходный код

```
.....
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur)).a;
 }
 return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа
export function forestIsBst(edges: GEdge[], all: Pair[]) {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || [])]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
 const ok = (n: Pair | undefined, lo: number, hi: number) => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n.a)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 const oneParent = new Set(edges.map((e) => gpid(e.b)));
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === gpid(p.a)));
 const complete = edges.length === all.length - 1 && roots.length === 1;
 return { heap, oneParent, acyclic, bst, complete, win };
}

/** Попытка хода: почему нельзя – или ОК. */
```

```

 }
 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
 }
 const xs = pairs.map((p) => p.key);
 const ys = pairs.map((p) => p.grade);
 return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
 a: Pair;
 b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
 edges: GameEdge[],
 x: Pair,
 y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
 if (x.key === y.key && x.grade === y.grade) return { ok: true };
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 (pid(e.a) === pid(y) && pid(e.b) === pid(x)))
 return { ok: false, reason: "такая линия уже проведена" };
 }
 if (x.grade === y.grade) return { ok: false, reason: "не может быть" };
 const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки (${child.key}, ${child.grade}) уже есть родитель` };
 // цикл: родитель уже сидит в компоненте ребёнка (ребёнок не может быть своим родителем)
 const childComp = new Set<string>([pid(child)]);
 let grew = true;

```

```

.....
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; з
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяе

```

```

.....
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
 };
 return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d");
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | null>();
 const live = useTreapLive();

```

```

 const parsed = useMemo(() => parsePairs(raw), [raw]);
 const userPairs = parsed.errors.length ? [] : parsed.pairs;

```

```

 const n = userPairs.length;

```

```

 const userTree = useMemo(() => (userPairs.length >= 1 ? buildTree(userPairs) : null), [userPairs]);
 const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left }, b: node }); }
 if (node.right) { es.push({ a: { key: node.key, group: node.group, right: node.right }, b: node }); }
 };
 walk(userTree);
 return es;
 }, [userTree]);

```

```

.....
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair, gameEdges));
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(gameEdges) : null));

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};

const pointDown = (p: Pair) => {

```



```

.....
 { /* Произвольный список */ }
 <div className="bg-slate-900/60 rounded-xl p-4 bo
 <div className="flex flex-wrap items-center jus
 <h4 className="text-sm font-bold text-slate-3
 <button onClick={() => { setRaw(DEMO_RAW); se
 py-1 rounded-lg text-xs font-bold bg-slate
 демо-набор
 </button>
 </div>
 <textarea
 value={raw}
 onChange={(e) => { setRaw(e.target.value); se
 rows={2}
 className="w-full bg-slate-950 border border-
 s:border-emerald-500"
 placeholder="3 5, 45 8, 1 -4 ..."
 />
 {parsed.errors.length > 0 && {
 <div className="mt-2 text-xs text-rose-300 bg
 {parsed.errors.map((e, i) => <div key={i}>X
 </div>
)}
 {parsed.errors.length === 0 && (parsed.equalX |
 <div className="mt-2 text-xs text-amber-300 b
 {parsed.equalY && <div>⚡ Есть равные y
 кто раньше в списке (сортировка ста
 {parsed.equalX && <div>⚡ Есть равные x
 de.key → left).</div>}
 </div>
)}
 </div>

 { /* Координатная плоскость — НАВЕРХУ */ }
 <div className="bg-emerald-950/20 rounded-xl p-4
 <div className="flex flex-wrap items-center jus
 <h4 className="text-base font-bold text-emera
 <div className="flex items-center gap-2">
 {phase === "sort" ? {

```

```

.....
 соединить сам
 </button>
 </>
 })
</div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} пар
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: сл
 {comparisons} ($n \cdot \log_2 n$).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не
 Игру </>
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказа

```

```

.....
 <button onClick={undoEdge} disabled={!gameEdge}
 white disabled:opacity-30">↶ Убрать линию</button>
 <button onClick={resetGame} disabled={!gameEdge}
 bg-emerald-700 text-white disabled:opacity-30"> Сброс
 <button onClick={showSolution} disabled={!gameEdge}
 teal disabled:opacity-30"> Показать решение</button>
 </div>
)}
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-4">
 <h4 className="text-sm font-bold text-emerald-50">Победное
 <TreeView root={gameTree} subtitle="Слева направо
 существует." />
 </div>
)}

{/* Дерево */}
{order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">Пользователь
 <TreeView root={userTree} subtitle="Слева направо
 существует." />
 </div>
)}
</div>
);
};

```

/\* ~~не выводить результат сразу, а выводить его по мере необходимости~~ РЕЖИМ SPLIT ~~не выводить результат сразу, а выводить его по мере необходимости~~ \*/

```

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
 const split = useMemo(() => splitTree(buildTree(POINTS), fullTree, x0), [fullTree, x0]);
 const steps = split.trace;

```

```

 </div>

 <div className="text-xs text-slate-300 bg-slate-900"
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2"
 <div className="bg-sky-950/20 rounded-xl p-3"
 <h5 className="text-sm font-bold text-sky-300"
 <TreeView root={lTree} width={480} height={480} />
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3"
 <h5 className="text-sm font-bold text-rose-300"
 <TreeView root={rTree} width={480} height={480} />
 </div>
 </div>
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900"
 <LiveChip live={live} />{' '}
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой — д
 </div>
 </div>
);
};

```

*/\* ~~you can test this document in the browser with the following code~~ РЕЖИМ MERGE ~~the code that test this document in the browser with the following code~~ \*/*

```

const MergeMode: React.FC = () => {
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 // Компилятор: i – номер шага слияния.
 useEffect(() => { if (live.i !== null) setStepIdx(live.i); }, [live.i]);
 const { l, r } = useMemo(() => splitTree(buildTree(P0, live.i)), [live.i]);
 const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
 const steps = merged.trace;

```

```

.....
 <TreeView root={merged.root} subtitle="merge("
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-900" style={bgStyle}>
 Грамотно про «слияние»: у корней сравнения
 в (то, что сохраняет порядок x) отправляется ,
</div>
</div>
);
};

```

*/\* ----- РЕЖИМ ERASE ----- \*/*

```

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
 const res = useMemo(() => eraseNode(buildTree(POINTS), key), [key]);
 const steps = res.trace;
 const live = useTreapLive();
 // Компилятор: x – ключ удаляемой вершины, i – шаг слияния
 useEffect(() => { if (live.x !== null && POINTS.some((p) => p.id === x)) setKey(x); }, [live.x]);
 useEffect(() => { if (live.i !== null) setStepIdx(Math.min(stepIdx, live.i)); }, [live.i]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId !== null) ids.push(s.atId);
 return ids;
 }, [steps, idx]);
 const pathSet = new Set(pathIds);

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить

```

```

.....
 ягивает merge двух детей. Удаляется любая
 </div>
</div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null);
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
 }, [full]);
 const nodeCount = slots.filter(Boolean).length;

 return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="
 ext-white" : "bg-slate-700 text-slate-300">`>
 <button onClick={() => setPick("treap")} className="
 text-white" : "bg-slate-700 text-slate-300">
 </div>

 {pick === "heap" ? (
 <div className="bg-slate-900/40 rounded-xl p-4

```

```

 <div className="text-xs text-slate-400 bg-slate-900 p-4" style="border: 1px solid #333; border-radius: 5px; margin-top: 10px;">
 Грабля «дети в массиве по $2k/2k+1$ »: формирова
 ивым: слои дырявые, и «адрес» ребёнка посчита
 </div>
 </div>
);
};

```

/\* РЕЖИМ SEARCH (поиск ≠ сортировка) :

```
const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО пр
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
 const target = 7;
 const sortedByKey = [...shuffled].sort((a, b) => a.key
 const [stage, setStage] = useState<0 | 1 | 2>(0); //
 const [bidx, setBidx] = useState(0);

 // шаги бинарного поиска на отсортированном массиве д
 const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number;
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);
 if (sortedByKey[mid].key === target) { steps.push
 нашли за ${steps.length + 1} сравнение(й).` });
 if (sortedByKey[mid].key < target) { steps.push({
 } – вся левая половина выброшена.` }); lo = mid
```

```

.....
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
 });
 }}}
</div>

<div className="text-xs text-slate-300 bg-slate-900"
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарни
</div>

<div className="text-xs text-slate-400 bg-slate-900"
 Есть ли разница, чем сортировать? Сравн
 ная граница, каждое сравнение даёт 1 бит. Но
 ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.
</div>
</div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);
 const [hover, setHover] = useState<Pair | null>(null);
 const [msg, setMsg] = useState<string>("Кликни точку-
 ");

```



```

<div className="bg-emerald-950/20 rounded-xl p-4" style={{width: 100%; height: 100%;}}>
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} className="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="none" stroke="black" stroke-width="1"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1"/>

 {edges.map((e, i) => (
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399" : "black"}
 strokeWidth={i === edges.length - 1 ? 3.5 : 1}
 className="transition-all duration-300"
 />
))}

 {POINTS.map((p) => {
 const isSelected = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) && !isSelected;
 return (
 <g key={`gp-${p.key}:${p.grade}`} onClick={() => {
 if (isHover) hover = p;
 if (isSelected) sel = p;
 }} className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)} r="10" fill="none" stroke="black" stroke-width="1"/>
 <circle cx={PX(p.key)} cy={PY(p.grade)} r="10" fill={isHover ? "#94a3b8" : "#a5b4fc"} stroke="black" stroke-width="1" transition="all duration-200" />
 <text x={PX(p.key)} y={PY(p.grade) + 30} fill="white" font-size="12" font-weight="bold">{p.key}</text>
 <text x={PX(p.key)} y={PY(p.grade) + 24} fill="white" font-size="12" font-weight="bold">{p.grade}</text>
 </g>
)
 })}
 </svg>
</div>

```

```
.....
 { id: "game", label: "Игра" },
];

/** Значения из Python-компилятора: x — ключ, y — приоритет
function useTreapLive(): { x: number | null; y: number } {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const x = vizNumber(vars?.x);
 const y = vizNumber(vars?.y);
 const i = vizNumber(vars?.i);
 return { x, y, i, linked: x !== null || y !== null || i !== null };
}

const LiveChip: React.FC<{ live: { x: number | null; y: number | null; i: number | null; linked: boolean } }> = ({ live }) => {
 const { x, y, i, linked } = live;
 return (

 Python: x={live.x ?? "-"} · y={live.y ?? "-"} · i={live.i ?? "-"}

);
};

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-2xl">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Папки
 </p>
 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Папки
 </p>
 </div>
 </div>
 </div>
);
};
```

## Универсальное пособие • полный исходный код

```
.....
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { ColoringViz } from "./ColoringViz";
import { ComponentsViz } from "./ComponentsViz";
import { ComplexityClassesViz } from "./ComplexityClasses";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";
```

```
export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}
```

```
/**
```

- \* Единый реестр интерактивных демонстраций.
- \* Используется и для панели под главой, и для всплывающих подсказок.
- \* и для модального окна «Открыть симулятор».

```

.....
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число – увидите, из чего оно сложилось",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: механика → большое дерево → собери сбалансированное",
 hint: "Три вкладки: покадровая механика поворотов в узлах,
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"splay-rotations": {
 title: "Splay: механика → большое дерево → собери сбалансированное",
 hint: "Три вкладки: покадровая механика поворотов в узлах,
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphViz },
"top-sort": { title: "Топологическая сортировка", Component: TopSortViz },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: SCCViz },
"graph-components": {
 title: "Компоненты связности: BFS-разметка",
 hint: "Каждый запуск обхода красит один «остров»; чекбокс «показать все»",
 Component: ComponentsViz,
},
"graph-articulation": { title: "Точки сочленения", Component: GraphViz },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesViz },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerViz },
"planarity-euler-formula": {
 title: "Планарность: K5/K3,3 + раскраска",
 hint: "Слева – непланарные графы с подсветкой пересечений",
 Component: () => {
 <div className="space-y-10">

```

```

.....
 "mst-boruvka": { title: "Борувка (коллективизация)",
 hint: "За фазу компонент становится вдвое меньше.", Component: () => {
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () => {
 "string-z-func": { title: "Z-функция", Component: () => {
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => {
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
 },
 },
 "complexity-classes": {
 title: "Классы сложности: матрёшка $P \subseteq NP \subseteq PSPACE$ ",
 hint: "Кликни класс или задачу; стрелки $\Leftarrow$ показывают включения",
 Component: ComplexityClassesViz,
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCards },
};

```

```
export const getViz = (id?: string): VizEntry | undefined;
```

```

.....
ФАЙЛ 76/92: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
.....

```

```

import React, { useEffect, useState } from 'react';
import { useVizRuntime, vizNumber, vizString } from '../viz';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

```

```

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;

```

```

.....
 8: { E: 9 },
 9: {} ,
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training' | 'search'>('training')

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
 const [textToScan, setTextToScan] = useState<string>('')
 const [currentIndex, setCurrentIndex] = useState<number>(0)
 const [currentNode, setCurrentNode] = useState<number>(0)
 const [isSearchDone, setIsSearchDone] = useState<boolean>(false)
 const [searchLog, setSearchLog] = useState<string>('')
 'Режим поиска: Введите текст и жмите «Следующий шаг»'
);
 const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 })
 const [foundMatches, setFoundMatches] = useState<{ index: number; node: number }>({ index: 0, node: 0 })
 const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0)

 // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
 // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
 const [trainStep, setTrainStep] = useState<number>(0)

 // — Синхронизация с Python-компилятором: s/текст, i/индекс, v/вершина
 // v — текущая вершина автомата. Влияет на режим поиска
 const runtime = useVizRuntime();
 const liveText = vizString(runtime?.variables?.s) ?? '';
 const liveI = vizNumber(runtime?.variables?.i);
 const liveNode = vizNumber(runtime?.variables?.v) ?? 0;
 useEffect(() => {
 if (liveText) {
 setTextToScan(liveText.toUpperCase().replace(/[^A-Z0-9_]/g, ''));
 resetSearchSimulation();
 }
 }, [liveText]);
 useEffect(() => {
 if (liveI !== null) setCurrentIndex(Math.max(0, Math.min(8, liveI)));
 if (liveNode !== null) setCurrentNode(Math.max(0, Math.min(9, liveNode)));
 }, [liveI, liveNode]);
}

```

```

.....
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У русла #${curr} (${WATERFALL_NODES[curr]
 _NODES[nextNode].label}). `;
 setJumpPath({ from: curr, to: nextNode, type: 'no
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char]
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ??
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Л
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${orig
 рыжок по fail-ссылке в #${curr} (${WATERFALL_
 . `;
 setJumpPath({ from: originalCurr, to: nextNode,
 }
 curr = nextNode;
 }

```

```

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

```

```

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) => {
 currentMatches.push({ index: currentInd
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term link;
}

```





```

.....
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. По
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetTrainingNode];

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border" >
 {/* Шапка и переключатель режимов */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех ф
 </p>
 </div>
 </div>
 </div>

 {/* Переключатель режимов */}
 <div className="bg-slate-900 p-1.5 rounded-xl b

```

```

 </p>
 <div className="space-y-1.5 mb-6 overflow
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font
 : 'bg-slate-800 text-slate-400
 }`}
 >
 {step
 {idx === trainStep && <span classNa
 </button>
)}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>

```

```

.....
 {currentTrainInfo.codeLine === 4 && <
ал IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-slate-200 font-bold m
 <Lightbulb className="w-4 h-4 text-amber
 </h5>
 <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
 </div>
</div>
) : {
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3
 <div className="bg-slate-900/60 p-5 rounded-x
 <div>
 <h5 className="text-white font-bold mb-2
 ⚡ Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}

```

```

 ue-900/40 active:scale-[0.98]'
 }` }
 >
 {currentIndex === 0 ? 'Начать пла
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60" >
 <div>
 <h5 className="text-white font-bold mb-3" >
 ❗ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-3xl" >
 <div className={`p-1.5 rounded flex items-center justify-between
border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && ⚙️
инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
border-rose-500 text-rose-300 font-bold`} >
 2. while (curr !== root && !trie[curr].char)
 {activeSearchCodeLine === 2 && 🔍
рыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
border-blue-400 text-blue-200 font-bold`} >
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && 🔍
ем по течению}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
border-emerald-500 text-emerald-300 font-bold`} >
 4. if (is_terminal[curr]) report(curr)
 {activeSearchCodeLine === 4 && 🔍
овладение!}
 </div>
 </div>
 </div>
</div>

```

```

.....
 <span className="w-3 h-0.5 border-t borde

 </div>
</div>

```

```

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-s
 <svg viewBox="0 0 800 520" className="w-full

```

```

 {/* Горизонтальные линии и подписи уровней */}
 {
 { depth: 0, y: 60, label: 'Depth 0 (Корень)' }
 { depth: 1, y: 160, label: 'Depth 1 (H, S)' }
 { depth: 2, y: 260, label: 'Depth 2 (HE, I)' }
 { depth: 3, y: 360, label: 'Depth 3 (HER, I)' }
 { depth: 4, y: 460, label: 'Depth 4 (HERS, I)' }
 }.map((level) => (
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14"
 />
 <text x="32" y={level.y + 4} fill="#94a3b8"
 {level.label}
 </text>
 </g>
)}}

```

```

{/* Рендер прямых переходов (потоков воды) */}
{Object.entries(TRIE_TRANSITIONS).map(([from, to]) => (

```



```

.....
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24' : '12'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : 'white'}
 stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : 'white'}
 strokeWidth={isCurrent || isTargetChild ? 2 : 1}
 className="transition-all duration-300"
 />

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? 12 : 10}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме обхода */}
 {isTargetChild && (
 <g transform={`translate(${node.x}, ${node.y})`}
 <rect x="-35" y="-12" width="70" height="24" fill="white" stroke="black" />
 <text x="0" y="2" fontSize="10" fill="black">
 ИЩЕМ FAIL
 </text>
 </g>
)}

 {/* Индикатор словарных слов */}
 {hasWords && (
 <g transform={`translate(${node.x + 10}, ${node.y + 10})`}
 <circle cx="0" cy="0" r="9" fill="white" stroke="black" />
 <text x="0" y="3" fontSize="10" fill="black">
 *
 </text>
 </g>
)}

```

```

.....
 <div className="mt-6 bg-slate-900/50 p-5 rounded"
 <h5 className="text-white font-bold mb-3 text"
 Улов лосося (Найденные слова
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic"
 Пока ничего не поймали. Запустите шаги сн
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border bord
 -sm font-bold shadow animate-[scaleUp_0.3s_
 >
 <CheckCircle2 className="w-4 h-4 text
 {match.word}
 <span className="text-[10px] bg-emera
 Индекс {match.index}

 </div>
)}}
 </div>
)}
 </div>
)}
</div>
);
};

```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)



```

// Вкладка 2D = своё демо (свой скелет): компилятор п
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode === "2d"
}, [chapterId, mode]);

return (
 <div className="w-full bg-slate-950 rounded-2xl bor
 <div className="flex gap-2 mb-4 overflow-x-auto n
 {{
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
 }.map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 }`}
 >
 {label}
 </button>
)))
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

/* 1D
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[
 []
);
};

```

```

 { /* массив a */ }
 <div className="flex gap-1.5 items-center mb-1" >
 <div className="w-[46px] shrink-0 text-right" >
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 const isDrivenAdd = shownDriven !== null
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'a' })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r.l ? r.l : i > r.r ? r.r : r))}
 className={` ${cellBase} cursor-pointer ${
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2"
 : inCover
 ? "bg-indigo-500 text-white ring-2"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 } `}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
)
 })}
 </div>
 </div>
);
}
}
}
</div>

```

```

 { /* массив P */ }
 <div className="flex gap-1.5 items-center" >
 <div className="w-[46px] shrink-0 text-right" >
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === shownPref;
 const isL = i === range.l;
 const isR = i === range.r + 1;

```

```

.....
 <p className="text-slate-300">
 <b className="text-emerald-400">Компилятор
 i =
 <span className="font-mono text-emerald-300"
 {hasLiveP && <= <span className="font-mono
 Пустой
 </p>
) : hover?.kind === "pref" ? (
 hover.i === 0 ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0<
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[{hover.i}]}
 {" "}
 – это сумма всего, что набежало от начала
 <span className="font-mono text-indigo-300"
 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[{hover.i}]}
 {" "}
 входит во все префиксы, начиная с{" "}
 <span className="font-mono text-emerald-300"
 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку,
 </p>
)
)

```

```

 }
 }
 return p;
}, [n, m]);

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);
 if (r1 === null || c1 === null || r2 === null || c2 === null) return;
 // Python-шаблон (и глава) используют 1-индексные координаты.
 // клетка (1, 1) – левый верхний элемент матрицы. В JS же
 // 0-индексные, поэтому переводим: (1,1) → (0,0).
 const to0 = (v: number, hi: number) => Math.max(0, v - 1);
 setRect({
 r1: to0(r1, n - 1),
 c1: to0(c1, m - 1),
 r2: to0(r2, n - 1),
 c2: to0(c2, m - 1),
 });
}, [vars, n, m]);

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

```

```

.....
{P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1"
 {row.map((v, j) => {
 const liveRow = vizArray(liveS?.[i]);
 const liveValue = liveRow?.[j];
 const blank = hasLiveS && (liveValue === null);
 const shownValue = hasLiveS ? liveValue : blank;
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hover?.j === j;
 const isCompilerCell = liveI === i && liveJ === j;
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner === "B"
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border-slate-800";
 return (
 <div
 key={j}
 onMouseEnter={() => setHover({ i, j })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-${isCompilerCell ? "pointer" : "default"} ${cornerColor} ${isHover ? "z-10" : ""}`}
 title={hasLiveS ? `S[${i}]I[${j}]` : `S[${i}]O[${j}]`}
 >
 {blank ? "." : String(shownValue)}
 {corner && !isHover && (
 <span className="absolute -top-1 -right-1"

```

ht">

ФАЙЛ 78/92: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 830 | РАЗМ

```
import React, { useContext, useEffect, useState } from
import {
 emitVizDemo,
 useVizRuntime,
 vizArray,
 vizNumber,
 vizRecord,
 VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () =>
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 <
 }
}
```



```

: arrID;
const liveTarget =
 liveI !== null && liveJ !== null && Number.isInteger
 ? { i: liveI, j: liveJ }
 : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:{step === 0}"
 type="button">
 <ChevronLeft size={20} />

```



```

 </div>
);
 }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {stID[hover.i]}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " " \}$

 min({arrID.slice(covered.from, covered.to)}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-50">
 <div className="grid grid-cols-{auto_repeat(4)}">
 {/* Headers */}
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-sm">
 2^0 (L=1)
 </div>
 </div>
 </div>

```

```

.....
 isSrc1 = i === liveTarget.i && j ===
 isSrc2 = i === liveTarget.i + (1 <<
 } else if (!compilerLinked && step > 1) {
 const currentStep = computeSteps[step]
 isActive = currentStep.i === i && currentStep.j === j
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 = true
 if (currentStep.i + (1 << (currentStep.j - j)) === i) isSrc2 = true
 }
 }

 // Готовые вычисленные уровни демо не нужны
 // уровень j=0 всегда берём из уже существующих
 const shownValue = compilerLinked ? compilerLinked : undefined
 const displayValue =
 shownValue === undefined || shownValue === null
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

 return (
 <div key={j} className="relative flex flex-col items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,300px)] h-[clamp(30px,9vw,300px)]"
) : (
 <motion.div
 onMouseEnter={() => (compilerLinked ? setCompilerLinked(true) : null)}
 onMouseLeave={() => setCompilerLinked(false)}
 onClick={() => (compilerLinked ? setCompilerLinked(false) : null)}
 >
 {displayValue}
 </motion.div>
)}
 >
 {displayValue}
 </div>
)
}

initial={{ opacity: 0, scale: 1 }}
animate={{
 // При st = [] каркас уже существует
 opacity: compilerLinked ? 1 : 0,
 scale: compilerLinked || isSrc1 || isSrc2 ? 1 : 1,
}}
className={`w-[clamp(30px,9vw,300px)] h-[clamp(30px,9vw,300px)]`}

```

```

.....
 stroke={isSrc1 ? "#10b981"
 strokeWidth="3"
 />
 </motion.svg>
)}
</div>
);
}}}
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
 {compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">

 i =
 – активна ячейка <span className="font-mono
 {(hasLiveTable || hasLiveSource) && <> = <sp
 ?.[liveTarget.j] ?? (liveTarget.j === 0 ? l
 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-
 ST[{c.i}][{c.j} - 1]}

 ,

```

## Универсальное пособие • полный исходный код

```
.....
const codeCell = liveR !== null && liveC !== null ? {
const compilerLinked = codeCell !== null || liveK !==
const shownK = liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;
```

```
// Уровень и клетка берутся прямо из k/г/с; номер строки
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);
```

```
const L = 1 << shownK;
const activeCell = codeCell || locked || hover;
```

```
return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scroll-smooth">
 <h3 className="text-xl font-bold text-indigo-400">
 <p className="text-sm text-slate-400 mb-6 text-left">
 Для наглядности показываем только <b className="text-rose-300">

 иц!
 </p>
 </h3>
 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null);
 className={`px-4 py-2 rounded-md text-sm
 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
```

```

.....
 : 'bg-slate-800/50 text-slate-100';
 let zIndex = 'z-0';

 const isActive = !!activeCell &&
 !l.c + L) && ((r - activeCell.r) % (1 << shownK) === 0);

 if (isActive) {
 if (isCurr) {
 highlightClass = 'bg-indigo-500';
 } else if (locked && locked.r === r && locked.c === c) {
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (shownK - 1);
 const isTop = r < activeCell.r;
 const isLeft = c < activeCell.c;

 if (isTop && isLeft) {
 highlightClass = 'bg-rose-500';
 } else if (isTop && !isLeft) {
 highlightClass = 'bg-blue-500';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emerald-500';
 } else {
 highlightClass = 'bg-amber-500';
 }
 zIndex = 'z-10';
 }
 }

 return (
 <div
 key={idx}
 onMouseEnter={() => { if(isActive) {
 onMouseLeave={() => { if(isActive) {

```

```

 </div>
 </div>
) : (
 <div>
 <p className="mb-4">При 1
 драты имеют размер 1
 <div className="bg-[#0a0f1e] rounded-xl p-4">
 // Те
 int<
 </div>
 <p className="mt-6 italic text-indigo-400">
 <Play size={14} className="fill-indigo-400">
 a.
 </p>
 </div>
)}
</div>
) : (
 <div className="flex flex-col gap-4 animate-pulse">
 <div className="bg-[#0a0f1e] rounded-xl p-4 shadow-inner overflow-x-auto">
 // Те
 int<
 </div>
 // Ка
 ST<
 1
 e="text-amber-300">1], <span
 1
 ext-amber-300">1]{k-<span className="
 pan>

 1
 ="text-amber-300">1]{k-<span classNa
 span>

 1
 white">c + pL]{k-<span className="te
 ame="text-slate-500">// Правый Низ

 {''}}));

```

```

 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}())}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
)}
</div>
)}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

```

```

.....
 const isInQuery = r >= r1 && r

 return (
 <div key={idx} className={\
y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>
)
 }}}

 <div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
 />

 /* Показываем 4 угла в самой матрице
 <div className="absolute pointer-e
 shadow-[0_0_10px_#f43f5e]" style={{ top: `
 lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
 <div className="absolute pointer-e
 shadow-[0_0_10px_#3b82f6]" style={{ top: `
 width: `calc(`${(Ly / 8) * 100}% - 16px)`,
 <div className="absolute pointer-e
 ld-400 shadow-[0_0_10px_#10b981]" style={{
 8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e
 00 shadow-[0_0_10px_#f59e0b]" style={{ top:
 00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
 </div>

```



```

.....
 <div className="bg-[#0a0fle] rounded-xl p-
text-slate-300 shadow-inner overflow-x-auto"
 <div className="absolute right-3 top-3"
 // 1.
 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<
 В
запроса)

 r1]]<span className="text-white bg-r

 во,
чтобы правый край уперся в c2)

 r1]]<span className="text-white bg-b

 рх,
чтобы нижний край уперся в r2)

 r2 - Lx + 1]]<span className="

 и влево
от c2)

 ">r2 - Lx + 1]]<span className="text
 {'}}'));
 </div>

 <div className="w-full flex justify-centr

```



```

body.classList.remove('theme-dark', 'theme-
body.classList.add('theme-' + theme);

// Just some visual feedback on buttons
document.querySelectorAll('[id^="theme-"]')
 btn.classList.remove('outline', 'outline
});

if (theme === 'dark') document.getElementById
if (theme === 'light') document.getElementById
if (theme === 'terminal') document.getElemen
);

document.getElementById('btn-theme-dark').c
t-white transition-colors' : 'px-3 py-1 rou
n-colors';
document.getElementById('btn-theme-light').c
ext-white transition-colors' : 'px-3 py-1 r
ion-colors';
document.getElementById('btn-theme-notebook
-600 text-white transition-colors' : 'px-3
ransition-colors';
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highligh
document.addEventListener('DOMContentLoaded', (
 const mobileMenuBtn = document.getElementById
 const mobileDrawer = document.getElementById
 const closeDrawerBtn = document.getElementById
 const drawerOverlay = document.getElementById

function toggleDrawer() {
 const isClosed = mobileDrawer.classList
 if (isClosed) {
 mobileDrawer.classList.remove("-tra

```

```
.....
 threshold: 0
 };

 const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link loc
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href'
 link.classList.add('bg-
 } else {
 link.classList.remove('
 }
 }));
 }
 }));
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>
```

.....  
ФАЙЛ 80/92: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1000

```
.....
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
```

```
.....
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
 display: none;
}

mjax-container {
 max-width: 100% !important;
}
mjax-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjax-container[display="true"]::-webkit-scrollbar
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::aft

 .uno-equation .text-2xl { font-size: 1.2rem
 .uno-equation.gap-4 { gap: 0.5rem !important

 /* Allow horizontal scroll for equations in
```

```
.....
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-color:
body.theme-light #top-controls { background-color:

body.theme-notebook {
 background-color: #fcf8eb !important; /* light beige */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg,
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7) !important;
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
body.theme-notebook .text-slate-400 { color: #546e7a !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-500 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
```

```
.....
<svg width="0" height="0" class="absolute">
```

```
 <defs>
```

```
 <marker id="arrow-yellow" markerWidth="6" m
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d
```

```
 </marker>
```

```
 <marker id="arrow-orange" markerWidth="6" m
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d
```

```
 </marker>
```

```
 <marker id="arrow-pink" markerWidth="6" mar
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1
```

```
 </marker>
```

```
 <marker id="arrow-cyan" markerWidth="6" mar
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf
```

```
 </marker>
```

```
 <marker id="arrow-indigo" markerWidth="6" m
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8
```

```
 </marker>
```

```
 <marker id="arrow-lime" markerWidth="6" mar
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d
```

```
 </marker>
```

```
 <marker id="arrow-green" markerWidth="6" ma
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade88
```

```
 </marker>
```

```
 <!-- ЛЕГО БЛОКИ -->
```

```
 <g id="lego-yellow">
```

```
 <rect x="0" y="0" width="40" height="20
```

```
 <rect x="6" y="-5" width="8" height="6"
```

```
 <rect x="26" y="-5" width="8" height="6"
```

```
 <line x1="2" y1="2" x2="38" y2="2" stro
```

```
 </g>
```

```
 <g id="lego-red">
```

```
 <rect x="0" y="0" width="40" height="20
```

```
 <rect x="6" y="-5" width="8" height="6"
```

```
 <rect x="26" y="-5" width="8" height="6"
```

```
 <line x1="2" y1="2" x2="38" y2="2" stro
```

```
 </g>
```

```
 <g id="lego-rem">
```

```
.....
<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-
```

```
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-
```

```
<!-- Левое меню (Оглавление) -->
```

```
<aside id="mobile-drawer" class="fixed inset-y-0
 nslate-x-full transition-transform duration-300
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
 <div class="h-full flex flex-col p-6 overflow-y-auto
 lg:rounded-2xl lg:border lg:border-slate-700
 <div class="flex justify-between items-center
 <h3 class="font-black text-transparent bg-clip-text
 ider text-sm">
```

```
 Содержание
```

```
 </h3>
```

```
 <button id="close-drawer-btn" class="text-slate-500
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6
 <path stroke-linecap="round" stroke-width="2" d="M6 18
 </svg>
 </button>
```

```
 </div>
```

```
 <nav class="space-y-3 text-sm font-medium">
```

```
 <details class="group" open>
```

```
 <summary class="list-none cursor-pointer">
```

```

 1. Алгебраические структуры
```

```

```

```
 </summary>
```

```
 <div class="pl-6 space-y-2 text-sm">
```

```

```

```

```

```

```

```

```

```
 </div>
```

```
 </details>
```



```
.....
 <a href="#q4-def" class="block h
 <a href="#q4-moivre" class="blo
 <a href="#q4-root" class="block
 <a href="#q4-count" class="bloc
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-5" class:
er-fuchsia-500 pl-3 font-bold text-fuchsia-1
 5. Группа корней из единицы

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q5-def" class="block i
 <a href="#q5-group" class="bloc
 <a href="#q5-prim" class="block
 <a href="#q5-crit" class="block
 <a href="#q5-cyclic" class="blo
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-6" class:
er-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q6-1" class="block ho
 <a href="#q6-2" class="block ho
 <a href="#q6-3" class="block ho
 <a href="#q6-4" class="block ho
 <a href="#q6-5" class="block ho
 <a href="#q6-6" class="block ho
```

```
.....
 <summary class="list-none cursor-pointer" style="background-color: #d9f2d9; padding: 5px; border: 1px solid #ccc;">

 9. Кольцо многочленов

 </summary>
 <div class="pl-6 space-y-2 text-left" style="padding-left: 10px;">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer" style="background-color: #fce4ec; padding: 5px; border: 1px solid #ccc;">

 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text-left" style="padding-left: 10px;">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer" style="background-color: #e0f7fa; padding: 5px; border: 1px solid #ccc;">

 11. НОД и НОК многочленов

 </summary>
```

```
.....
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
 14. Кратность корня. Корень

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 deg)
 K
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критерий

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer-red-500 pl-3 font-bold text-red-400">
```

△ Ликбез: Алгоритмы выживания

</a>

</nav>

<script>

```
document.querySelectorAll('details.' +
 detail.addEventListener('toggle', () => {
 if (detail.open) {
 document.querySelectorAll('details.' +
 if (other !== detail) {
 other.removeAttribute('open');
 }
 });
 }
 });
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 "#q5", div[id^="q6"], div[id^="q7"], div[id^="q8"],
 v[id^="q13"], div[id^="q14"], div[id^="q15"]
);
 let isScrolling = false;

 document.querySelectorAll('.grow').forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 200);
 });
 });

 const observer = new IntersectionObserver(entries => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });
 });
```

```
.....
modal.style.top = '0';
modal.style.left = '0';
modal.style.width = '100vw';
modal.style.height = '100vh';
modal.style.backgroundColor = 'white';
modal.style.zIndex = '99999';
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a202c';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = 'white';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузера

ние веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = 'black';
textarea.style.padding = '10px';
```

```

 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(closeB);
 }
 btnContainer.appendChild(closeB);
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal);
 }

 function downloadMD() {
 let clone = document.querySelector('#md-content');

 // Smart markdown parsing based on DOM
 // Smart markdown parsing based on DOM
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
 caption = el.nextElementSibling.alt;
 }
 el.textContent = '\n\n[Изображение: ' + caption + ']';
 });
 });
 });
 });
 });

 clone.querySelectorAll('.img-caption').forEach(el => {
 let text = clone.innerText;
 text = text.replace(/\n{3,}/g, '\n\n');

 if (window !== window.top) {
 fallbackModal(text, 'Markdown');
 return;
 }

 const blob = new Blob([text], {

```

```

body.bg-white .text-slate-500 {
body.bg-white {class*="text-slate-500"}

/* Specific backgrounds
body.bg-white aside > div {
body.bg-white {class*="text-slate-500"}

nt; }

body.bg-white {class*="text-slate-500"}

nt; }

body.bg-white {class*="text-slate-500"}
body.bg-white .analogy-block {
body.bg-white .img-placeholder {
body.bg-white .info-block {

body.bg-white .analogy-block {
or: #94a3b8 !important; }
body.bg-white .img-caption {

/* Tweak card blocks */
body.bg-white .card-red {

body.bg-white aside {
#334155 !important; }
body.bg-white .text-white {

/* Ensure active links are visible
body.bg-white aside a.text-white {
body.bg-white aside a.text-white {
body.bg-white aside a.text-white {
body.bg-white aside a.text-white {
body.bg-white aside a.text-white {
body.bg-white aside a.text-white {
body.bg-white aside a:has(img) {

body.bg-white table tr {
body.bg-white table tr {
body.bg-white .text-teal {
body.bg-white .text-rose {

```

```

.....
 <div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-ce
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-wh
">* </button>
 <button id="theme-terminal"
-center justify-center text-green-400 hover
терминала"> </button>
 </div>
 </div>
 </div>
 </div>

```



```
.....
 21.
 22.

</div>
</header>

<!-- for how to set and how to make the container container WTF CONTAINER for how to set and how
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4
 <div class="grid grid-cols-3 gap-4
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slat

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slat

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slat

 <!-- Row 2: Базовые операции / Евклид
```

```

.....
 <div class="text-xs text-orange"
 <div class="font-bold text-slate-700"

 <a href="#question-5" onclick="setMainQuestion(5)"
-4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setMainQuestion(13)"
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange"
 <div class="font-bold text-slate-700"

 <a href="#question-4" onclick="setMainQuestion(4)"
-4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setMainQuestion(14)"
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange"
 <div class="font-bold text-slate-700"

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setMainQuestion(15)"
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange"
 <div class="font-bold text-slate-700"


```

```
.....
 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
 <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

 <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500">
 <div class="font-bold text-slate-900">

</div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 rounded-r
 <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
 <path stroke-linecap="round" stroke-width="2"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-l-4 border-red-500/30
</div>

 <div class="prose prose-invert max-w-3xl">
 <p class="text-lg">
 Хватит тупить. Доказательств
 </p>
 <p>есть для того, чтобы жать на кнопки. Вот жес
 </p>

 <h2 class="text-2xl font-bold border-l-4 border-red-500/30
(Меметика)</h2>

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-blue-400">
 <p class="text-sm text-slate-400">
```

```
.....
 <h3 class="text-xl text-yellow-400">
 <p class="text-sm text-slate-500">
 <ul class="list-disc pl-5 space-y-2">

валось.

Просто перепиши формулу с буквой k .

усок k , замени его на жареного карася из

</div>

<!-- Двойная делимость -->
<div class="bg-slate-800 p-6">
 <h3 class="text-xl text-orange-400">
 <p class="text-sm text-slate-500">
 <ul class="list-disc pl-5 space-y-2">

вверх по уравнениям).

рху вниз, показывая, что $\Delta \dot{r}_i$

х. Твой d больше.

</div>

<h2 class="text-2xl font-bold border-top-2px solid black padding-top-10px">
оритмы экзекуции)</h2>

<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-500">
 <p class="text-xs font-mono text-slate-400">
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```

.....
 </div>

 <li class="bg-slate-800/50"

 <div>
 <h4 class="text-cyan"
 <p class="text-sm t
х двух, потом последних.

 <code class="text-x
 </div>

 <li class="bg-slate-800/50"

 <div>
 <h4 class="text-cyan"
 <p class="text-sm t
есть сложение – есть ноль. Если есть умноже
 <code class="text-x
 </div>

 <li class="bg-slate-800/50"

 <div>
 <h4 class="text-cyan"
 <p class="text-sm t
авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>

</div>
</section>
</div>

<div id="questions-container">

```

```
.....
* в конце документа.
*/

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a,term,hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding
.export-head { border-bottom: 1px solid #1e293b; padding
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; fon
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; fo
```

```
.....
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
</section>`;
}

function wrap(opts: { title: string; subtitle: string;
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
```

```

.....
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^\\s*(\\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением. */
export async function downloadBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}> ${c.title}</a>`
 .join("\\n");

```



## Универсальное пособие • полный исходный код

```
.....
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
```

```
.....
};
dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
dump("НЕТ 2D (аналогия есть)", gapViz);
console.log(`Пропущенные номера билетов 1-24: ${missi
console.log(`Визуализаторы без главы: ${orphanViz.joi
}
```

.....  
ФАЙЛ 84/92: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
.....
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[.]*+?^${}()|[\]\[\]]/g, "\\
const re = new RegExp(
 `(?!([\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
```

Универсальное пособие • полный исходный код

```
.....
 });
}
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

.....  
ФАЙЛ 85/92: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 28 | РАЗМ

```
.....

/**
 * Оркестратор: бандлит TSX-генератор и запускает его.
 * Использование: node scripts/export/build-guide-pdf.m
 */
import { build } from "esbuild";
import { pathToFileURL } from "node:url";
import path from "node:path";
import fs from "node:fs";

const root = path.resolve(import.meta.dirname, "../..");
const outfile = path.join(root, "tmp/guide-pdf-bundle.c

await build({
 entryPoints: [path.join(root, "scripts/export/guide-p
 outfile,
 bundle: true,
 platform: "node",
 format: "cjs",
 target: "node20",
 jsx: "automatic",
 loader: { ".jpg": "dataurl", ".png": "dataurl", ".css
 external: ["pdfkit", "svg-to-pdfkit", "pdfkit/js/pdfk
 logLevel: "warning",
});

const outArg = process.argv[2];
```

```
.....
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
.....
```

Универсальное пособие • полный исходный код

```
.....
};

const categoryOf = (rel) => CATEGORIES.find(({rel}) => rel === rel);

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel[0] === d || rel[0] === d + path.sep)) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 }
}
```

## Универсальное пособие • полный исходный код

```
function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { time: true });
 const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОММЕНТАРИИ)");
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:          ${files.length}`);
 push(`Суммарный объём кода:              ${humanSize(totalBytes)}`);
 push(`Глав/билетов в пособии:            ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
 push(HR);
 chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
 push(HR);
 let current = null;
 files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 }
 });
}
```

## Универсальное пособие • полный исходный код

```
.....
 console.log(`      файлов:  ${files.length}`);
 console.log(`      глав:    ${chapters.length}`);
 console.log(`      строк:   ${txt.split("\n").length}`);
 console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}
```

```
main();
```

```
.....
ФАЙЛ 88/92: scripts/export/common.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ
```

```
.....
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
```

```
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";
```

```
export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
```

```
/** Куда кладем финальные артефакты (эта папка отдаётся клиенту)
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают)
export const PAGES_DIR = path.join(ROOT, "tmp", "export");
```

```
export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");
```

```
/** Параметры вёрстки страницы (подобраны под A4 @150dpi)
export const PAGE = {
```

Универсальное пособие • полный исходный код

```
.....
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick";
 };
}
```

ФАЙЛ 89/92: scripts/export/guide-pdf-entry.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 673 | РАЗМЕР: 10.5 КБ

```
.....
/**
 * Автосборка PDF-конспекта `guide_all_topics.pdf`.
 *
 * Что внутри:
 * • весь ТЕКСТ страниц пособия (больше текста, чем HTML-версия);
 * • картинки визуализаций – React-компоненты рендерят статический
 * (renderToStaticMarkup), из разметки берутся SVG-картинки;
 * • НИКАКОГО кода компилятора и интерфейсных кнопок.
 *
 * Результат: public/export/guide_all_topics.pdf, ~40–70 КБ.
 */
```

```
import * as fs from "node:fs";
import * as path from "node:path";
import { createElement } from "react";
import { renderToStaticMarkup } from "react-dom/server";
import PDFDocument from "pdfkit";
// @ts-ignore - у пакета нет типов
import SVGtoPDF from "svg-to-pdfkit/source.js";
import { chapters } from "../../src/data/content";
import { VizChapterContext } from "../../src/data/vizState";
import { VIZ_REGISTRY, getViz } from "../../src/components/vizRegistry";
```

```
// ----- шрифты -----
```

```
const FONT_DIRS = ["/usr/share/fonts/truetype/dejavu",
function findFont(file: string): string | null {
 for (const dir of FONT_DIRS) {
```



```

.....
 html
 .replace(/<(script|style)[\s\S]*?<\/\1>/gi, " ")
 .replace(/<br\s*\/?>/gi, "\n")
 .replace(/<\/(p|div|li|h[1-6]|tr|summary)>/gi, "\n")
 .replace(/<[>]+>/g, " ")
)
 .replace(/[\t]+/g, " ")
 .replace(/ *\n */g, "\n")
 .replace(/\n{2,}/g, "\n")
 .trim();
 }

/** Найти закрывающий тег с учётом вложенности однотипных
function findClose(html: string, openEnd: number, tag: string): number {
 const re = new RegExp(`<\/?${tag}([\\s[>]*)?>`, "gi");
 re.lastIndex = openEnd;
 let depth = 1;
 let m: RegExpExecArray | null;
 while ((m = re.exec(html))) {
 depth += m[0][1] === "/" ? -1 : 1;
 if (depth === 0) return m.index;
 }
 return html.length;
}

function findCloseAny(html: string, openEnd: number, tag: string): { end: number; inner: string } {
 const close = html.toLowerCase().indexOf(`<\/${tag}`);
 if (close < 0) return { end: html.length, inner: html };
 const inner = html.slice(openEnd, close);
 const end = html.indexOf(">", close) + 1;
 return { end, inner };
}

/** Разобрать HTML главы на учебные блоки (текст → 90%,
export function htmlToBlocks(html: string): Block[] {
 const blocks: Block[] = [];
 let i = 0;
 const skipTo = ["<pre", "<h2", "<h3", "<h4", "<summary"]

```

```

.....
 const text = stripTags(inner);
 if (text) blocks.push({ kind: "p", text: "» " + text });
 // содержимое details выводим тоже: оно идёт сразу
 const detEnd = html.toLowerCase().indexOf("</details>");
 if (detEnd >= 0) {
 blocks.push(...htmlToBlocks(html.slice(end, detEnd)));
 i = detEnd + "</details>".length;
 } else i = end;
 continue;
}
if (lower.startsWith("<li", at) || (liM && at === 1)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("", at);
 const text = stripTags(html.slice(openEnd, close));
 if (text) blocks.push({ kind: "li", text });
 i = close < 0 ? html.length : close + 5;
 continue;
}
if (lower.startsWith("<tr", at)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("</tr>", at);
 const rowHtml = html.slice(openEnd, close < 0 ? html.length : close);
 const cells: string[] = [];
 const tdRe = /<(t[dh])(\s{^}*)?>([\s\S]*?)<\/\1>/g;
 let tm: RegExpExecArray | null;
 while ((tm = tdRe.exec(rowHtml))) cells.push(stripTags(tm[2]));
 if (cells.some(Boolean)) blocks.push({ kind: "table", cells });
 i = close < 0 ? html.length : close + 5;
 continue;
}
// <p>
const openEnd = html.indexOf(">", at) + 1;
const close = html.toLowerCase().indexOf("</p>", at);
const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
if (text) blocks.push({ kind: "p", text });
i = close < 0 ? html.length : close + 4;
}
return blocks;

```

```

.....
 // выкинуть элементы с tailwind-классами без инла
 svgs.push(svg);
 idx = pos;
}
const imgRe = /<img[^>]*src="(data:image\/(png|jpeg
let im: RegExpExecArray | null;
while ((im = imgRe.exec(html))) images.push(im[1]);
const grids = extractDivGrids(html);
return { svgs, images, grids };
} catch (e) {
 return { svgs: [], images: [], grids: [], error: St
}
}

/**
 * Визуализации бывают и без SVG: матрицы из div-ячеек
 * префиксные суммы). Собираем такие «таблицы» в текст
 */
function extractDivGrids(html: string): string[][] {
 const grids: string[][] = [];
 const rows: string[] = [];
 const rowMark = /<div class="[^"]*(?:flex gap-1\.5|gr
 let m: RegExpExecArray | null;
 while ((m = rowMark.exec(html))) {
 // сбалансированно находим конец строки-контейнера
 let depth = 1;
 let pos = m.index + m[0].length;
 while (depth > 0 && pos < html.length) {
 const no = html.indexOf("<div", pos);
 const nc = html.indexOf("</div>", pos);
 if (nc < 0) break;
 if (no >= 0 && no < nc) { depth++; pos = no + 4;
 else { depth--; pos = nc + 6; }
 }
 const inner = html.slice(m.index + m[0].length, pos
 // теги → разделители: порядок значений сохраняется
 const cells = decodeEntities(inner.replace(/<[^>]+>
 .split(/\s+/))

```

```

 this.bodyFont();
 }
 setupFonts() {
 if (F_REG) this.doc.registerFont("Body", F_REG);
 if (F_BOLD) this.doc.registerFont("BodyBold", F_BOLD);
 if (F_MONO) this.doc.registerFont("Mono", F_MONO);
 if (F_MONO) this.doc.registerFont("MonoBold", F_MONO);
 }
 bodyFont() { this.doc.font(F_REG ? "Body" : "Helvetica"); }
 boldFont() { this.doc.font(F_BOLD ? "BodyBold" : "Helvetica-Bold"); }
 monoFont() { this.doc.font(F_MONO ? "Mono" : "Courier"); }

 get y() { return this.doc.y; }
 set y(v: number) { this.doc.y = v; }
 get pageBottom() { return A4[1] - M.bottom; }
 ensureSpace(need: number) {
 if (this.y + need > this.pageBottom) this.doc.addPage();
 }
 footerAll(skipCover = true) {
 const range = this.doc.bufferedPageRange();
 const oldBottom = this.doc.page.margins.bottom;
 for (let p = range.start; p < range.start + range.count; p++) {
 if (skipCover && p === range.start) continue;
 this.doc.switchToPage(p);
 this.doc.page.margins.bottom = 0;
 this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(
 .text(String(p + 1), 0, A4[1] - 30, { width: A4[1] - 30 }
);
 this.doc.page.margins.bottom = oldBottom;
 }
 }
}

```

```

const CH_LINE = 13.4; // межстрочный в теле
const BODY_SIZE = 9.6;

```

```

function drawTextBlock(d: Doc, text: string, opts: { size: number; align:
 boolean } = {}) {
 const size = opts.size ?? BODY_SIZE;

```

```

.....
 if (fit <= 0 && from === 0 && lines.length > 1) { d
 fit = Math.max(fit, 1);
 const to = Math.min(lines.length, from + fit);
 drawPortion(from, to, yTop);
 from = to;
 if (from < lines.length) { d.doc.addPage(); yTop = 1
 }
 d.y = yTop + Math.min(lines.length, Math.floor((d.pag
 d.doc.moveDown(3));
}

function drawTableBlock(d: Doc, rows: string[]) {
 const size = 8.4;
 d.ensureSpace(size * 3);
 const text = rows.map((c) => c.replace(/\s*\n\s*/g, "
 d.doc.fontSize(size).fillColor("#374151").text(text, 1
 d.doc.moveDown(2.2);
}

function drawGrid(d: Doc, rows: string[]) {
 const size = 8;
 const lead = size * 1.6;
 d.ensureSpace(rows.length * lead + 16);
 const top = d.y;
 d.monoFont().fontSize(size).fillColor("#111827");
 const cellW = Math.min(
 34,
 (CONTENT_W - 16) / Math.max(...rows.map((r) => r.sp
));
 rows.forEach((row, ri) => {
 const cells = row.split("\u2009");
 cells.forEach((cval, ci) => {
 const x = M.left + 6 + ci * cellW;
 const y = top + 4 + ri * lead;
 d.doc.save().rect(x - 2, y - 2, cellW - 4, lead -
 d.doc.text(cval, x + 2, y + lead * 0.12, { width:
 });
 });
});

```

```

.....
 const buf = Buffer.from(b64, "base64");
 const maxW = CONTENT_W * 0.82;
 const avail = d.pageBottom - d.y;
 if (avail < 200) d.doc.addPage();
 try {
 d.doc.image(buf, M.left + (CONTENT_W - maxW) / 2, d
 } catch { return; }
 d.y = d.y + 10 + maxW * 0.6; // примерная высота фото
 d.doc.moveDown(2);
 }

// ----- сборка -----

const FRONT_TOC_LINES = 44; // строк содержания на стра

function tocEntries() {
 return chapters.map((c) => ({ id: c.id, title: c.titl
})

function tocPagesNeeded(): number {
 return Math.max(1, Math.ceil(tocEntries().length / FR
})

interface BuildOpts {
 outPath: string;
 record: boolean;
 /** id → стартовая страница (для второго прохода) */
 pageMap?: Map<string, number>;
 frontPages: number;
}

function buildOne(opts: BuildOpts): Stats {
 return new Promise<Stats>((resolve, reject) => {
 const stream = opts.record ? fs.createWriteStream("
 const d = new Doc(stream as unknown as fs.WriteStre
 const doc = d.doc;

 const finish = (stats: Stats) => {

```

```

 doc.addPage();
 drawToHeader();
 }
 const pageNo2 = (opts.pageMap?.get(ch.id) ?? 0);
 const isTopic = /^\\d/.test(ch.title);
 doc.fontSize(isTopic ? 10.5 : 10).font(isTopic ?
 .fillColor(isTopic ? "#1f2937" : "#6b7280"));
 // заголовок + точечная отбивка + номер страниц
 const title = ch.title.length > 86 ? ch.title.s
 const w = CONTENT_W - 40;
 const tw = doc.widthOfString(title);
 const pw = doc.widthOfString(String(pageNo2));
 doc.text(title, M.left, d.y, { width: w, lineBr
 const dots = ".".repeat(Math.max(3, Math.floor(
 doc.fillColor("#d1d5db").text(dots, M.left + tw
 doc.fillColor("#111827").text(String(pageNo2),
 d.y += 16.2;
 line++;
 }));
 // добиваем до расчётного числа страниц содержания
 const used = Math.ceil(entries.length / FRONT_TOC
 for (let p = used; p < opts.frontPages - 1; p++)
 doc.addPage();
 if (!opts.record) console.log(`TOC: ${entries.leng
 }

 // — главы —
 const stats: Stats = { chapterStart: new Map(), tot

 chapters.forEach((ch, ci) => {
 const startPageOf = () => {
 // текущая страница = pages (0-based) + 1
 return doc.bufferedPageRange().start + doc.buff
 };
 if (ci > 0) doc.addPage();
 const pageNow = opts.record ? doc.bufferedPageRan
 d.chapterStart.set(ch.id, pageNow);
 if (!opts.record && ci < 3) console.log(`глава ${

```

```

.....
 const art = renderVizArt(ch.id);
 if (opts.record) {
 const n = (art?.svgs.length ?? 0) + (art?.images.length ?? 0);
 console.log(`viz ${ch.id}: svg=${art?.svgs.length} images=${art?.images.length}
 for ? " ОШИБКА: " + art.error : ""`);
 if (n !== 0) console.log(` Δ ${ch.id}: визуализация`);
 }
 if (art && (art.svgs.length || art.images.length)) {
 d.ensureSpace(60);
 doc.moveDown(0.6);
 doc.fontSize(10).font(F_BOLD ? "BodyBold" : "Body");
 d.text("■ Визуализация: " + vizEntry.title, 0, 0);
 d.y = doc.y + 6;
 let drawn = 0;
 for (const img of art.images) {
 if (drawn >= 4) break;
 drawImage(d, img);
 drawn++;
 }
 for (const svg of art.svgs) {
 if (drawn >= 5) break;
 drawSvg(d, svg);
 drawn++;
 }
 for (const grid of art.grids) {
 if (drawn >= 7) break;
 drawGrid(d, grid);
 drawn++;
 }
 } else if (art?.error) {
 // SSR упал – рисуем текстовую заглушку, PDF не падает
 }
}
});

```

```

stats.chapterStart = d.chapterStart;
stats.total = doc.bufferedPageRange().count;
finish(stats);

```



ФАЙЛ 90/92: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ: 3.5 КБ

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT,
 PAGES_DIR,
 TXT_PATH,
 PAGE,
 ensureDir,
 humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/ /g, " ");
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min(expanded.match(/^ /g).length, PAGE.cols));
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent.length;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}
```

```

.....
const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`      отрендерено ${done}\n`);
 }
 }
}

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(", ")}`);

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);

```

```
permissions:
 contents: read
 pages: write
 id-token: write

concurrency:
 group: pages
 cancel-in-progress: true

jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
```

## Универсальное пособие • полный исходный код

```
.....
исходный код → full_code_all_pages.txt (script)
#
└───────────→ page-0001.png ... page-NNNN.png (
#
└───────────→ full_code_all_pages.pdf
#
└───────────→ guide_all_topics.pdf
PDF-конспект: текст всех тем + с
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.
```

```
on:
 push:
 branches:
 - "*"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бесконеч
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолч
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
```

- ```
run: npm run export:png
```
- name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Step 4 – PDF-конспект (текст тем + визуал)
run: npm run export:guide
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full\_code\_all\_pages`"
 echo "| \`\${public}/export/full\_code\_all\_pages`"
 echo "| \`\${public}/export/guide\_all\_topics.pdf`"
 echo "Страниц в конспекте: \$(node -e "console.log(require('fs').readdirSync('tmp/export-pages').length)")"
 echo "| PNG-страниц | \${ls tmp/export-pages | wc -l}"
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 public/export/guide_all_topics.pdf
 if-no-files-found: error
 retention-days: 30
 - name: Upload artifacts (PNG-страницы)